

说在前面：

现在**拿到offer超级难**，甚至连面试电话，一个都搞不到。

尼恩的技术社群中（50+），很多小伙伴凭借“左手云原生+右手大数据”的绝活，拿到了offer，并且是非常优质的offer，据说年终奖都足足18个月。

而云原生的核心组件是 Docker + K8S，但是 Docker 又很难。在这里，尼恩从架构师视角出发，Docker + K8S 核心原理做一个宏观的介绍。

由于内容确实太多，所以写两个pdf电子书，并且后续会持续升级：

(1) 《 Docker 学习圣经 》PDF

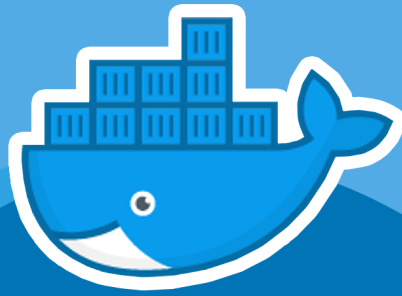
(2) 《 K8S 学习圣经 》PDF

带大家穿透Docker + K8S，实现Docker + K8S 自由，让大家不迷路。

本书《 Docker 学习圣经 》PDF的 V1版本，后面会持续迭代和升级。供后面的小伙伴参考，提升大家的 3高 架构、设计、开发水平。

注：本文以 PDF 持续更新，最新尼恩 架构笔记、面试题 的PDF文件，请从这里获取：[码云](#)

《 Docker 学习圣经 》PDF 封面



Docker学习圣经

尼恩 编著

一本书实现docker自由

疯狂创客圈Java高并发社区出版

CSDN @40岁资深老架构师尼恩

Docker基础

作为大神或者准架构师/架构师，一定要了解一下docker的底层原理。

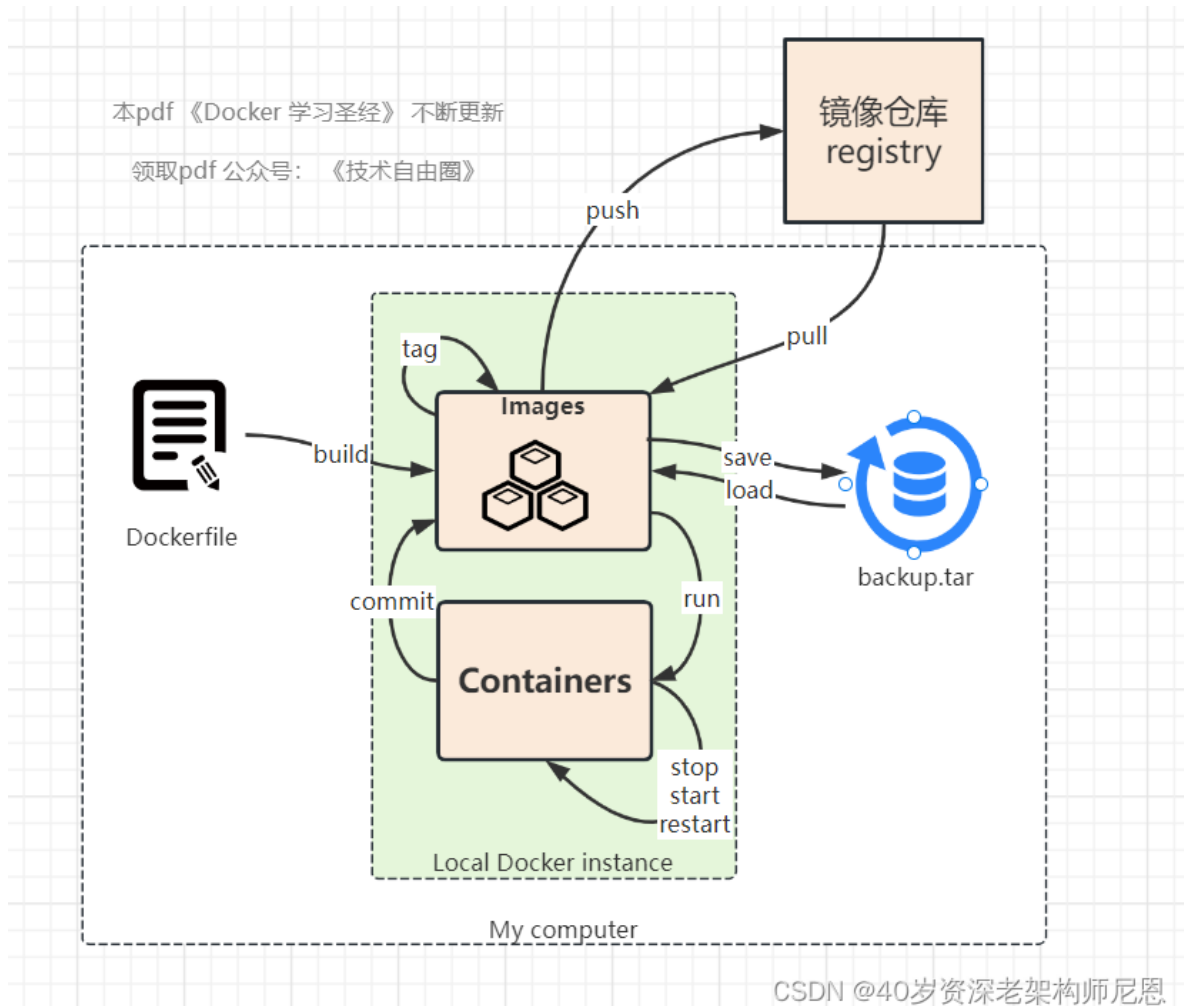
首先还是简单，说明一下Docker 巨大的价值

Docker 巨大的价值



Docker 是一个开源的应用容器引擎，基于 Go 语言开发。

Docker 遵从 Apache2.0 协议开源。



Docker 的本质:

先来说说Docker 的本质

Docker 可以让开发者打包他们的应用以及依赖包到一个轻量级、可移植的容器中，然后发布到任何流行的 Linux 机器上，实现轻量级虚拟化。

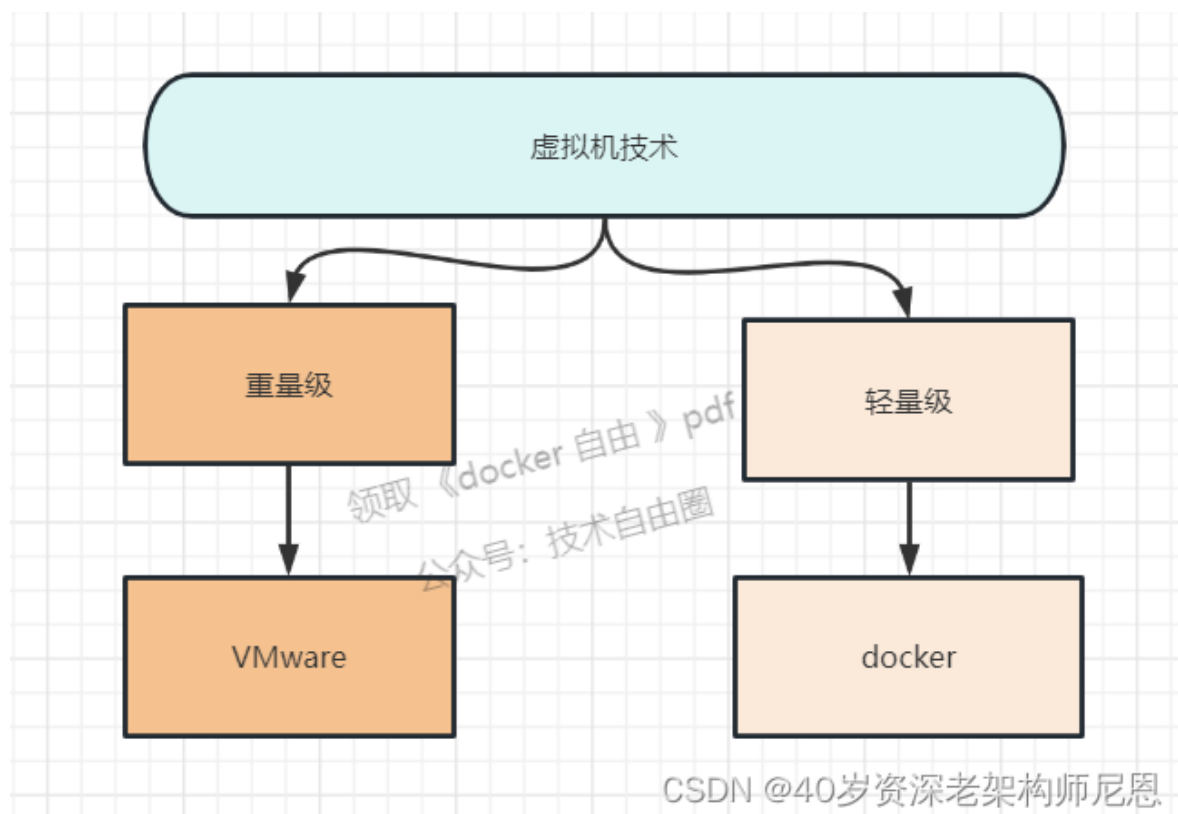
docker为什么有这么巨大的价值呢？

因为，在容器技术出来之前，大家都是使用虚拟机技术，比如在 window中装一个VMware，通过这个软件我们可以虚拟出来一台或者多台电脑，实现硬件资源的细粒度分割和使用隔离。

但是，虚拟机技术太笨重啦！模式太重。

Docker容器技术，也是一种虚拟化技术，也是实现硬件资源的细粒度分割和使用隔离。但是，Docker 是一种轻量级的虚拟机技术。

Docker容器是完全使用沙箱机制，相互之间不会有任何接口（类似 iPhone 的 app），更重要的是容器性能开销极低。



Docker 从 17.03 版本之后分为 CE（Community Edition: 社区版）和 EE（Enterprise Edition: 企业版），

对于开发人员来说，用 CE（Community Edition）社区版就可以了

Docker的广泛应用场景：

- Web 应用的自动化打包和发布。
- 自动化测试和持续集成、发布。
- 在服务型环境中部署和调整数据库或其他的后台应用。
- 从头编译或者扩展现有的 OpenShift 或 Cloud Foundry 平台来搭建自己的 PaaS 环境。

一般来说，测试环境、生产环境，基本都已经全面docker化了。

可见，docker技术，是那么那么的重要。

Docker的在DevOps（开发、运维）场景的应用

一般来说，怎么的微服务应用，有多套环境：

- （1）开发
- （2）测试
- （3）预生产
- （4）生产

四套环境，导致环境配置是十分的麻烦，每一个环境都要部署各种组件(如Redis、ES、zk)，非常的费时费力。

更要命的是，在生产环境上，吞吐量一上来，需要动态扩容。

使用docker，咱们可以将DevOps（开发、运维）的工作，高速完成：

（1）快速完成 发布工作

开发环境一般是 Windows/mac，最后发布到Linux。

没有docker之前，使用jar包发布，配上大量的shell脚本，然后各种配置，及其复杂。

有了docker之后，做好镜像，开发打包部署上线，一套流程做完！

（2）快速完成 交付工作

传统的交付工作，要给用户提供各种安装的帮助文档，安装程序，基础环境安装，依赖的中间件安装，等等等等。

有了docker之后，能更快速的交付和部署。给他一套镜像，通过Docker命令，一键运行，啥都是好的。

那你看，docker 是不是，真香。

（3）更便捷的升级和扩缩容

使用了 Docker之后，我们项目打包为一个镜像，部署应用就和搭积木一样

扩展服务器A，启动一个容器就ok。

扩展服务器B，启动一个容器就ok。

如果要动态扩展，使用K8S这类分布式容器管理基础设施，配上一个HPA 控制器组件，就能自动的完成动态扩容，动态缩容。

那你看，docker 是不是，真香。

（4）服务器的性能可以被压榨到极致

Docker是内核级别的虚拟化，可以在一个物理机上可以运行很多的容器实例。

服务器的性能可以被压榨到极致

那你看，docker 是不是，真香。

接下来，随着40岁老架构师一起，来穿透docker的原理和实操吧。

只有先穿透docker，才能穿透K8S，最终穿透云原生+大数据，实现你的技术自由。

Docker的历史

2010年，几个的年轻人在美国的旧金山成立了一家公司 dotcloud。dotcloud 是一个Paas平台的创业公司，从事LXC（Linux Container容器）有关的容器技术。

Linux Container容器是一种内核虚拟化技术，可以提供轻量级的虚拟化，以便隔离进程和资源。他们将自己的技术（容器化技术）命名就是 Docker。

Docker刚刚延生的时候，没有引起行业的注意！

虽然获得了创业孵化器(Y Combinator)的支持、也获得过一些融资，但随着IT巨头们(微软、谷歌、亚马逊等厂商)也进入PaaS凭他，dotCloud举步维艰，眼看就活不下去！

2013年，dotCloud的创始人，28岁的Solomon Hykes做了一个艰难的决定，将dotCloud的核心引擎开源，这项核心引擎技术能够将Linux容器中的应用程序、代码打包，轻松的在服务器之间进行迁移。

2013发布了 Docker-compose 组件提供容器的编排工具。

2014年 Docker 发布1.0版本，2015年Docker 提供 Docker-machine，支持 windows 平台。

docker火了。

这个基于LXC技术的核心管理引擎开源后，让全世界的技术人员感到惊艳。

大家感叹这一切太方便了！！于是，越来越多的人发现docker的优点，使用他！

虽然，Docker 项目在开源社区大受追捧，同时也被业界诟病的是： Docker 公司对于 Docker 发展具有绝对的话语权，比如 Docker 公司推行了 libcontainer 难以被社区接受。

为了防止 Docker 这项开源技术被Docker 公司控制，几个核心贡献代码的厂商诸如 Redhat，谷歌的倡导下，成立了 OCI 开源社区，制定了 OCI 开放容器标准，*Open Container Initiative*（OCI，开放容器标准）。

OCI 开源社区旨在于将 Docker 的发展权利回归社区，当然反过来讲，Docker 公司也希望更多的厂商安心贡献代码到Docker 项目，促进 Docker 项目的发展。

Docker 将自己容器格式和运行时 runC 捐给了 OCI，OCI 在此基础上制定了 2 个标准：

运行时标准 Runtime Specification (runtime-spec)

镜像标准 Image Specification (image-spec)：

于是通过OCI建立了 runc 项目，替代 libcontainer，这为开发者提供了除 Docker 之外的容器化实现的选择。

OCI 社区提供了 runc 的维护，而 runc 是基于 OCI 规范的运行容器的工具。

换句话说，你可以通过 runc，提供自己的容器实现，而不需要依赖 Docker。

当然，Docker 的发行版底层也是用的 runc。在 Docker 宿主机上执行 runc，你会发现它的大多数命令和 Docker 命令类似，感兴趣的读者可以自己实践如何用 runc 启动容器。

至2017年，Docker 项目转移到 Moby 项目，基于 Moby 项目，Docker 提供了两种发行版，Docker CE 和 Docker EE，Docker CE 就是目前大家普遍使用的版本，Docker EE 成为付费版本，提供了容器的编排，Service 等概念。

Docker 公司承诺 Docker 的发行版会基于 Moby 项目。这样一来，通过 Moby 项目，你也可以自己打造一个定制化的容器引擎，而不会被 Docker 公司绑定。

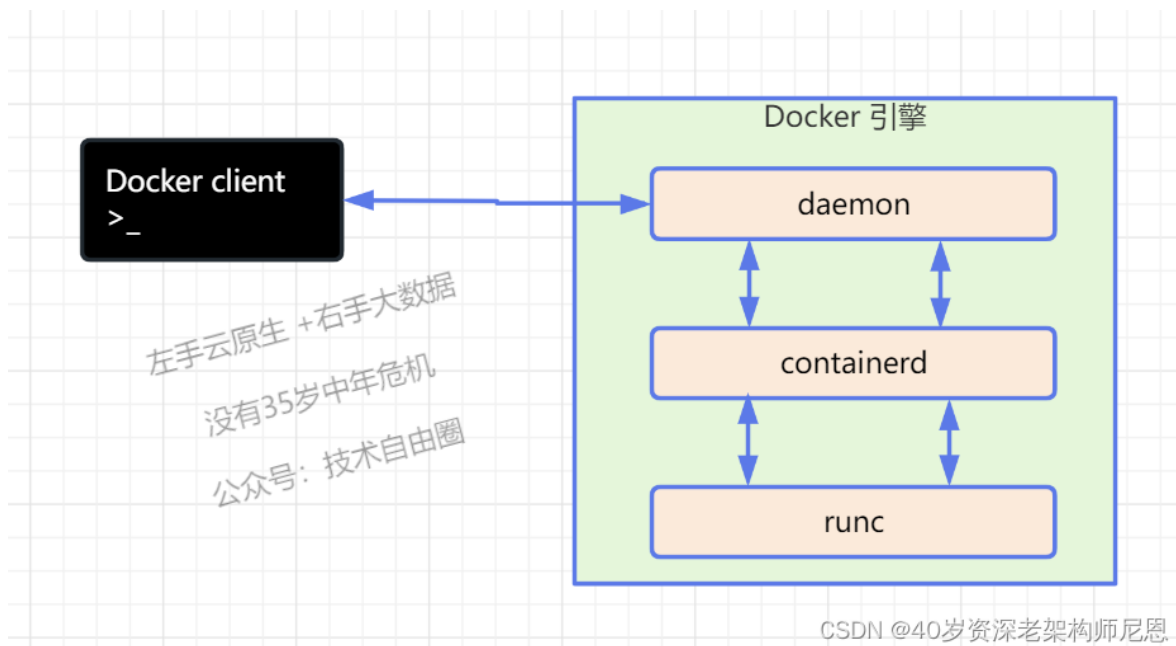
Docker 的入门知识

从大家常用的Docker Engine开始说起。

Docker Engine

当人们说“Docker”时，他们通常是指 Docker Engine，它是一个客户端 - 服务器应用程序，

Docker 引擎由如下主要的组件构成：Docker 客户端（Docker Client）、Docker 守护进程（Docker daemon）、containerd 以及 runc。



Docker Engine 从 CLI 中接受docker 命令，完成容器的管理：

例如使用 docker run 、 docker ps 来列出正在运行的容器、

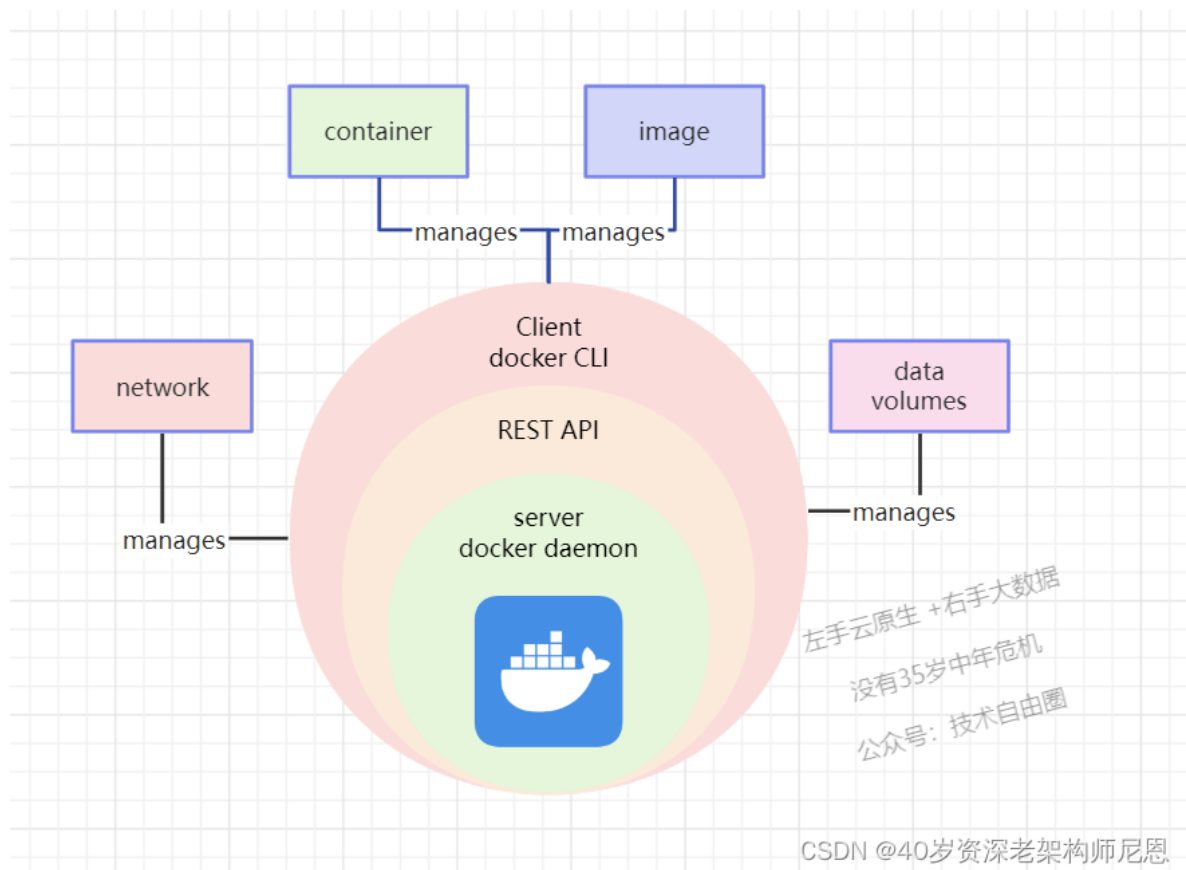
例如使用docker images 来列出镜像，

等等。

Docker是一个Client-Server结构的系统，Docker守护进程运行在主机上，

Client通过Socket连接从客户端访问Docker守护进程。Docker守护进程从客户端接受命令，并按照命令，管理运行在主机上的容器。

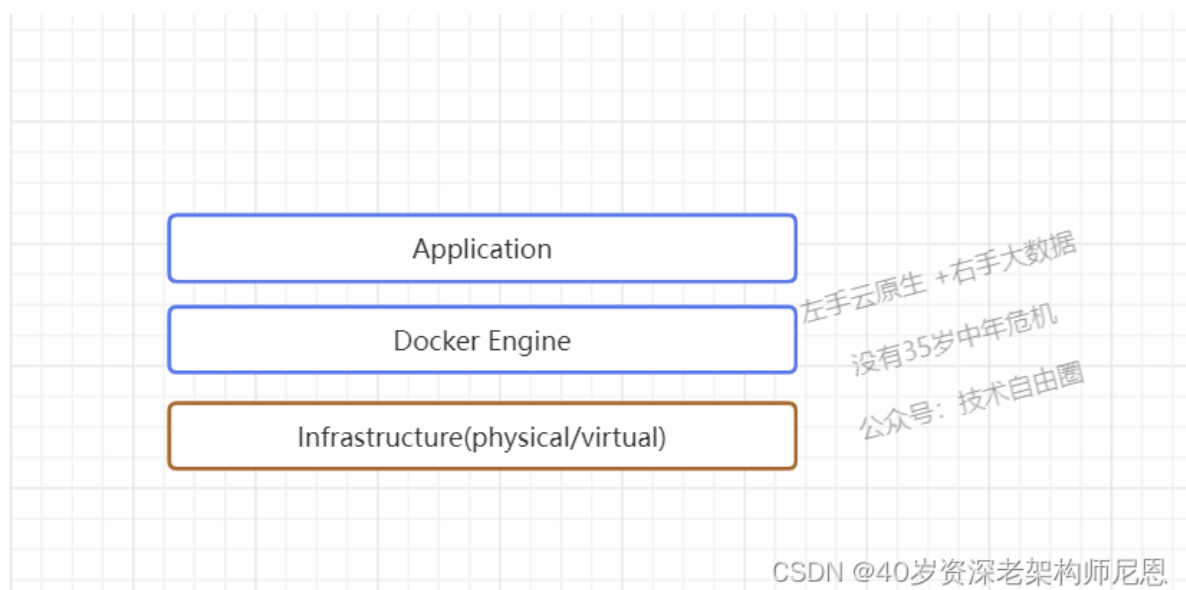
- 后台进行(dockerd)
- REST API Server
- CLI接口 (docker)



Docker Platform

- Docker提供了一个开发，打包，运行app的平台
- 把app和底层infrastructure隔离开来

其三层模型如图：



到底什么是docker:

到底什么是docker:

- docker是一个软件，可以运行在window、linux、mac等各种操作系统上。
- docker 是一个开源的应用容器引擎，基于Go 语言开发并遵从 Apache2.0 协议开源，项目代码托管在github上进行维护
- docker 可以让开发者打包他们的应用以及依赖包到一个轻量级、可移植的容器中，然后发布到任何流行的 Linux 机器上。
- 容器是完全使用沙箱机制，相互之间不会有任何接口,更重要的是容器性能开销极低。

什么是容器?

什么是容器?

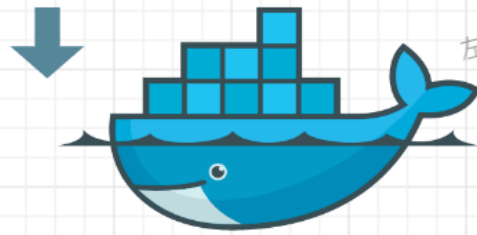
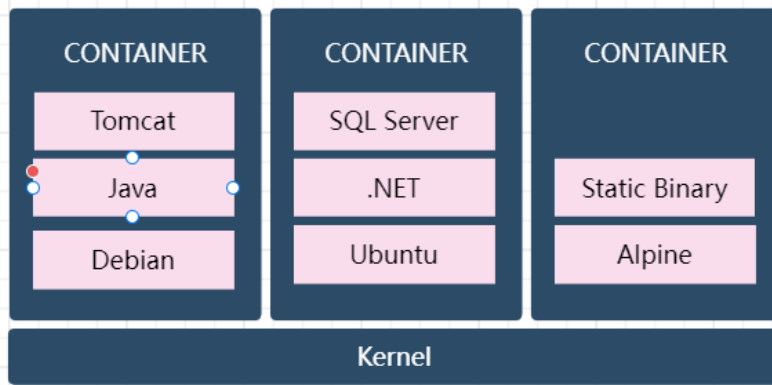
- 对软件和其依赖的标准化打包
- 应用之间相互隔离
- 共享同一个OS Kernel
- 可以运行在很多主流操作系统上

注：容器和虚拟机的区别在于容器是APP层面的隔离，而虚拟化是物理资源层面的隔离

容器解决了什么问题?

- 解决了开发和运维之间的矛盾
- 在开发和运维之间搭建了一个桥梁，是实现devops最佳解决方案

一个docker 容器，是一个运行时环境，可以简单理解为进程运行的集装箱。



docker

左手云原生 + 右手大数据
没有35岁中年危机
公众号: 技术自由圈

CSDN @40岁资深老架构师尼恩

docker基本组成

docker主机(Host): 安装了Docker程序的机器 (Docker直接安装在操作系统之上) ;

docker仓库(Registry): 用来保存各种打包好的软件镜像; 仓库分为公有仓库和私有仓库。(很类似maven)

docker镜像(Images): 软件打包好的镜像; 放在docker仓库中;

docker容器(Container): 镜像启动后的实例称为一个容器; 容器是独立运行的一个或一组应用

Docker 包括三个基本概念:

- **镜像 (Image)** : Docker 镜像 (Image) , 就相当于是一个 root 文件系统。比如官方镜像 ubuntu:16.04 就包含了完整的一套 Ubuntu16.04 最小系统的 root 文件系统。
- **容器 (Container)** : 镜像 (Image) 和容器 (Container) 的关系, 就像是面向对象程序设计中的类和实例一样, 镜像是静态的定义, 容器是镜像运行时的实体。容器可以被创建、启动、停止、删除、暂停等。
- **仓库 (Repository)** : 仓库可看成是一个代码控制中心, 用来保存镜像。

Docker 使用客户端-服务器 (C/S) 架构模式, 使用远程API来管理和创建Docker容器。

Docker 容器通过 Docker 镜像来创建。

概念	说明
Docker 镜像(Images)	Docker 镜像是用于创建 Docker 容器的模板，比如 Ubuntu 系统。
Docker 容器(Container)	容器是独立运行的一个或一组应用，是镜像运行时的实体。
Docker 客户端(Client)	Docker 客户端通过命令行或者其他工具使用 Docker SDK (https://docs.docker.com/develop/sdk/) 与 Docker 的守护进程通信。
Docker 主机(Host)	一个物理或者虚拟的机器用于执行 Docker 守护进程和容器。
Docker Registry	Docker 仓库用来保存镜像，可以理解为代码控制中的代码仓库。Docker Hub(https://hub.docker.com) 提供了庞大的镜像集合供使用。一个 Docker Registry 中可以包含多个仓库 (Repository)；每个仓库可以包含多个标签 (Tag)；每个标签对应一个镜像。通常，一个仓库会包含同一个软件不同版本的镜像，而标签就常用于对应该软件的各个版本。我们可以通过 <仓库名>:<标签> 的格式来指定具体是这个软件哪个版本的镜像。如果不给出标签，将以 latest 作为默认标签。

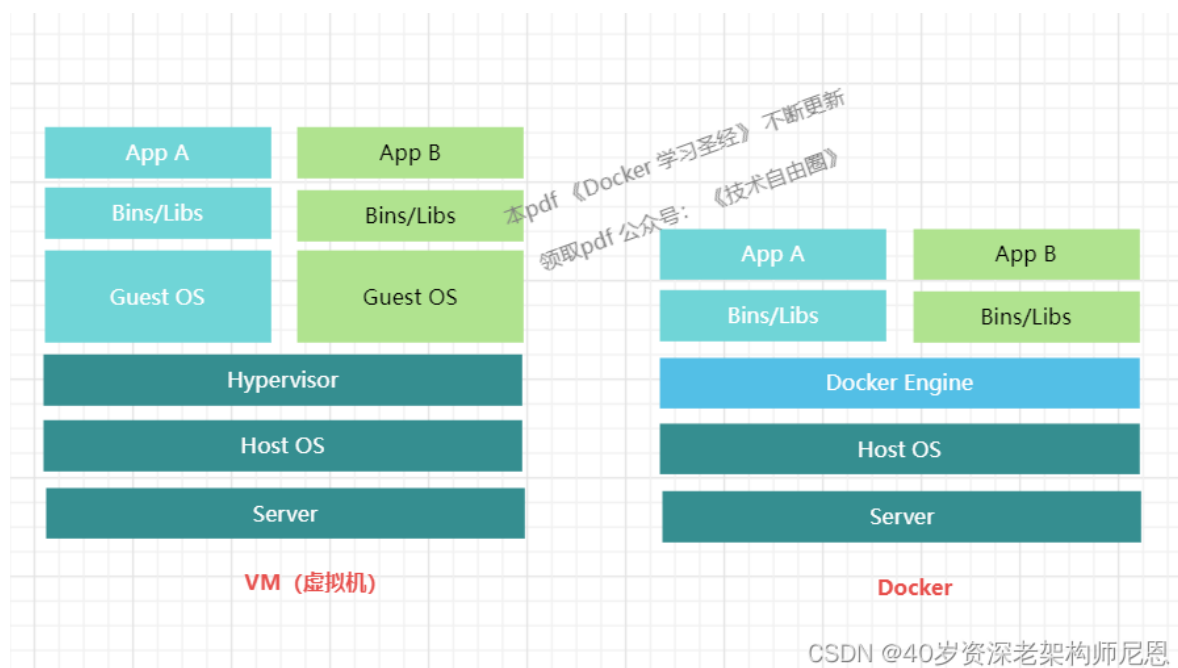
Docker 与虚拟机有何区别

Docker 的误解：Docker 是轻量级的虚拟机。

很多人将docker理解为， Docker 实现了类似于虚拟化的技术，能够让应用跑在一些轻量级的容器里。这么理解其实是错误的。

docker和kvm都是虚拟化技术，它们的主要差别：

- 1、Docker有着比虚拟机更少的抽象层
- 2、docker利用的是宿主机的内核，VM需要的是Guest OS



二者的不同：

- VM(VMware)在宿主机、宿主机操作系统的基础上创建虚拟层、虚拟化的操作系统、虚拟化的仓库，然后再安装应用；
- Container(Docker容器)，在宿主机、宿主机操作系统上创建Docker引擎，在引擎的基础上再安装应用。

所以说，新建一个容器的时候，docker不需要像虚拟机一样重新加载一个操作系统，避免引导。docker是利用宿主机的操作系统，省略了这个复杂的过程，秒级！

虚拟机是加载Guest OS，这是分钟级别的

与传统VM特性对比：

作为一种轻量级的虚拟化方式，Docker在运行应用上跟传统的虚拟机方式相比具有显著优势：

- Docker 容器很快，启动和停止可以在秒级实现，这相比传统的虚拟机方式要快得多。
- Docker 容器对系统资源需求很少，一台主机上可以同时运行数千个Docker容器。
- Docker 通过类似Git的操作来方便用户获取、分发和更新应用镜像，指令简明，学习成本较低。
- Docker 通过Dockerfile配置文件来支持灵活的自动化创建和部署机制，提高工作效率。
- Docker 容器除了运行其中的应用之外，基本不消耗额外的系统资源，保证应用性能的同时，尽量减小系统开销。
- Docker 利用Linux系统上的多种防护机制实现了严格可靠的隔离。从1.3版本开始，Docker引入了安全选项和镜像签名机制，极大地提高了使用Docker的安全性。

特性	容器	虚拟机
启动速度	秒级	分钟级
硬盘使用	一般为MB	一般为GB
性能	接近原生	弱于原生
系统支持量	单机支持上千个容器	一般几十个

docker与操作系统比较

docker是一种轻量级的虚拟化方式。与传统操作系统技术的特性比较如下表：

特 性	容 器	虚 拟 机
启动速度	秒级	分钟级
性能	接近原生	较弱
内存代价	很小	较多
硬盘使用	一般为MB	一般为GB
运行密度	单机支持上千个容器	一般几十个
隔离性	安全隔离	完全隔离
迁移性	优秀	一般

传统的虚拟机方式提供的是相对封闭的隔离。

Docker利用Linux系统上的多种防护技术实现了严格的隔离可靠性，并且可以整合众多安全工具。

从 1.3.0版本开始，docker重点改善了容器的安全控制和镜像的安全机制，极大提高了使用docker的安全性。

Docker 的安装

安装**docker**前置条件

当我们安装 Docker 的时候，会涉及两个主要组件：

- Docker CLI：客户端
- Docker daemon：有时也被称为“服务端”或者“引擎”

环境准备

硬件总体要求，可以参考尼恩的本地硬件情况：

1 硬件要求。

本文硬件总体要求如下表：

序号	硬件	要求
1	CPU	至少2核
2	内存	至少8G
3	硬盘	至少100G磁盘空间

2 本地虚拟机环境

软件	版本
Win	win10以上
virtual box	6以上
vagrant	2以上

docker+K8S学习环境非常复杂，尼恩搞这个前前后后起码折腾了一周，其中，很多头疼的工作，包括linux内核升级、磁盘扩容等等，苦不堪言。

现在把这个环境，以虚拟机box镜像的方式，导出来直接给大家，大家一键导入后，直接享受docker 的实操，享受K8S的实操，可以说，爽到不要不要的。

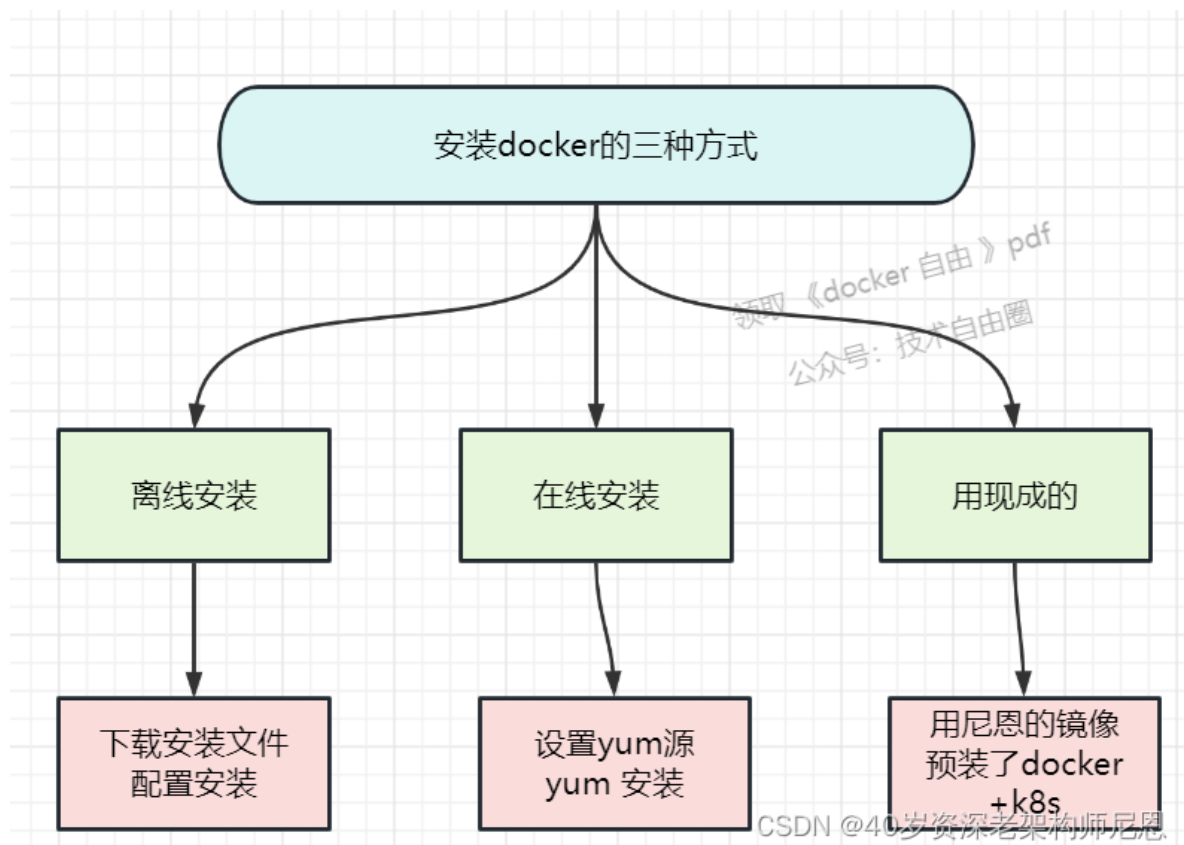
以上软件和 尼恩个人的虚拟机box镜像，可以找尼恩获取。

docker安装的三种方式

- 安装docker的三种方式
- (1) 离线安装

(2) 在线安装

(3) 用现成的



方式一：离线安装docker

这里以 19.03.9 版本进行介绍，其他版本是一样的。

这种安装方式，可以用于没有 互联网的 场景。

比如，很多公司，并不能直接上外网。

一、基础环境

- 1、操作系统：CentOS 7.3
- 2、Docker版本：[19.03.9 官方下载地址](#)
- 3、官方参考文档：<https://docs.docker.com/install/linux/docker-ce/binaries/#install-static-binaries>

二、Docker安装

1、下载

wget https://download.docker.com/linux/static/stable/x86_64/docker-19.03.9.tgz

注意：如果事先下载好了可以忽略这一步

2、解压

把压缩文件存在指定目录下(如root)，并进行解压

```
tar -zxvf docker-19.03.9.tgz
```

```
cd root
[root@localhost ~]# tar -zxvf docker-19.03.6.tgz
docker/
docker/containerd
docker/docker
docker/ctr
docker/dockerd
docker/runc
docker/docker-proxy
docker/docker-init
docker/containerd-shim
```

3、将解压出来的docker文件内容移动到 /usr/bin/ 目录下

```
cp docker/* /usr/bin/
```

4、将docker注册为service

```
cat /etc/systemd/system/docker.service
```

```
vi /etc/systemd/system/docker.service
```

```
[Unit]
Description=Docker Application Container Engine
Documentation=https://docs.docker.com
After=network-online.target firewalld.service
Wants=network-online.target

[Service]
Type=notify
# the default is not to use systemd for cgroups because the delegate issues
still
# exists and systemd currently does not support the cgroup feature set required
# for containers run by docker
ExecStart=/usr/bin/dockerd
ExecReload=/bin/kill -s HUP $MAINPID

# Having non-zero Limit*s causes performance problems due to accounting overhead
# in the kernel. We recommend using cgroups to do container-local accounting.

LimitNOFILE=infinity
LimitNPROC=infinity
LimitCORE=infinity

# Uncomment TasksMax if your systemd version supports it.
# Only systemd 226 and above support this version.
#TasksMax=infinity
TimeoutStartSec=0

# set delegate yes so that systemd does not reset the cgroups of docker
containers
```

```

Delegate=yes

# kill only the docker process, not all processes in the cgroup

KillMode=process

# restart the docker process if it exits prematurely

Restart=on-failure
StartLimitBurst=3
StartLimitInterval=60s

[Install]
WantedBy=multi-user.target

```

5、启动

chmod +x /etc/systemd/system/docker.service #添加文件权限并启动docker

systemctl daemon-reload #重载unit配置文件

systemctl start docker #启动Docker

systemctl enable docker.service #设置开机自启

```

[root@localhost ~]# vi /etc/systemd/system/docker.service
[root@localhost ~]# chmod +x /etc/systemd/system/docker.service
[root@localhost ~]# systemctl daemon-reload
[root@localhost ~]# systemctl start docker
[root@localhost ~]# systemctl enable docker.service
Created symlink from /etc/systemd/system/multi-user.target.wants/docker.service
to /etc/systemd/system/docker.service.

```

6、验证

systemctl status docker #查看Docker状态

docker -v #查看Docker版本

docker info

```

[root@localhost ~]# systemctl status docker
docker.service - Docker Application Container Engine
   Loaded: loaded (/etc/systemd/system/docker.service; enabled; vendor preset:
disabled)
   Active: active (running) since Sat 2021-10-09 15:25:44 CST; 29s ago
     Docs: https://docs.docker.com
    Main PID: 1916 (dockerd)
      CGroup: /system.slice/docker.service
              └─1916 /usr/bin/dockerd
                  └─1927 containerd --config /var/run/docker/containerd/containerd.toml
--log-level info

Oct 09 15:25:43 localhost.localdomain dockerd[1916]: time="2021-10-
09T15:25:43.671407996+08:00" level=info msg="scheme \"unix\" not r...e=grpc

```

```
Oct 09 15:25:43 localhost.localdomain dockerd[1916]: time="2021-10-09T15:25:43.671440368+08:00" level=info msg="ccResolverWrapper: se...e=grpc"
Oct 09 15:25:43 localhost.localdomain dockerd[1916]: time="2021-10-09T15:25:43.671462935+08:00" level=info msg="ClientConn switching ...e=grpc"
Oct 09 15:25:43 localhost.localdomain dockerd[1916]: time="2021-10-09T15:25:43.750687781+08:00" level=info msg="Loading containers: start."
Oct 09 15:25:44 localhost.localdomain dockerd[1916]: time="2021-10-09T15:25:44.072960862+08:00" level=info msg="Default bridge (docke...dress"
Oct 09 15:25:44 localhost.localdomain dockerd[1916]: time="2021-10-09T15:25:44.153444071+08:00" level=info msg="Loading containers: done."
Oct 09 15:25:44 localhost.localdomain dockerd[1916]: time="2021-10-09T15:25:44.175249299+08:00" level=info msg="Docker daemon" commit...9.03.6
Oct 09 15:25:44 localhost.localdomain dockerd[1916]: time="2021-10-09T15:25:44.175337834+08:00" level=info msg="Daemon has completed ...ation"
Oct 09 15:25:44 localhost.localdomain systemd[1]: Started Docker Application Container Engine.
Oct 09 15:25:44 localhost.localdomain dockerd[1916]: time="2021-10-09T15:25:44.195084106+08:00" level=info msg="API listen on /var/ru....sock"
Hint: Some lines were ellipsized, use -l to show in full.
[root@localhost ~]# docker -v
Docker version 19.03.6, build 369ce74a3c
[root@localhost ~]# docker info
```

方式二：在线安装docker

如果可以连接公网，建议在线安装。

这里注意 linux和 docker的版本。

尼恩安装 docker 最新版本的时候，发现依赖了 Centos 8 以上的版本。

在线安装docker步骤

1. 更新yum

```
yum update
```

2. 安装工具包

```
yum -y install yum-utils
```

3. 设置yum源

```
yum-config-manager --add-repo http://mirrors.aliyun.com/docker-ce/linux/centos/docker-ce.repo
```

以腾讯源为例

```
https://mirrors.cloud.tencent.com/docker-ce/linux/centos/docker-ce.repo
```

4. 安装Docker-Ce(社区版)

```
yum install docker-ce
```

5. 查看docker版本（用来确认是否安装成功）

```
# 输入 docker-v 后如果出现下面的内容则代表安装成功
[root@VM-24-9-centos ~]# docker -v
Docker version 20.10.23, build 7155243
```

6. Docker镜像加速（国内使用）

```
# 需要确定/etc下面是否有docker这个文件夹，若没有则需要使用下面的命令进行创建
mkdir -p /etc/docker

# 创建配置文件daemon.json
vi /etc/docker/daemon.json

# 写入以下内容
{
  "registry-mirrors": [
    "https://mirror.ccs.tencentyun.com" # 可以替换为其他厂商的地址
  ]
}

# 重载一下配置
systemctl daemon-reload
```

7. 启动Docker服务

```
systemctl start docker
```

8. 验证docker是否可以使用

```
[root@VM-24-9-centos ~]# docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

方式三：用现成的（大大的省事）

docker+K8S 是一整套技术体系。

但是，docker+K8S学习环境非常复杂，尼恩搞这个 前前后后起码折腾了一周。

其中，很多头疼的工作，包括linux内核升级、磁盘扩容等等，苦不堪言。

现在把这个环境，以虚拟机box镜像的方式，导出来直接给大家，

大家一键导入后，直接**享受docker 的实操，享受K8S的实操**，

可以说，爽到不要不要的。

SSH session to root@cdhl

SSH compression : x

SSH-browser : x

X11-forwarding : x (disabled or not supported by server)

DISPLAY : 192.168.0.6:0.0

For more info, ctrl+click on help or visit our website

领取《docker 自由》pdf
公众号：技术自由圈

This system is built by the Bento project by Chef Software

More information can be found at <https://github.com/chef/bento>

Last login: Sun Mar 12 12:06:39 2023

[root@centos1 ~]# docker -v

Docker version 20.10.23, build 7155243

[root@centos1 ~]# docker ps

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
e9eae999da9	apacherocketmq/rocketmq:4.6.0	"sh mqbroker -c /opt..."	6 days ago	Up 44 seconds	9876/tcp, 18989/tc
p, 10911-10912/tcp, 0.0.0.0:20921-20922->20921-20922/tcp, :::20921-20922->20921-20922/tcp	apacherocketmq/rocketmq:4.6.0	"sh mqbroker -c /opt..."	6 days ago	Up 44 seconds	9876/tcp, 18989/tc
fd4e13f20ada	apacherocketmq/rocketmq:4.6.0	"sh mqnamesrv"	6 days ago	Up 44 seconds	10909/tcp, 10911-1
p, 10911-10912/tcp, 0.0.0.0:20911-20912->20911-20912/tcp, :::20911-20912->20911-20912/tcp	apacherocketmq/rocketmq:4.6.0	"sh -c 'java \$JAVA_O..."	6 days ago	Up 44 seconds	0.0.0.0:29001->900
a71b7e7357d3	apacherocketmq/rocketmq:4.6.0	"sh mqnamesrv"	6 days ago	Up 44 seconds	10909/tcp, 10911-1
0912/tcp, 0.0.0.0:19876->9876/tcp, :::19876->9876/tcp	styletang/rocketmq-console-ng	"sh mqnamesrv"	6 days ago	Up 45 seconds (healthy)	127.0.0.1:1514->10
4767f0e9f73c	goharbor/harbor-log:v2.3.2	"docker-entrypoint.s..."	9 days ago	Up 45 seconds	0.0.0.0:6379->6379
1/tcp, :::29001->9001/tcp	registry.cn-hangzhou.aliyuncs.com/zhengqing/redis:6.0.8	"docker-entrypoint.s..."	4 weeks ago	Up 44 seconds	0.0.0.0:3306->3306
cb864706808e	apacherocketmq/rocketmq:4.6.0	"sh mqnamesrv"	4 weeks ago	Up 44 seconds	0.0.0.0:2181->2181
0912/tcp, 0.0.0.0:19877->9877/tcp, :::19877->9877/tcp	mysql:5.7				
47af8e699185	goharbor/harbor-log:v2.3.2				
514/tcp					
0344212df5d6	registry.cn-hangzhou.aliyuncs.com/zhengqing/redis:6.0.8				
/tcp, :::6379->6379/tcp					
1d00f3ade2b6	mysql:5.7				
/tcp, :::3306->3306/tcp, 33060/tcp					
129a46feed77	zookeeper:3.4.13				
/tcp, :::2181->2181/tcp, 0.0.0.0:2888->2888/tcp, :::2888->2888/tcp, 0.0.0.0:3888->3888/tcp, :::3888->3888/tcp					

CSDN @40岁资深老架构师尼恩

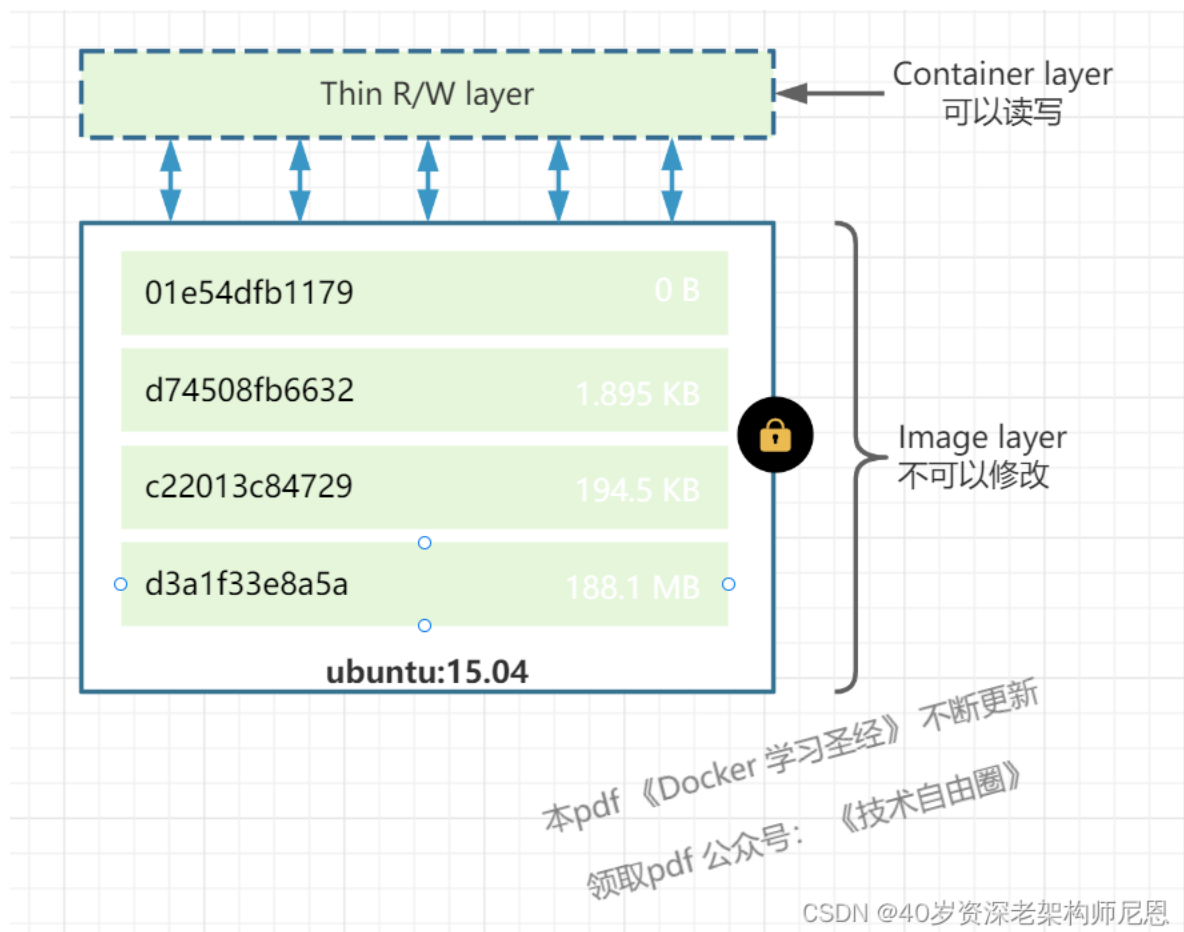
以上环境和 尼恩个人的虚拟机box镜像，可以找尼恩获取。

Docker Container概述

什么是Container 容器

- 通过Image创建(copy)
- 在Image layer之上建立一个container layer（可读写）
- 类比面向对象：类和实例
- Image负责app的存储和分发，Container负责运行app

page:20/129 of 尼恩Java硬核架构班：狠卷3高架构，卷透底层技术，走向技术自由！

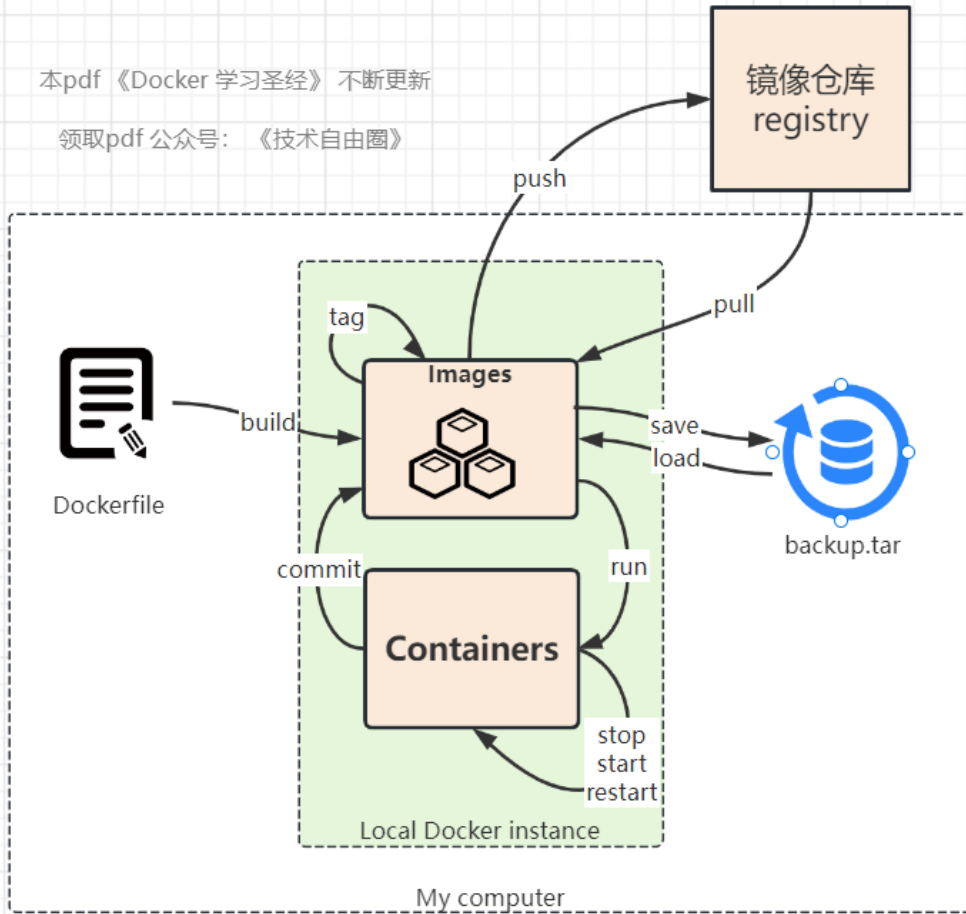


容器与镜像的关系

- 镜像：镜像是只读文件，提供运行程序完整的软硬件资源。
- 容器：容器是镜像的实例，由docker负责创建，容器之间彼此隔离。

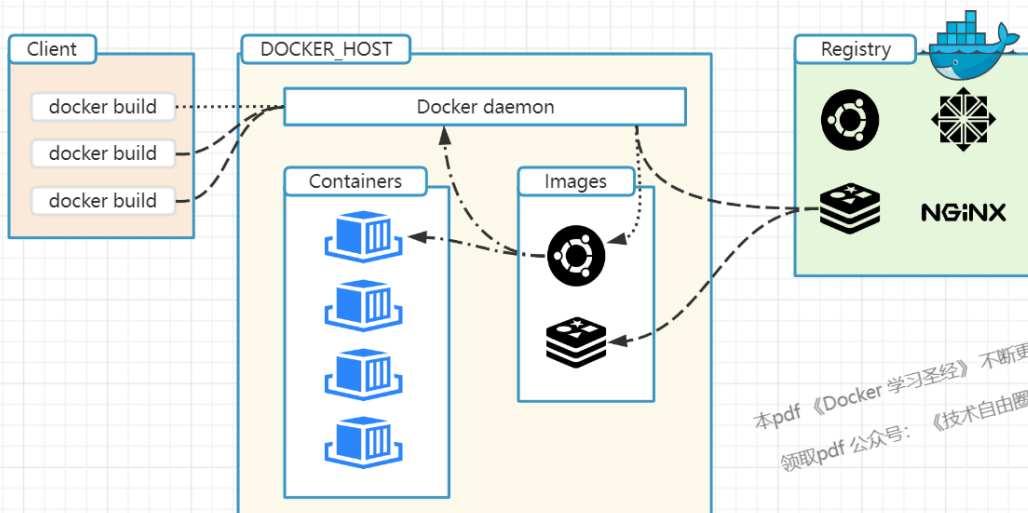
本pdf《Docker 学习圣经》不断更新

领取pdf 公众号：《技术自由圈》



CSDN @40岁资深老架构师尼恩

Docker本地容器相关的操作



本pdf《Docker 学习圣经》不断更新
领取pdf 公众号：《技术自由圈》

CSDN @40岁资深老架构师尼恩

Container相关命令

- 创建容器 `docker run centos`
- 创建容器 `docker run -it centos`

- 查看活跃容器 `docker ps`
- 查询容器状态 `docker container ls -a`
- 移除容器 `docker container rm + [container ID]`
- 移除镜像 `docker image rm + [image ID]`
- 显示所有containerID `docker container ls -aq`
- 移除所有的container `docker rm $(docker container ls -aq)`

创建容器

创建名为"centos6"的容器，并在容器内部和宿主机中查看容器中的进程信息

```
docker run -itd -p 6080:80 -p 6022:22 docker.io/lemonbar/centos6-ssh:latest
```

结果如下

```
[root@VM-4-17-centos ~]# docker run -itd -p 80:80 -p 6022:22
docker.io/lemonbar/centos6-ssh:latest
Unable to find image 'lemonbar/centos6-ssh:latest' locally
latest: Pulling from lemonbar/centos6-ssh
a3ed95caeb02: Pull complete
f79eb1f22352: Pull complete
67c1aaa530c8: Pull complete
80447774eee7: Pull complete
6d67b3a80e5a: Pull complete
f1819e4b2f8f: Pull complete
09712b5b9acc: Pull complete
8bc987c5494f: Pull complete
c42b021d0ff2: Pull complete
Digest: sha256:093c2165b3c6fe05d5658343456f9b59bb7ecc690a7d3a112641c86083227dd1
Status: Downloaded newer image for lemonbar/centos6-ssh:latest
a4f1c9b8abcda78c8764cc285183dfa56cd1aa4ce6d111d4d9e77f3a57f3d5fc
```

查看活跃容器 docker ps

`docker ps`

```
[root@centos1 ~]# docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
e9eae999da9	apacherocketmq/rocketmq:4.6.0	"sh mqbroker -c /opt..."	7 days ago	Up 3 hours	9876/tcp, 10909/t
10911-10912/tcp	0.0.0.0:20921-20922->20921-20922/tcp, ::20921-20922->20921-20922/tcp	"sh mqbroker -c /opt..."	7 days ago	Up 3 hours	9876/tcp, 10909/t
fd4e13f20ada	apacherocketmq/rocketmq:4.6.0	"sh mqnamesrv"	7 days ago	Up 3 hours	10909/tcp, 10911-
10911-10912/tcp	0.0.0.0:20911-20912->20911-20912/tcp, ::20911-20912->20911-20912/tcp	"sh mqnamesrv"	7 days ago	Up 3 hours	0.0.0.0:29001->90
a71b7e7357d3	apacherocketmq/rocketmq:4.6.0	"sh mqnamesrv"	7 days ago	Up 3 hours	10909/tcp, 10911-
2/tcp	0.0.0.0:19876->9876/tcp, ::19876->9876/tcp	"sh mqnamesrv"	7 days ago	Up 3 hours	10909/tcp, 10911-
4767f0e9f73c	styletang/rocketmq-console-ng	"sh -c 'java \$JAVA_O..."	7 days ago	Up 3 hours	127.0.0.1:1514->1
cp, ::29001->9001/tcp					
cb86470680e	apacherocketmq/rocketmq:4.6.0	"sh mqnamesrv"	7 days ago	Up 3 hours	10909/tcp, 10911-
2/tcp	0.0.0.0:19877->9876/tcp, ::19877->9876/tcp				
47af8e699185	goharbor/harbor-log:v2.3.2	"/bin/sh -c /usr/loc..."	8 days ago	Up 3 hours (healthy)	127.0.0.1:1514->1
/tcp					
0344212df5d6	registry.cn-hangzhou.aliyuncs.com/zhengqing/redis:6.0.8	"docker-entrypoint.s..."	9 days ago	Up 3 hours	0.0.0.0:6379->637
p, :::6379->6379/tcp					
1d00f3ade2b6	mysql:5.7	"docker-entrypoint.s..."	4 weeks ago	Up 3 hours	0.0.0.0:3306->330
p, :::3306->3306/tcp					
129a46feed77	zookeeper:3.4.13	"/docker-entrypoint..."	4 weeks ago	Up 3 hours	0.0.0.0:2181->218
p, :::2181->2181/tcp	0.0.0.0:2888->2888/tcp, ::2888->2888/tcp, 0.0.0.0:3888->3888/tcp, ::3888->3888/tcp				

```
[root@centos1 ~]#
```

CSDN @40岁资深老架构师尼恩

查看全部容器

查询容器状态

```
docker ps -a
```

```
docker container ls -a
```

两个命令，效果差不多

停止容器

```
docker stop id
```

删除容器

```
docker rm id
```

查看容器的进程信息

docker top :查看容器中运行的进程信息，支持 ps 命令参数。

语法

```
docker top [OPTIONS] CONTAINER [ps OPTIONS]
```

容器运行时不一定有/bin/bash终端来交互执行top命令，而且容器还不一定有top命令，

可以使用docker top来实现查看container中正在运行的进程。

```
docker top 容器名称
```

尼恩的虚拟机中，存在容器zookeeper/ mysql，如果想查看zookeeper/ mysql 容器内的运行进程信息，

可以使用下述命令：

```
docker top zookeeper
docker top mysql
```

```
[root@centos1 ~]# docker top
"docker top" requires at least 1 argument.
See 'docker top --help'.

Usage:  docker top CONTAINER [ps OPTIONS]

Display the running processes of a container
[root@centos1 ~]# docker ps
CONTAINER ID   IMAGE                                COMMAND                                CREATED        STATUS        PORTS
e9eae999da9    apacherocketmq/rocketmq:4.6.0      "sh mqbroker -c /opt..."          7 days ago    Up 3 hours    9876/tcp, 10909/tcp,
10911-10912/tcp, 0.0.0.0:20921-20922->20921-20922/tcp
fd4e13f20ada    apacherocketmq/rocketmq:4.6.0      "sh mqbroker -c /opt..."          7 days ago    Up 3 hours    9876/tcp, 10909/tcp,
10911-10912/tcp, 0.0.0.0:20911-20912->20911-20912/tcp
a71b7e7357d3    apacherocketmq/rocketmq:4.6.0      "sh mqnamesrv"                     7 days ago    Up 3 hours    10909/tcp, 10911-1091
2/tcp, 0.0.0.0:19876->9876/tcp, ::19876->9876/tcp
4767f0e9f73c    styletang/rocketmq-console-ng      "sh -c 'java $JAVA_0..."          7 days ago    Up 3 hours    0.0.0.0:29001->9001/t
cp, ::29001->9001/tcp
cb864706800e    apacherocketmq/rocketmq:4.6.0      "sh mqnamesrv"                     7 days ago    Up 3 hours    10909/tcp, 10911-1091
2/tcp, 0.0.0.0:19877->9876/tcp, ::19877->9876/tcp
47af8e699185    goharbor/harbor-log:v2.3.2         "/bin/sh -c /usr/loc..."          8 days ago    Up 3 hours (healthy) 127.0.0.1:1514->10514
/tcp
0344212df5d6    registry.cn-hangzhou.aliyuncs.com/zhengqing/redis:6.0.8 "docker-entrypoint.s..."          9 days ago    Up 3 hours    0.0.0.0:6379->6379/tc
p, ::6379->6379/tcp
1d00f3ade2b6    mysql:5.7                           "docker-entrypoint.s..."          4 weeks ago    Up 3 hours    0.0.0.0:3306->3306/tc
p, ::3306->3306/tcp
129a46feed77    zookeeper:3.4.13                   "/docker-entrypoint.s..."          4 weeks ago    Up 3 hours    0.0.0.0:2181->2181/tc
p, ::2181->2181/tcp, 0.0.0.0:2888->2888/tcp, ::2888->2888/tcp, 0.0.0.0:3888->3888/tcp, ::3888->3888/tcp
[root@centos1 ~]# docker top zookeeper
UID          PID          PPID         C           STIME        TTY          TIME         CMD
vagrant      3485         3306         0           12:06        ?            00:00:06     /usr/lib/jvm/jav
a-1.8-openjdk/jre/bin/java -Dzookeeper.log.dir=/logs -Dzookeeper.root.logger=INFO,CONSOLE -cp /zookeeper-3.4.13/bin/./build/classes:/zookeeper-3.4.13/bin/
./build/lib/*:./zookeeper-3.4.13/bin/./lib/slf4j-log4j12-1.7.25.jar:/zookeeper-3.4.13/bin/./lib/slf4j-api-1.7.25.jar:/zookeeper-3.4.13/bin/./lib/netty
-3.10.6.Final.jar:/zookeeper-3.4.13/bin/./lib/log4j-1.2.17.jar:/zookeeper-3.4.13/bin/./lib/jline-0.9.94.jar:/zookeeper-3.4.13/bin/./lib/audience-annotati
ons-0.5.0.jar:/zookeeper-3.4.13/bin/./zookeeper-3.4.13.jar:/zookeeper-3.4.13/bin/./src/java/lib/*:./jar:/conf: -Dcom.sun.management.jmxremote -Dcom.sun.mana
gement.jmxremote.local.only=false org.apache.zookeeper.server.quorum.QuorumPeerMain /conf/zoo.cfg
[root@centos1 ~]# docker top mysql
UID          PID          PPID         C           STIME        TTY          TIME         CMD
systemd+    3332         3233         0           12:06        ?            00:00:04     mysqld --charact
er-set-server=utf8 --collation-server=utf8_general_ci
[root@centos1 ~]#
```

CSDN @40岁资深老架构师尼恩

如何查找容器名称？

很多的命令，用到容器名称

如何查找容器名称，可以使用下面的命令

```
[root@localhost ~]# docker ps --format "{{.Names}}"
```

结果如下：

```
[root@centos1 ~]# docker ps --format "{{.Names}}"
rmqbroker-ha-b
rmqbroker-ha-a
rmqnamesrv-ha-a
rmqconsole-ha
rmqnamesrv-ha-b
harbor-log
redis-standalone
mysql
zookeeper
[root@centos1 ~]#
```

CSDN @40岁资深老架构师尼恩

docker最为常用的几个命令

docker的守护进程查看

systemctl status docker

docker 镜像查看

docker image ls

docker 容器查看

docker ps

Docker Registry配置和查看

cat /etc/docker/daemon.json

配置私有仓库

```
cat>/etc/docker/daemon.json<<EOF

{

    "registry-mirrors":
    ["http://10.24.2.30:5000","https://tnxkcs01.mirrors.aliyuncs.com"],

    "insecure-registries":["10.24.2.30:5000"]

}

EOF
```

##

Docker容器进入的4种方式

在使用Docker创建了容器之后，大家比较关心的就是如何进入该容器了，其实进入Docker容器有好几种方式，这里我们就讲一下常用的几种进入Docker容器的方法。

进入Docker容器比较常见的几种做法如下：

- 使用docker attach
- 使用SSH
- 使用nsenter
- 使用exec

方式1：使用docker attach进入Docker容器

Docker提供了attach命令来进入Docker容器。

接下来我们创建一个守护态的Docker容器，然后使用docker attach命令进入该容器。

```
sudo docker run -itd ubuntu:14.04 /bin/bash
```

然后我们使用docker ps查看到该容器信息，接下来就使用docker attach进入该容器

```
[root@centos1 ~]# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
c1437f4bd302   ubuntu:14.04                        "/bin/bash"             16 seconds ago Up 15 seconds
e9eae999da9    apacherocketmq/rocketmq:4.6.0      "sh mqbroker -c /opt..." 7 days ago    Up 3 hours    9876/tcp, 10909/tcp
p, 10911-10912/tcp, 0.0.0.0:20921-20922->20921-20922/tcp, :::20921-20922->20921-20922/tcp
fd4e13f28ada    apacherocketmq/rocketmq:4.6.0      "sh mqbroker -c /opt..." 7 days ago    Up 3 hours    9876/tcp, 10909/tcp
p, 10911-10912/tcp, 0.0.0.0:20911-20912->20911-20912/tcp, :::20911-20912->20911-20912/tcp
671b7e7357d3    apacherocketmq/rocketmq:4.6.0      "sh mqnamesrv"           7 days ago    Up 3 hours    10909/tcp, 10911-1
0912/tcp, 0.0.0.0:19876->9876/tcp, :::19876->9876/tcp
4767f0e9f73c    styletang/rocketmq-console-ng      "sh -c 'java $JAVA_O..." 7 days ago    Up 3 hours    0.0.0.0:29001->900
1/tcp, :::29001->9001/tcp
cb864706809e    apacherocketmq/rocketmq:4.6.0      "sh mqnamesrv"           7 days ago    Up 3 hours    10909/tcp, 10911-1
0912/tcp, 0.0.0.0:19877->9876/tcp, :::19877->9876/tcp
47af0e699185    goharbor/harbor-log:v2.3.2         "/bin/sh -c /usr/loc..." 8 days ago    Up 3 hours (healthy)
514/tcp
0344212df5d6    registry.cn-hangzhou.aliyuncs.com/zhengqing/redis:6.0.8 "docker-entrypoint.s..." 9 days ago    Up 3 hours
/tcp, :::6379->6379/tcp
1d00f3ade2b6    mysql:5.7                           "docker-entrypoint.s..." 4 weeks ago    Up 3 hours    0.0.0.0:3306->3306
/tcp, :::3306->3306/tcp, 33060/tcp
129a46feed77    zookeeper:3.4.13                   "/docker-entrypoint..." 4 weeks ago    Up 3 hours    0.0.0.0:2181->2181
/tcp, :::2181->2181/tcp, 0.0.0.0:2888->2888/tcp, :::2888->2888/tcp, 0.0.0.0:3888->3888/tcp, :::3888->3888/tcp
zookeeper
[root@centos1 ~]#
```

```
docker attach c1437f4bd302
```

```
[root@centos1 ~]# docker attach c1437f4bd302
root@c1437f4bd302: /#
root@c1437f4bd302: /#
```

本pdf《Docker学习圣经》不断更新

领取pdf 公众号：《技术自由圈》

CSDN @40岁资深老架构师尼恩

可以看到我们已经进入到该容器中了。

但在，使用该命令有一个问题：

当多个窗口同时使用该命令进入该容器时，所有的窗口都会同步显示。

如果有一个窗口阻塞了，那么其他窗口也无法再进行操作。

因为这个原因，所以docker attach命令不太适合于生产环境，平时自己开发应用时可以使用该命令。

方式2：使用SSH进入Docker容器

在生产环境中排除了使用docker attach命令进入容器之后，相信大家第一个想到的就是ssh。

在镜像（或容器）中安装SSH Server，这样就能保证多人进入。

容器且相互之间不受干扰了，相信大家在当前的生产环境中（没有使用Docker的情况）也是这样做的。

但是使用了Docker容器之后不建议使用ssh进入到Docker容器内。

方式3：使用nsenter进入Docker容器

在上面两种方式都不适合的情况下，还有一种比较方便的方法，即使用nsenter进入Docker容器。

1 什么是nsenter？

nsenter命令是一个可以在指定进程的命令空间下运行指定程序的命令。它位于util-linux包中。

util-linux 是一个开放源码的软件包，是一个对任何 Linux 系统的基本工具套件。含有一些标准 Unix 工具，如 login。

util-linux 软件包包含许多工具。其中比较重要的是加载、卸载、格式化、分区和管理硬盘驱动器，打开 tty 端口和得到内核消息。

nsenter用途？

一个最典型的用途就是进入容器的网络命令空间。相当多的容器为了轻量级，是不包含较为基础的命令的，比如说ip address, ping, telnet, ss, [tcpdump](#)等等命令，这就给调试容器网络带来相当大的困扰：只能通过docker inspect ContainerID命令获取到容器IP，以及无法测试和其他网络的连通性。这时就可以使用nsenter命令仅进入该容器的网络命名空间，使用宿主机的命令调试容器网络。

在了解了什么是nsenter之后，系统默认将我们需要的nsenter安装到主机中， nsenter --help 查看帮助

```
[root@centos1 ~]# nsenter --help

Usage:
nsenter [options] [<program> [<argument>...]]

Run a program with namespaces of other processes.

Options:
-a, --all                enter all namespaces
-t, --target <pid>       target process to get namespaces from
-m, --mount[=<file>]     enter mount namespace
-u, --uts[=<file>]        enter UTS namespace (hostname etc)
-i, --ipc[=<file>]        enter System V IPC namespace
-n, --net[=<file>]        enter network namespace
-p, --pid[=<file>]        enter pid namespace
-C, --cgroup[=<file>]    enter cgroup namespace
-U, --user[=<file>]       enter user namespace
-S, --setuid <uid>        set uid in entered namespace
-G, --setgid <gid>        set gid in entered namespace
    --preserve-credentials do not touch uids or gids
-r, --root[=<dir>]        set the root directory
-w, --wd[=<dir>]          set the working directory
-F, --no-fork             do not fork before exec'ing <program>
-Z, --follow-context      set SELinux context according to --target PID

-h, --help               display this help
-V, --version             display version

For more details see nsenter(1).
```

本pdf《Docker学习圣经》不断更新
领取pdf公众号：《技术自由圈》

CSDN @40岁资深老架构师尼恩

nsenter --help 查看帮助

```
$ nsenter --help

nsenter [options] [program [arguments]]

options:
-t, --target pid: 指定被进入命名空间的目标进程的pid
-m, --mount[=file]: 进入mount命令空间。如果指定了file，则进入file的命令空间
-u, --uts[=file]: 进入uts命令空间。如果指定了file，则进入file的命令空间
-i, --ipc[=file]: 进入ipc命令空间。如果指定了file，则进入file的命令空间
-n, --net[=file]: 进入net命令空间。如果指定了file，则进入file的命令空间
-p, --pid[=file]: 进入pid命令空间。如果指定了file，则进入file的命令空间
-U, --user[=file]: 进入user命令空间。如果指定了file，则进入file的命令空间
-G, --setgid gid: 设置运行程序的gid
```



```
-S, --setuid uid: 设置运行程序的uid
-r, --root[=directory]: 设置根目录
-w, --wd[=directory]: 设置工作目录
```

如果没有给出program, 则默认执行\$SHELL。

2、nsenter安装

如果没有安装的话, 按下面步骤安装即可 (注意是主机而非容器或镜像)

具体的安装命令如下:

```
wget https://www.kernel.org/pub/linux/utils/util-linux/v2.24/util-linux-2.24.tar.gz
tar -xzf util-linux-2.24.tar.gz
cd util-linux-2.24/
./configure --without-ncurses
make nsenter
sudo cp nsenter /usr/local/bin
```

3、nsenter的使用

nsenter可以访问另一个进程的名称空间。

所以为了连接到某个容器我们还需要获取该容器的第一个进程的PID。

怎么办呢? 可以使用docker inspect命令来拿到该进程的PID。

docker inspect 命令使用如下:

```
sudo docker inspect --help
```

inspect命令可以分层级显示一个镜像或容器的信息。

比如我们当前有一个正在运行的容器

```
[root@centos1 ~]# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
c1437f4bd302   ubuntu:14.04                        "/bin/bash"             16 seconds ago Up 15 seconds
e9eaf999da9    apacherocketmq/rocketmq:4.6.0      "sh mqbroker -c /opt..." 7 days ago    Up 3 hours    9876/tcp, 10909/tc
fd4e13f20ada    apacherocketmq/rocketmq:4.6.0      "sh mqbroker -c /opt..." 7 days ago    Up 3 hours    9876/tcp, 10909/tc
p, 10911-10912/tcp, 0.0.0.0:20921-20922->20921-20922/tcp, :::20921-20922->20921-20922/tcp
a71b7e7357d3    apacherocketmq/rocketmq:4.6.0      "sh mqnamesrv"          7 days ago    Up 3 hours    10909/tcp, 10911-1
0912/tcp, 0.0.0.0:19876->9876/tcp, :::19876->9876/tcp
4767f0e9f73c    styletang/rocketmq-console-ng      "sh -c 'java $JAVA_O..." 7 days ago    Up 3 hours    0.0.0.0:29001->900
1/tcp, :::29001->9001/tcp
cb864706800e    apacherocketmq/rocketmq:4.6.0      "sh mqnamesrv"          7 days ago    Up 3 hours    10909/tcp, 10911-1
0912/tcp, 0.0.0.0:19877->9876/tcp, :::19877->9876/tcp
47af8e699185    goharbor/harbor-log:v2.3.2         "/bin/sh -c /usr/loc..." 8 days ago    Up 3 hours (healthy) 127.0.0.1:1514->10
514/tcp
0344212df5d6    registry.cn-hangzhou.aliyuncs.com/zhengqing/redis:6.0.8 "docker-entrypoint.s..." 9 days ago    Up 3 hours    0.0.0.0:6379->6379
1/tcp, :::6379->6379/tcp
1d00f3ade2b6    mysql:5.7                            "docker-entrypoint.s..." 4 weeks ago    Up 3 hours    0.0.0.0:3306->3306
/tcp, :::3306->3306/tcp
129a46feed77    zookeeper:3.4.13                    "docker-entrypoint.s..." 4 weeks ago    Up 3 hours    0.0.0.0:2181->2181
/tcp, :::2181->2181/tcp, 0.0.0.0:2888->2888/tcp, :::2888->2888/tcp, 0.0.0.0:3888->3888/tcp, :::3888->3888/tcp
[root@centos1 ~]#
```

可以使用docker inspect来查看该容器的详细信息。

```
sudo docker inspect c1437f4bd302
```

```
[root@centos1 ~]# sudo docker inspect c1437f4bd302
[
  {
    "Id": "c1437f4bd302184d28165e44559afcdefcfa07a3972905ad54af05e020d124e5",
    "Created": "2023-03-12T07:29:47.271235067Z",
    "Path": "/bin/bash",
    "Args": [],
    "State": {
      "Status": "running",
      "Running": true,
      "Paused": false,
      "Restarting": false,
      "OOMKilled": false,
      "Dead": false,
      "Pid": 22299,
      "ExitCode": 0,
      "Error": "",
      "StartedAt": "2023-03-12T07:29:47.974033748Z",
      "FinishedAt": "0001-01-01T00:00:00Z"
    },
    "Image": "sha256:13b66b487594a1f2b75396013bc05d29d9f527852d96c5577cc4f187559875d0",
    "ResolvConfPath": "/var/lib/docker/containers/c1437f4bd302184d28165e44559afcdefcfa07a3972905ad54af05e020d124e5/resolv.",
    "HostnamePath": "/var/lib/docker/containers/c1437f4bd302184d28165e44559afcdefcfa07a3972905ad54af05e020d124e5/hostname",
    "HostsPath": "/var/lib/docker/containers/c1437f4bd302184d28165e44559afcdefcfa07a3972905ad54af05e020d124e5/hosts",
    "LogPath": "/var/lib/docker/containers/c1437f4bd302184d28165e44559afcdefcfa07a3972905ad54af05e020d124e5/c1437f4bd302184d28165e44559afcdefcfa07a3972905ad54af05e020d124e5-json.log",
    "Name": "/xenodochial_khayyam",
    "RestartCount": 0,
    "Driver": "overlay2",
    "Platform": "linux",
    "MountLabel": "",
    "ProcessLabel": "",
    "AppArmorProfile": "",
    "ExecIDs": null,
    "HostConfig": {
      "Binds": null,
      "ContainerIDFile": "",
      "LogConfig": {
        "Type": "json-file",
        "Config": {}
      },
      "NetworkMode": "bridge",
      "PortBindings": {},
      "RestartPolicy": {
        "Name": "no",
        "MaximumRetryCount": 0
      },
      "SecurityOpt": null,
      "StorageOpt": null,
      "VolumeDriver": "local",
      "VolumesFrom": null
    },
    "NetworkSettings": {
      "Bridge": "br-172.17.0.1",
      "SandboxID": "d87b3831349820b7bfbcc44026669a7ad183b6cc4909e24e58e3c5102a07bef41",
      "Gateway": "172.17.0.1",
      "IPAddress": "172.17.0.2",
      "IPPrefixLen": 16,
      "IPv6Gateway": "",
      "GlobalIPv6Address": "",
      "GlobalIPv6PrefixLen": 0,
      "MacAddress": "02:42:ac:11:00:02",
      "DriverOpts": null
    }
  }
]
```

本pdf《Docker 学习圣经》不断更新
领取pdf 公众号：《技术自由圈》
CSDN @40岁资深老架构师尼恩

由其该信息非常多，此处只截取了其中一部分进行展示。如果要显示该容器第一个进行的PID可以使用如下方式

```
sudo docker inspect -f {{.State.Pid}} c1437f4bd302
```

```
NetworkID": "bc5d01541ea72f66c052f049d9cda2a58517a161df66e565b627580ed165377e",
"EndpointID": "d87b3831349820b7bfbcc44026669a7ad183b6cc4909e24e58e3c5102a07bef41",
"Gateway": "172.17.0.1",
"IPAddress": "172.17.0.2",
"IPPrefixLen": 16,
"IPv6Gateway": "",
"GlobalIPv6Address": "",
"GlobalIPv6PrefixLen": 0,
"MacAddress": "02:42:ac:11:00:02",
"DriverOpts": null
}
}
}
}
]
[root@centos1 ~]#
[root@centos1 ~]# sudo docker inspect -f {{.State.Pid}} c1437f4bd302
22299
[root@centos1 ~]#
```

本pdf《Docker 学习圣经》不断更新
领取pdf 公众号：《技术自由圈》
CSDN @40岁资深老架构师尼恩

在拿到该进程PID之后我们就可以使用nsenter命令访问该容器了。

```
sudo nsenter --target 22299 --mount --uts --ipc --net --pid
```

其中的 22299 即刚才拿到的进程的PID

输入该命令便进入到容器中

```
$ nsenter --target 上面查到的进程id --mount --uts --ipc --net --pid
```

解释nsenter指令中进程id之后的参数的含义：

- mount参数是进入到mount namespace中 (文件系统)
- uts参数是进入到uts namespace中 (主机名与域名)
- ipc参数是进入到System V IPC namespace中 (信号量、消息队列和共享内容)
- net参数是进入到network namespace中 (网络设备、网络栈、端口)
- pid参数是进入到pid namespace中 (进程编号)
- user参数是进入到user namespace中 (用户和用户组)

看看下面的例子，进入到容器的 network namespace中，看看IP地址，是不是变了。

```
[root@centos1 ~]# nsenter --target 22299 --net --pid
[root@centos1 ~]# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 172.17.0.2 netmask 255.255.0.0 broadcast 172.17.255.255
    ether 02:42:ac:11:00:02 txqueuelen 0 (Ethernet)
    RX packets 34 bytes 2753 (2.6 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

本pdf《Docker 学习圣经》不断更新
领取pdf 公众号：《技术自由圈》
CSDN @40岁资深老架构师尼恩

docker隔离应用应用涉及到的六大名称空间

1、pid 命名空间(进程ID)

不同用户的进程就是通过 pid 命名空间隔离开的，且不同命名空间中可以有相同 pid。

所有的 LXC (Linux 容器) 进程在 Docker 中的父进程为 Docker 进程，每个 LXC 进程具有不同的命名空间。

同时由于允许嵌套，因此可以很方便的实现嵌套的 Docker 容器。

2、net 命名空间(网络)

有了 pid 命名空间，每个命名空间中的 pid 能够相互隔离，但是网络端口还是共享 host 的端口。

网络隔离是通过 net 命名空间实现的，每个 net 命名空间有独立的网络设备，IP 地址，路由表，/proc/net 目录。这样每个容器的网络就能隔离开来。

Docker 默认采用 veth 的方式，将容器中的虚拟网卡同 host 上的一个 Docker 网桥 docker0 连接在一起。

3、ipc 命名空间(进程间通信)

容器中进程交互还是采用了 Linux 常见的进程间交互方法(interprocess communication - IPC)，包括信号量、消息队列和共享内存等。

然而同 VM 不同的是，容器的进程间交互实际上还是 host 上具有相同 pid 命名空间中的进程间交互，因此需要在 IPC 资源申请时加入命名空间信息，每个 IPC 资源有一个唯一的 32 位 id。

4、mnt 命名空间(挂载文件系统)

类似 chroot，将一个进程放到一个特定的目录执行。

mnt 命名空间允许不同命名空间的进程看到的文件结构不同，这样每个命名空间中的进程所看到的文件目录就被隔离开了。

同 chroot 不同，每个命名空间中的容器在 /proc/mounts 的信息只包含所在命名空间的 mount point。

5、UTS 命名空间(主机名/域名)

UTS("UNIX Time-sharing System") 命名空间允许每个容器拥有独立的 hostname 和 domain name，使其在网络上可以被视作一个独立的节点而非 主机上的一个进程。

6、user 命名空间(用户)

每个容器可以有不同的用户和组 id，也就是说可以在容器内用容器内部的用户执行程序而非主机上的用户。

nsenter查看docker的连接

由于使用DOCKER的时候，ESTABLISHED连接不会出现在netstat中，在运行中的docker容器中列出打开的套接字的方法

查看连接：

```
sudo docker inspect -f {{.State.Pid}}      e9eae999da9

$ sudo nsenter -t 3473 -n netstat | grep ESTABLISHED
```

```
[root@centos1 ~]# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS
c1437f4bd302   ubuntu:14.04                        "/bin/bash"             24 minutes ago Up 24 minutes
e9eae999da9     apacherocketmq/rocketmq:4.6.0       "sh mqbroker -c /opt..." 7 days ago    Up 4 hours
cp, 10911-10912/tcp, 0.0.0.0:20921-20922->20921-20922/tcp, :::20921-20922->20921-20922/tcp    Up 4 hours
fd4e13f20ada     apacherocketmq/rocketmq:4.6.0       "sh mqbroker -c /opt..." 7 days ago    Up 4 hours
cp, 10911-10912/tcp, 0.0.0.0:20911-20912->20911-20912/tcp, :::20911-20912->20911-20912/tcp    Up 4 hours
a71b7e7357d3     apacherocketmq/rocketmq:4.6.0       "sh mqnamesrv"          7 days ago    Up 4 hours
10912/tcp, 0.0.0.0:19876->9876/tcp, :::19876->9876/tcp    Up 4 hours
4767f0e9f73c     stylatang/rocketmq-console-ng       "sh -c 'java $JAVA_O..." 7 days ago    Up 4 hours
91/tcp, :::29001->9001/tcp                                Up 4 hours
cb864706800e     apacherocketmq/rocketmq:4.6.0       "sh mqnamesrv"          7 days ago    Up 4 hours
10912/tcp, 0.0.0.0:19877->9876/tcp, :::19877->9876/tcp    Up 4 hours
47af8e699185     goharbor/harbor-log:v2.3.2          "/bin/sh -c /usr/loc..." 8 days ago    Up 4 hours (health
9514/tcp                                                  harbor-log
9344212df5d6     registry.cn-hangzhou.aliyuncs.com/zhengqing/redis:6.0.8 "docker-entrypoint.s..." 9 days ago    Up 4 hours
9/tcp, :::6379->6379/tcp                                redis-standalone
1d00f3ade2b6     mysql:5.7                           "docker-entrypoint.s..." 4 weeks ago    Up 4 hours
6/tcp, :::3306->3306/tcp, 33060/tcp                      mysql
129a46feed77     zookeeper:3.4.13                    "/docker-entrypoint.s..." 4 weeks ago    Up 4 hours
1/tcp, :::2181->2181/tcp, 0.0.0.0:2888->2888/tcp, :::2888->2888/tcp, 0.0.0.0:3888->3888/tcp, :::3888->3888/tcp    zookeeper
[root@centos1 ~]# e9eae999da9sudo docker inspect -f {{.State.Pid}}      e9eae999da9
-bash: e9eae999da9sudo: command not found
[root@centos1 ~]# sudo docker inspect -f {{.State.Pid}}      e9eae999da9
3473
[root@centos1 ~]# sudo nsenter -t 3473 -n netstat | grep ESTABLISHED
tcp        0      0 centos1:44284      cdh1:19876        ESTABLISHED
tcp        0      0 centos1:58222      cdh1:19877        ESTABLISHED
```

本pdf《Docker 学习圣经》不断更新
领取pdf 公众号：《技术自由圈》
CSDN @40岁资深老架构师尼恩

方式4：使用docker exec进入Docker容器

除了上面几种做法之外，docker在1.3.X版本之后还提供了一个新的命令exec用于进入容器，这种方式相对更简单一些，下面我们来看一下该命令的使用：

```
sudo docker exec --help
```

```
[root@centos1 ~]# sudo docker exec --help
Usage: docker exec [OPTIONS] CONTAINER COMMAND [ARG...]
Run a command in a running container
Options:
  -d, --detach            Detached mode: run command in the background
  --detach-keys string    Override the key sequence for detaching a container
  -e, --env list          Set environment variables
  --env-file list         Read in a file of environment variables
  -i, --interactive       Keep STDIN open even if not attached
  --privileged            Give extended privileges to the command
  -t, --tty               Allocate a pseudo-TTY
  -u, --user string       Username or UID (format: <name|uid>[:<group|gid>])
  -w, --workdir string    Working directory inside the container
```

本pdf《Docker 学习圣经》不断更新
领取pdf 公众号：《技术自由圈》

CSDN @40岁资深老架构师尼恩

接下来我们使用该命令进入一个已经在运行的容器

```
sudo docker ps
sudo docker exec -it 容器id /bin/bash
```

```
[root@centos1 ~]# sudo docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        NAMES
c1437f4bd302   ubuntu:14.04                       "/bin/bash"             27 minutes ago Up 27 minutes xenodochial_khayyam
e9eae999da9    apacherocketmq/rocketmq:4.6.0      "sh mqbroker -c /opt..." 7 days ago    Up 4 hours    rmqbroker-ha-b
cp, 10911-10912/tcp, 0.0.0.0:20921-20922->20921-20922/tcp, ::20921-20922->20921-20922/tcp
fd4e13f20ada    apacherocketmq/rocketmq:4.6.0      "sh mqbroker -c /opt..." 7 days ago    Up 4 hours    rmqbroker-ha-a
cp, 10911-10912/tcp, 0.0.0.0:20911-20912->20911-20912/tcp, ::20911-20912->20911-20912/tcp
a71b7e7357d3    apacherocketmq/rocketmq:4.6.0      "sh mqnamesrv"           7 days ago    Up 4 hours    rmqnamesrv-ha-a
10912/tcp, 0.0.0.0:19876->9876/tcp, ::19876->9876/tcp
4767f0a9f73c    styletang/rocketmq-console-ng      "sh -c 'java $JAVA_O..." 7 days ago    Up 4 hours    rmqconsole-ha
01/tcp, ::29001->9001/tcp
cb864706800e    apacherocketmq/rocketmq:4.6.0      "sh mqnamesrv"           7 days ago    Up 4 hours    rmqnamesrv-ha-b
10912/tcp, 0.0.0.0:19877->9877/tcp, ::19877->9877/tcp
47af8e699185    goharbor/harbor-log:v2.3.2         "/bin/sh -c /usr/loc..." 8 days ago    Up 4 hours    harbor-log
0514/tcp
0344212df5d6    registry.cn-hangzhou.aliyuncs.com/zhengqing/redis:6.0.8 "docker-entrypoint.s..." 10 days ago    Up 4 hours    redis-standalone
9/tcp, ::6379->6379/tcp
1d00f3ade2b6    mysql:5.7                           "docker-entrypoint.s..." 4 weeks ago    Up 4 hours    mysql
6/tcp, ::3306->3306/tcp, 33060/tcp
129a46feed77    zookeeper:3.4.13                   "/docker-entrypoint.s..." 4 weeks ago    Up 4 hours    zookeeper
1/tcp, ::2181->2181/tcp, 0.0.0.0:2888->2888/tcp, ::2888->2888/tcp, 0.0.0.0:3888->3888/tcp, ::3888->3888/tcp
[root@centos1 ~]# sudo docker exec -it e9eae999da9 /bin/bash
[rocketmq@e9eae999da9 bin]$
```

本pdf《Docker 学习圣经》不断更新
领取pdf 公众号：《技术自由圈》

CSDN @40岁资深老架构师尼恩

在容器内部和宿主机中查看容器中的进程信息

进入一个名称为 rmqbroker-ha-b 的容器，查看进程信息

```
docker exec -it rmqbroker-ha-b /bin/bash ps -ef
```

结果如下：

```
[root@centos1 ~]# docker exec -it rmqbroker-ha-b ps -ef
UID          PID    PPID    C   STIME TTY          TIME CMD
rocketmq      1      0   0 12:06 ?        00:00:00 sh mqbroker -c /opt/rocketmq
rocketmq      7      1   0 12:06 ?        00:00:00 sh /home/rocketmq/rocketmq-4
rocketmq     28      7   0 12:06 ?        00:01:32 /usr/lib/jvm/java-1.8.0-open
rocketmq     178     0 12 16:01 pts/0    00:00:00 ps -ef
```

本pdf《Docker 学习圣经》不断更新
领取pdf 公众号：《技术自由圈》

CSDN @40岁资深老架构师尼恩

我们可以使用 `docker exec` 命令进入容器PID名空间，并执行应用。

通过 `ps -ef` 命令，可以看到每个 rmqbroker-ha-b 容器都包含一个PID为1的进程，容器的启动进程是 "sh mqbroker -c /opt/rocketmq....."，具有特殊意义。

利用 `docker top` 命令，可以让我们从宿主机操作系统中看到容器的进程信息。

```
[root@centos1 ~]# docker top rmqbroker-ha-b
UID          PID    PPID    C   STIME TTY          TIME CMD
3000         3473    3401     0   12:06 ?            00:00:00 sh mqbroker -c
/opt/rocketmq-4.6.0/conf/broker.conf autoCreateTopicEnable=true &
3000         4383    3473     0   12:06 ?            00:00:00 sh /home/rocket
mq/rocketmq-4.6.0/bin/runbroker.sh org.apache.rocketmq.broker.BrokerStartup -c /opt/rocketmq-4.6.0/conf/broker.conf autoCreateTopicEnable=true &
3000         4473    4383     0   12:06 ?            00:01:34 /usr/lib/jvm/ja
va-1.8.0-openjdk-1.8.0.232.b09-0.el7_7.x86_64/bin/java -server -Xms1952m -Xmx1952m -Xmn400m -XX:+UseG1GC -XX:G1HeapRegionSize=16m -XX:G1ReservePercent=25 -X
XX:InitiatingHeapOccupancyPercent=30 -XX:SoftRefLRUPolicyMSPerMB=0 -XX:SurvivorRatio=8 -verbose:gc -Xloggc:/dev/shm/mq_gc_%p.log -XX:+PrintGCDetails -XX:+P
rintGCDateStamps -XX:+PrintGCApplicationStoppedTime -XX:+PrintAdaptiveSizePolicy -XX:-UseGCLogFileRotation -XX:NumberOfGCLogFiles=5 -XX:GCLogFileSize=30m -
XX:-OmitStackTraceInFastThrow -XX:+AlwaysPreTouch -XX:MaxDirectMemorySize=1952m -XX:-UseLargePages -XX:-UseBiasedLocking -Djava.ext.dirs=/usr/lib/jvm/java-
1.8.0-openjdk-1.8.0.232.b09-0.el7_7.x86_64/jre/lib/ext:/home/rocketmq/rocketmq-4.6.0/bin/./lib -server -Xms256m -Xmx512m -Xmn128m -cp ./home/rocketmq/roc
ketmq-4.6.0/bin/./conf:org.apache.rocketmq.broker.BrokerStartup -c /opt/rocketmq-4.6.0/conf/broker.conf autoCreateTopicEnable=true &
```

本pdf《Docker 学习圣经》不断更新
领取pdf 公众号：《技术自由圈》

CSDN @40岁资深老架构师尼恩

"sh mqbroker -c /opt/rocketmq....." 这个命令的进程，在容器里边是1，在容器外部 3473

查看其父进程信息

ps -ef | grep 3401

```
[root@centos1 ~]# ps -ef | grep 3401
root      3401      1  0 12:06 ?        00:00:00 /usr/bin/containerd-shim-runc-v2 -namespace moby -id e9eae999da9183b9be0b3239881bc6b9c2070f13057c322dfed3d072820e962 -address /run/containerd/containerd.sock
3000      3473     3401  0 12:06 ?        00:00:00 sh mqbroker -c /opt/rocketmq-4.6.0/conf/broker.conf autoCreateTopicEnable=true &
root      26491    24084  0 16:16 pts/1    00:00:00 grep --color=auto 3401
[root@centos1 ~]#
```

本pdf《Docker 学习圣经》不断更新

领取pdf 公众号：《技术自由圈》

CSDN @40岁资深老架构师尼恩

```
root      3401      1  0 12:06 ?        00:00:00 /usr/bin/containerd-shim-runc-v2 -namespace moby -id e9eae999da9183b9be0b3239881bc6b9c2070f13057c322dfed3d072820e962 -address /run/containerd/containerd.sock
3000      3473     3401  0 12:06 ?        00:00:00 sh mqbroker -c /opt/rocketmq-4.6.0/conf/broker.conf autoCreateTopicEnable=true &
root      26491    24084  0 16:16 pts/1    00:00:00 grep --color=auto 3401
```

我们使用docker run 启用一个容器时，docker 会给每个容器都启动一个containerd-shim-runc-v2 父进程，这个进程又启动了一个ttrpc server(类似grpc/httpserver)，containerd 通过 ttrpc和containerd-shim-runc-v2 通信来管理容器

从父亲进程可以看到：容器的本质是进程。

containerd-shim-runc-v2 后面的参数：namespace用来做命名空间隔离，cgroup用来做资源限制。

containerd-shim-runc-v2 进程很特殊，它们跑在一些特定的namespace和cgroup下。

站在这些进程的角度看，它们会以为自己跑在一个独立的机器上，看不到其他进程，也看不到其他文件。

这是其实是操作系统为它虚拟出来的一个独立的、隔离的环境，是假的。

查看子进程信息

```
[root@VM-4-17-centos ~]# ps aux | grep 27880
```

```
mq/rocketmq-4.6.0/bin/runbroker.sh org.apache.rocketmq.broker.BrokerStartup -c /opt/rocketmq-4.6.0/conf/broker.conf autoCreateTopicEnable=true &
3000      4473      0  0 12:06 ?        00:01:44 /usr/lib/jvm/java-1.8.0-openjdk-1.8.0.232.b09-0.el7_7.x86_64/bin/java -server -Xms1952M -Xmx1952M -Xmn400M -XX:+UseG1GC -XX:G1HeapRegionSize=16m -XX:G1ReservePercent=25 -XX:InitiatingHeapOccupancyPercent=30 -XX:SoftRefLRUPolicyMSPerMB=0 -XX:SurvivorRatio=8 -verbose:gc -Xloggc:/dev/shm/mq_gc_%p.log -XX:+PrintGCDetails -XX:PrintGCDateStamps -XX:+PrintGCApplicationStoppedTime -XX:+PrintAdaptiveSizePolicy -XX:+UseGCLogFileRotation -XX:NumberOfGCLogFiles=5 -XX:GCLogFileSize=30m -XX:-OmitStackTraceInFastThrow -XX:+AlwaysPreTouch -XX:MaxDirectMemorySize=1952M -XX:+UseLargePages -XX:-UseBiasedLocking -Djava.ext.dirs=/usr/lib/jvm/java-1.8.0-openjdk-1.8.0.232.b09-0.el7_7.x86_64/jre/lib/ext:/home/rocketmq/rocketmq-4.6.0/bin/../lib -server -Xms256m -Xmx512m -Xmn128m -cp ./home/rocketmq/rocketmq-4.6.0/bin/../conf:org.apache.rocketmq.broker.BrokerStartup -c /opt/rocketmq-4.6.0/conf/broker.conf autoCreateTopicEnable=true &
[root@centos1 ~]# ps aux | grep 3473
3000      3473      0  0 11:52 2500 ?        Ss   12:06   0:00 sh mqbroker -c /opt/rocketmq-4.6.0/conf/broker.conf autoCreateTopicEnable=true &
root      27221    0.0  0.0 221924 1176 pts/1    S+   16:25   0:00 grep --color=auto 3473
[root@centos1 ~]#
```

CSDN @40岁资深老架构师尼恩

总计三个命令

```
docker top rmqbroker-ha-b #查看容器进程
```

```
ps -ef | grep 3401 #查看父进程
```

```
ps aux | grep 3473 #查看子进程(容器)
```


Docker本地镜像载入与载出

两种办法

- 保存镜像（保存镜像载入后获得跟原镜像id相同的镜像）
- 保存容器（保存容器载入后获得跟原镜像id不同的镜像）

拉取镜像

通过命令可以从镜像仓库中拉取镜像，默认从[Docker Hub](https://hub.docker.com/) 获取。

命令格式：

docker image pull :

```
docker image pull rancher/rke-tools:v0.1.52
```

```
[rancher/rke-tools:v0.1.52
```

保存镜像

- docker save 镜像id -o /home/mysql.tar
- docker save 镜像id > /home/mysql.tar

```
docker save docker.io/rancher/rancher-agent -o /home/rancher-agent.tar
```

```
docker save f29ece87a195 -o /home/rancher-agent.tar
```

```
docker save docker.io/rancher/rke-tools -o /home/rke-tools-v0.1.52.tar
```

载入镜像

- docker load -i mysql.tar

```
docker load -i /usr/local/rancher-v2.3.5.tar
```

```
docker load -i /usr/local/rancher-agent.tar
```

```
docker inspect f29ece87a1954772accb8a2332ee8c3fe460697e3f102ffbd76eb9bc4f4f1d0
```

```
docker load -i /usr/local/rke-tools-v0.1.52.tar
```

```
docker load -i mysql.tar
```

```
[root@localhost ~]# docker load -i /usr/local/rancher-v2.3.5.tar
```

```

43c67172d1d1: Loading layer
[=====>] 65.57MB/65.57MB
21ec61b65b20: Loading layer
[=====>] 991.2kB/991.2kB
1d0dfb259f6a: Loading layer
[=====>] 15.87kB/15.87kB
f55aa0bd26b8: Loading layer
[=====>] 3.072kB/3.072kB
e0af200d6950: Loading layer
[=====>] 126.1MB/126.1MB
088ed892f9ad: Loading layer
[=====>] 6.656kB/6.656kB
6aa3142b4130: Loading layer
[=====>] 34.5MB/34.5MB
f4e84c05ab29: Loading layer
[=====>] 70.41MB/70.41MB
11a6e4332b53: Loading layer
[=====>] 224.8MB/224.8MB
46d1ac556da7: Loading layer
[=====>] 3.072kB/3.072kB
0f8b224a5802: Loading layer
[=====>] 57.87MB/57.87MB
519eba7d586a: Loading layer
[=====>] 99.58MB/99.58MB
3f8bb7c0c150: Loading layer
[=====>] 4.608kB/4.608kB
c22c9a5a8211: Loading layer
[=====>] 3.072kB/3.072kB
Loaded image: rancher/rancher:v2.3.5

```

打个tag

```
docker tag f29ece87a1954772accb8a2332ee8c3fe460697e3f102ffbd76eb9bc4f4f1d0
rancher/rancher-agent:v2.3.5
```

```
docker tag f29ece87a195 172.18.8.104/rancher/rancher-agent:v2.3.5
```

```
docker tag 6e421b8753a2 172.18.8.104/rancher/rke-tools:v0.1.52
83fe4871cf67
```

```
docker rmi image_name
```

```
docker rmi -f 172.18.8.104/rancher/coredns-coredns:1.6.5
```

```
docker rmi -f 172.18.8.104/rancher/coredns-coredns:v3.4.3-rancher1
```

```
docker rmi hub.dodge.net/ubuntu:latest
```


保存镜像

- `docker export 镜像id -o /home/mysql-export.tar`
- `docker save 镜像tag -o /home/mysql-export.tar`

载入镜像

- `docker import mysql-export.tar`

Harbor私有镜像仓库

Harbor（港口，港湾）是一个用于存储和分发Docker镜像的企业级Registry服务器。

除了Harbor这个私有镜像仓库之外，还有Docker官方提供的Registry。

相对Registry，Harbor具有很多优势：

1. 提供分层传输机制，优化网络传输 Docker镜像是是分层的，而如果每次传输都使用全量文件(所以用FTP的方式并不适合)，显然不经济。必须提供识别分层传输的机制，以层的UUID为标识，确定传输的对象。
2. 提供WEB界面，优化用户体验 只用镜像的名字来进行上传下载显然很不方便，需要有一个用户界面可以支持登陆、搜索功能，包括区分公有、私有镜像。
3. 支持水平扩展集群 当有用户对镜像的上传下载操作集中在某服务器，需要对相应的访问压力作分解。
4. 良好的安全机制 企业中的开发团队有很多不同的职位，对于不同的职位人员，分配不同的权限，具有更好的安全性。

Harbor安装

harbor是用过docker-compose 编排的。

所以 安装的过程中，会检查docker、docker-compose 进程，确保提前启动。

这些，在咱们的虚拟机里边，docker、docker-compose 进程已经预装好了的。

查看docker是否安装成功；

```
[root@centos1 ~]# docker -v
Docker version 20.10.23, build 7155243
```

查看docker-compose是否安装成功；

```
docker-compose -version

[root@centos1 ~]# docker-compose -version
docker-compose version 1.25.1, build a82fef07
[root@centos1 ~]#
```

接下来，开始安装harbor

1 下载 Harbor的压缩包；

```
https://github.com/goharbor/harbor/releases
```

咱们用这个包： harbor-offline-installer-v2.3.2.tgz

咱们的学习网盘当中，尼恩已经上传了哈

2 上传压缩包到虚拟机，并解压；

```
[root@centos1 ~]# cd /usr/local/  
[root@centos1 local]# mkdir harbor  
[root@centos1 local]# cd harbor/  
  
tar -zxvf harbor-offline-installer-v2.3.2.tgz
```

```
[root@centos1 harbor]# tar -zxvf harbor-offline-installer-v2.3.2.tgz  
harbor/harbor.v2.3.2.tar.gz  
harbor/prepare  
harbor/LICENSE  
harbor/install.sh  
harbor/common.sh  
harbor/harbor.yml.tpl
```

CSDN @40岁资深老架构师尼恩

3 创建harbor访问域名证书

OpenSSL是一个强大的安全套接字层密码库，Apache使用它加密HTTPS，OpenSSH使用它加密SSH，但是，你不应该只将其作为一个库来使用，它还是一个多用途的、跨平台的密码工具。

参考：

[Harbor docs | Configure HTTPS Access to Harbor \(goharbor.io\)](https://goharbor.io/docs/1.10.0/Configure-HTTPS-Access-to-Harbor/)

```
mkdir -p /usr/local/harbor/ssl && cd /usr/local/harbor/ssl
openssl genrsa -out tls.key 4096

openssl req -x509 -new -nodes -sha512 -days 3650 \
-subj "/C=CN/ST=Beijing/L=Beijing/O=example/OU=Personal/CN=cdh1" \
-key tls.key \
-out tls.cert
```

- 第一步创建ssl文件夹用来存储证书,
- 第二步生成key (私钥),
- 最后一步使用生成的key自签证书。自签证书包含公钥

days后面是你自签证书的有效时间可以自行修改。

'CN='后面就写你自己的IP地址或者你自己的域名。

生成完成之后显示如下

```
mkdir -p /usr/local/harbor/ssl && cd /usr/local/harbor/ssl
[root@centos1 harbor]# mkdir -p /usr/local/harbor/ssl && cd /usr/local/harbor/ssl
[root@centos1 ssl]# openssl genrsa -out tls.key 2048
Generating RSA private key, 2048 bit long modulus (2 primes)
.....+++++
.+++++
e is 65537 (0x010001)
[root@centos1 ssl]# openssl req -new -x509 -key tls.key -out tls.cert -days 720 -subj CN=cdh1
name is expected to be in the format /type0=value0/type1=value1/type2=... where characters may be escaped by \. This name is no
problems making Certificate Request
[root@centos1 ssl]# ll
total 4
-rw-r----- 1 root root 1679 Mar  2 16:53 tls.key
[root@centos1 ssl]# openssl req -new -x509 -key tls.key -out tls.cert -days 720 -subj /CN=cdh1
[root@centos1 ssl]# ll
total 8
-rw-r----- 1 root root 1099 Mar  2 16:54 tls.cert
-rw-r----- 1 root root 1679 Mar  2 16:53 tls.key
```

CSDN @40岁资深老架构师尼恩

4 配置harbor

修改Harbor的配置 (harbor.yml) ;

修改主机地址: hostname: 192.168.56.121;

修改端口 (默认端口80) : port: 85;

```
cd /usr/local/harbor/harbor    #进入到harbor目录

cp  harbor.yml.tpl    harbor.yml

vim harbor.yml    #编辑harbor的配置文件

#修改以下内容

hostname = 192.168.56.121    #修改harbor的启动ip, 这里需要依据系统ip设置

port: 85    #harbor的端口, 有两个端口, http协议(80)和https协议(443)

harbor_admin_password = 123456    #修改harbor的admin用户的密码

data_volume: /harbor/data    #修改harbor存储位置
```

故harbor.yml中certificate填写为如上tls.cert的文件目录地址：/usr/local/harbor/ssl/tls.cert

故harbor.yml中private_key填写为如上tls.key的文件目录地址：/usr/local/harbor/ssl/tls.key

```
# Configuration file of Harbor

# The IP address or hostname to access admin UI and registry service.
# DO NOT use localhost or 127.0.0.1, because Harbor needs to be accessed by external clients.
hostname: 192.168.56.121

# http related config
http:
  # port for http, default is 80. If https enabled, this port will redirect to https port
  port: 85

# https related config
https:
  # https port for harbor, default is 443
  port: 443
  # The path of cert and key files for nginx
  certificate: /usr/local/harbor/ssl/tls.cert
  private_key: /usr/local/harbor/ssl/tls.key

# Uncomment following will enable tls communication between all harbor components
# internal tls:
#   # set enabled to true means internal tls is enabled
#   enabled: true
#   # put your cert and key files on dir
#   dir: /etc/harbor/tls/internal

# Uncomment external_url if you want to enable external proxy
# And when it enabled the hostname will no longer used
# external_url: https://reg.mydomain.com:8433

# The initial password of Harbor admin
# It only works in first time to install harbor
# Remember Change the admin password from UI after launching Harbor.
harbor_admin_password: 12345

# Harbor DB configuration
database:
  # The password for the root user of Harbor DB. Change this before any production use.
  password: root123
  # The maximum number of connections in the idle connection pool. If it <= 0, no idle connections are retained.
  max_idle_conns: 100
  # The maximum number of open connections to the database. If it <= 0, then there is no limit on the number of open connections.
  max_open_conns: 0
  # Nothing in register
```

5 ./prepare 准备

启动之前需要使用./prepare命令进行一些预置工作，下载相关依赖；

此时需要开启docker服务，不然会报错；

这个过程可能有点长，需要耐心等待；

```
./prepare
```

```
total 594380
-rw-r--r-- 1 root root 3361 Aug 18 2021 common.sh
-rw-r--r-- 1 root root 60861132 Aug 18 2021 harbor.v2.3.2.tar.gz
-rw-r--r-- 1 root root 7840 Aug 18 2021 harbor.yml.tpl
drwxr-xr-x 2 root root 6 Mar 2 16:00 input
-rwxr-xr-x 1 root root 2500 Aug 18 2021 install.sh
-rw-r--r-- 1 root root 11247 Aug 18 2021 LICENSE
-rwxr-xr-x 1 root root 1801 Aug 18 2021 prepare
[root@centos1 harbor]# cp harbor.yml.tpl harbor.yml
[root@centos1 harbor]# vim harbor.yml
[root@centos1 harbor]# vim harbor.yml
[root@centos1 harbor]# ./prepare
prepare base dir is set to /usr/local/harbor/harbor
Unable to find image 'goharbor/prepare:v2.3.2' locally
v2.3.2: Pulling from goharbor/prepare
4fc7e8c5dda0: Pull complete
e101789e272e: Pull complete
3356ca67bae9: Pull complete
3d33c71b0e90: Pull complete
ab76e2d8736b: Pull complete
9d7f4a6ec28e: Pull complete
2f6837c65c74: Pull complete
bc8d1bca5b85: Pull complete
Digest: sha256:e6ae1e30c8897ca54bcc89682dbb10c2dbd08d6c9bba647eacdef6fa776d6428
Status: Downloaded newer image for goharbor/prepare:v2.3.2
WARNING:root:WARNING: HTTP protocol is insecure. Harbor will deprecate http protocol in the future. Please make sure to upgrade to https
Generated configuration file: /config/portal/nginx.conf
Generated configuration file: /config/log/logrotate.conf
Generated configuration file: /config/log/rsyslog_docker.conf
Generated configuration file: /config/nginx/nginx.conf
Generated configuration file: /config/core/env
Generated configuration file: /config/core/app.conf
Generated configuration file: /config/registry/config.yml
Generated configuration file: /config/registryctl/env
Generated configuration file: /config/registryctl/config.yml
Generated configuration file: /config/db/env
Generated configuration file: /config/jobservice/env
Generated configuration file: /config/jobservice/config.yml
Generated and saved secret to file: /data/secret/keys/secretkey
Successfully called func: create_root_cert
Generated configuration file: /compose_location/docker-compose.yml
Clean up the input dir
[root@centos1 harbor]#
```

 英 · 语 · 中 · 文 · 互 · 译 · 工 · 具
CSDN @40岁资深老架构师尼恩

6 ./install.sh

准备工作完成后，使用./install.sh进行Harbor的安装；
这个过程会持续一段时间，耐心等待；

```
./install.sh
```

```
loaded secret from file: /secret/keys/secretkey
Generated configuration file: /compose_location/docker-compose.yml
Clean up the input dir

[Step 3]: starting Harbor ...
Creating network "harbor_harbor" with the default driver
Creating harbor-log ... done
Creating harbor-db ... done
Creating redis ... done
Creating registryctl ... done
Creating harbor-portal ... done
Creating registry ... done
Creating harbor-core ... done
Creating nginx ... done
Creating harbor-jobservice ... done

✓ ---- Harbor has been installed and started successfully. ----

Now you should be able to visit the admin portal at http://192.168.56.11.
For more details, please visit https://github.com/goharbor/harbor .

[root@centos harbor]#
```

<https://blog.csdn.net/thisIsDennis>

访问

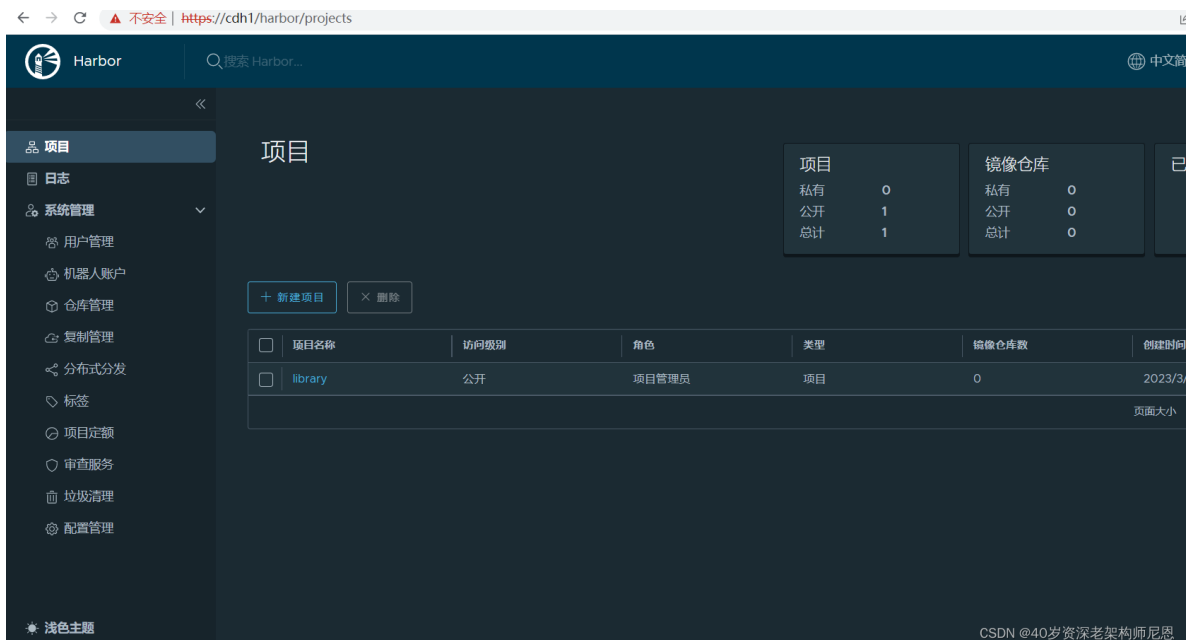
<http://cdh1:85/>

登录

默认登录名：admin；

默认登录密码：Harbor12345；被尼恩改成了 12345

具体可以查看harbor.yml；



7 停止或者重启 Harbor

cd /usr/local/harber/harbor/

```
docker-compose up -d 启动
docker-compose start 启动
docker-compose stop 停止
docker-compose restart 重新启动
```

```
[root@centos1 ~]# cd /usr/local/harber/
[root@centos1 harber]# ll
total 0
drwxr-xr-x. 8 root root 100 Mar  2 16:18 data
drwxr-xr-x. 3 root root 180 Mar  2 16:25 harbor
[root@centos1 harber]# cd /usr/local/harber/data/
[root@centos1 data]# ll
total 0
drwxr-xr-x. 2          10000 10000  6 Mar  2 16:18 ca_download
drwx-----. 3 systemd-coredump input 18 Mar  2 16:19 database
drwxr-xr-x. 2          10000 10000  6 Mar  2 16:18 job_logs
drwxr-xr-x. 2 systemd-coredump input 22 Mar  2 16:29 redis
drwxr-xr-x. 2          10000 10000  6 Mar  2 16:18 registry
drwxr-xr-x. 5 root          root 46 Mar  2 16:18 secret
[root@centos1 data]# cd /usr/local/harber/harbor/
[root@centos1 harbor]# docker-compose up -d
Starting harbor-log ... done
Starting registryctl ... done
Starting redis ... done
Starting harbor-db ... done
Starting registry ... done
Starting harbor-portal ... done
Starting harbor-core ... done
Starting harbor-jobservice ... done
Starting nginx ... done
[root@centos1 harbor]#
```

CSDN @40岁资深老架构师尼恩

修改docker配置文件，使docker支持harbor

编辑客户机/etc/docker/daemon.json文件

```
{"insecure-registries":["192.168.56.121:85"]}
```

重启客户机docker服务

systemctl restart docker #或者(service docker restart)

```
[root@centos1 harbor]# cat /etc/docker/daemon.json
{
  "registry-mirrors": [
    "https://bjtzu1jb.mirror.aliyuncs.com",
    "http://f1361db2.m.daocloud.io",
    "https://hub-mirror.c.163.com",
    "https://docker.mirrors.ustc.edu.cn",
    "https://reg-mirror.qiniu.com",
    "https://dockerhub.azk8s.cn",
    "https://registry.docker-cn.com"
  ],
  "insecure-registries": ["192.168.56.121:85"]
}
```

```
[root@centos1 ssh]# cat /etc/docker/daemon.json
{
  "registry-mirrors": [
    "https://bjtzu1jb.mirror.aliyuncs.com",
    "http://f1361db2.m.daocloud.io",
    "https://hub-mirror.c.163.com",
    "https://docker.mirrors.ustc.edu.cn",
    "https://reg-mirror.qiniu.com",
    "https://dockerhub.azk8s.cn",
    "https://registry.docker-cn.com",
    "https://cdh1"
  ]
}
```

Harbor使用

可以 尝试上传镜像到 Harbor。

如果是另一台机器，需要添加本地hosts

```
echo '192.168.56.121 cdh' >> /etc/hosts
```

因为我们的证书是自签的是不受到其他机器信任的，所以我们要在 harbor的客户端机器上放上我们的证书，才能拉取镜像。

使用下面的代码，在要使用 harbor的客户端机器 上保存证书，其中 cdh1 部分使用你自己的域名或者是IP地址。

```
mkdir -p /etc/docker/certs_d/cdh1
scp 192.168.56.122:/usr/local/harbor/ssl/tls.cert /etc/docker/certs_d/cdh1/
```

本地docker 登录的话

```
mkdir -p /etc/docker/certs.d/cdh1
cp /usr/local/harbor/ssl/tls.cert /etc/docker/certs.d/cdh1/ca.crt

mkdir -p /etc/docker/certs.d/192.168.56.121
```

然后使用如下语句登录harbor，如果没有使用80端口一定要用IP地址加上你的端口号，冒号之后填端口号，查看自己harbor.yml里http或https里的端口详情。

```
docker login cdh1
```

报错 证书问题，证书不含SAN

什么是含有SAN的证书

docker 的新版本使用**golang 1.15+**版本上，老的x509 证书不行了

SAN(Subject Alternative Name) 是 SSL 标准 x509 中定义的一个扩展。使用了 SAN 字段的 SSL 证书，可以扩展此证书支持的域名，使得一个证书可以支持多个不同域名的解析。

subjectAltName 在 RFC 5280 4.2.1.6.中提供了详细的说明，subjectAltName 是 X.509 version 3 的一个扩展项，该扩展项用于标记和界定证书持有者的身份。

在 X.509 格式的证书中，一般使用 `Issuer` 项标记证书的颁发者信息，该项必须是一个非空的 Distinguished Name 名称。除此之外还可以使用扩展项 `issuerAltName` 来标记颁发者的其他名称，这是一个非关键的扩展项。

对于证书持有者，一般使用 `Subject` 项标记，并使用 `subjectAltName` 扩展项提供更详细的持有者身份信息。`subjectAltName` 全称为 Subject Alternative Name，缩写为 SAN。它可以包括一个或者多个的电子邮件地址，域名，IP地址和 URI 等，详细定义如下：

```
SubjectAltName ::= GeneralNames
GeneralNames ::= SEQUENCE SIZE (1..MAX) OF GeneralName

GeneralName ::= CHOICE {
    otherName                [0]     OtherName,
    rfc822Name                [1]     IA5String,
    dNSName                   [2]     IA5String,
```



```

x400Address          [3]      ORAddress,
directoryName        [4]      Name,
ediPartyName         [5]      EDIPartyName,
uniformResourceIdentifier [6]    IA5String,
ipAddress            [7]      OCTET STRING,
registeredID         [8]      OBJECT IDENTIFIER
}

```

当颁发的证书不存在 `Subject` 项的时候，证书必须包含扩展项 `subjectAltName`，并且标记为关键（critical）的。当颁发的证书存在 `Subject` 项的时候，必须将扩展项 `subjectAltName` 标记为非关键（no-critical）的。注意：用于颁发证书的 CA 证书是必须包含 `Subject` 项的。

根据 [RFC 6125](#) 中的规定，当一个网站使用证书标记自己的身份时，如果证书中包含 `subjectAltName`，在识别证书持有者时会忽略 `Subject` 子项，而是通过 `subjectAltName` 来识别证书持有者。

在早期颁发的证书中一般通过 `Subject` 的 `CommonName` 来识别持有者的身份，不包含 `subjectAltName` 扩展项。

这会导致最新版本的浏览器 Chrome、Firefox 等在通过 HTTPS 访问 web 网站时，触发 `NET::ERR_CERT_COMMON_NAME_INVALID` 错误。

SSL证书格式

证书主要的文件类型和协议有: PEM、DER、PFX、JKS、KDB、CER、KEY、CSR、CRT、CRL、OCSP、SCEP等。

1. KEY

一般指PEM格式的私钥文件。

2. CRT

证书文件。可以是PEM格式。

3. PEM

PEM格式的证书文件 (*.pem) 由Base64编码的二进制内容和开头行 (-----BEGIN CERTIFICATE-----)、结束行 (-----END CERTIFICATE-----) 组成，支持使用notepad++等文本编辑器打开。对于CER、CRT格式的证书，您可通过直接修改证书文件扩展名的方式将其转换成PEM格式。例如，将server.crt证书文件直接重命名为server.pem。

4. CSR

证书请求文件(Certificate Signing Request)。生成 X509 数字证书前,一般先由用户提交证书申请文件,然后由 CA 来签发证书。大致过程如下(X509 证书申请的格式标准为 pkcs10 和 rfc2314):

1. 用户生成自己的公私钥对;
2. 构造自己的证书申请文件,符合 PKCS10 标准。该文件主要包括了用户信息、公钥以及一些可选的属性信息,并用自己的私钥给该内容签名;
3. 用户将证书申请文件提交给 CA;
4. CA 验证签名,提取用户信息,并加上其他信息(比如颁发者等信息),用 CA 的私钥签发数字证书;
5. 说明:数字证书(如x.509)是将用户(或其他实体)身份与公钥绑定的信息载体。一个合法的数字证书不仅要符合 X509 格式规范,还必须有 CA 的签名。用户不仅有自己的数字证书,还必须有对应的私

钥。X509v3 数字证书主要包含的内容有:证书版本、证书序列号、签名算法、颁发者信息、有效时间、持有者信息、公钥信息、颁发者 ID、持有者 ID 和扩展项。

6. DER

辨别编码规则 (DER) 可包含所有私钥、公钥和证书。它是大多数浏览器的缺省格式, 并按 ASN1 DER 格式存储。它是无报头的 - PEM 是用文本报头包围的 DER。

6. PFX

公钥加密标准 12 (PKCS12) 可包含所有私钥、公钥和证书。其以二进制格式存储, 也称为 PFX 文件。通常可以将Apache/OpenSSL使用的“KEY文件 + CRT文件”格式合并转换为标准的PFX文件, 你可以将PFX文件格式导入到微软IIS 5/6、微软ISA、微软Exchange Server等软件。转换时需要输入PFX文件的加密密码。

7. JKS

通常可以将Apache/OpenSSL使用的“KEY文件 + CRT文件”格式转换为标准的Java Key Store(JKS)文件。JKS文件格式被广泛的应用在基于JAVA的WEB服务器、应用服务器、中间件。你可以将JKS文件导入到TOMCAT、WEBLOGIC 等软件。

8. KDB

通常可以将Apache/OpenSSL使用的“KEY文件 + CRT文件”格式转换为标准的IBM KDB文件。KDB文件格式被广泛的应用在IBM的WEB服务器、应用服务器、中间件。你可以将KDB文件导入到IBM HTTP Server、IBM Websphere 等软件。

9. OCSP

在线证书状态协议(OCSP,Online Certificate Status Protocol,rfc2560)用于实时表明证书状态。OCSP 客户端通过查询 OCSP 服务来确定一个证书的状态,可以提供给使用者一个或多个数字证书的有效性资料, 它建立一个可实时响应的机制, 让用户可以实时确认每一张证书的有效性, 解决由CRL引发的安全问题。。OCSP 可以通过 HTTP协议来实现。rfc2560 定义了 OCSP 客户端和服务端的消息格式。

10. CER

一般指使用DER格式的证书。

11. CRL

证书吊销列表 (Certification Revocation List) 是一种包含撤销的证书列表的签名数据结构。CRL 是证书撤销状态的公布形式,CRL 就像信用卡的黑名单,用于公布某些数字证书不再有效。CRL 是一种离线的证书状态信息。它以一定的周期进行更新。CRL 可以分为完全 CRL和增量 CRL。在完全 CRL 中包含了所有的被撤销证书信息,增量 CRL 由一系列的 CRL 来表明被撤销的证书信息,它每次发布的 CRL 是对前面发布 CRL 的增量扩充。基本的 CRL 信息有:被撤销证书序列号、撤销时间、撤销原因、签名者以及 CRL 签名等信息。基于 CRL 的验证是一种不严格的证书认证。CRL 能证明在 CRL 中被撤销的证书是无效的。但是,它不能给出不在 CRL 中的证书的状态。如果执行严格的认证,需要采用在线方式进行认证,即 OCSP 认证。一般是由CA签名的一组电子文档, 包括了被废除证书的唯一标识 (证书序列号), CRL用来列出已经过期或废除的数字证书。它每隔一段时间就会更新, 因此必须定期下载该清单, 才会取得最新信息。

12. SCEP

简单证书注册协议。基于文件的证书登记方式需要从您的本地计算机将文本文件复制和粘贴到证书发布中心, 和从证书发布中心复制和粘贴到您的本地计算机。SCEP可以自动处理这个过程但是CRLs仍然需要手工的在本地计算机和CA发布中心之间进行复制和粘贴。

13. PKCS7

加密消息语法(pkcs7),是各种消息存放的格式标准。这些消息包括:数据、签名数据、数字信封、签名数字信封、摘要数据和加密数据。

14. PKCS12

- pkcs12 (个人数字证书标准)用于存放用户证书、crl、用户私钥以及证书链。pkcs12 中的私钥是加密存放的。

生成含有SAN的证书

- 使用OPENSSL命令行来生成KEY+CSR2个文件,
- 使用KEYTOOL来生成JKS和CSR文件

1.生成CA证书私钥。

openssl genrsa -out ca.key 4096

```
[root@docker-compose-harbor CA]# openssl genrsa -out ca.key 4096
Generating RSA private key, 4096 bit long modulus
.....++
.....++
e is 65537 (0x10001)
[root@docker-compose-harbor CA]#
[root@docker-compose-harbor CA]#
[root@docker-compose-harbor CA]# ls
ca.key
[root@docker-compose-harbor CA]#
```

2、生成CA证书

```
openssl req -x509 -new -nodes -sha512 -days 3650 \
-subj "/C=CN/ST=Beijing/L=Beijing/O=example/OU=Personal/CN=cdh1" \
-key ca.key \
-out ca.crt
```

自签名的证书，不被浏览器信任，适合内部或者测试使用。

3、生成服务器证书

生成私钥

```
[root@docker-compose-harbor CA]# openssl genrsa -out cdh1.key 4096
Generating RSA private key, 4096 bit long modulus
.....++
.....++
e is 65537 (0x10001)
[root@docker-compose-harbor CA]#
[root@docker-compose-harbor CA]# ll -h
total 12K
-rw-r--r--. 1 root root 3.2K Jul 20 04:52 cdh1.key
-rw-r--r--. 1 root root 2.0K Jul 20 04:51 ca.crt
-rw-r--r--. 1 root root 3.2K Jul 20 04:48 ca.key
[root@docker-compose-harbor CA]#
```

生成证书签名请求 (CSR)

制作过程中，系统会产生2个密钥，一个是私钥，存放在服务器上，一个是CSR文件公钥，需要ca签名。

```
openssl req -sha512 -new \
-subj "/C=CN/ST=Beijing/L=Beijing/O=example/OU=Personal/CN=cdh1" \
-key cdh1.key \
-out cdh1.csr
```

需要依次输入国家，地区，城市，组织，组织单位，Common Name和Email。

其中Common Name，可以写自己的名字或者域名，如果要支持https，Common Name应该与域名保持一致，否则会引起浏览器警告。

可以将证书发送给证书颁发机构（CA），CA验证过请求者的身份之后，会出具签名证书，需要花钱。

另外，如果只是内部或者测试需求，也可以使用OpenSSL实现自签名。

执行过程

```
[root@docker-compose-harbor CA]# openssl req -sha512 -new \
> -subj "/C=CN/ST=Beijing/L=Beijing/O=example/OU=Personal/CN=cdh1" \
> -key cdh1.key \
> -out cdh1.csr

[root@docker-compose-harbor CA]# ll -h
total 20K
-rw-r--r--. 1 root root 1.7K Jul 20 04:59 cdh1.csr
-rw-r--r--. 1 root root 3.2K Jul 20 04:52 cdh1.key
-rw-r--r--. 1 root root 2.0K Jul 20 04:51 ca.crt
-rw-r--r--. 1 root root 3.2K Jul 20 04:48 ca.key
[root@docker-compose-harbor CA]#
生成一个x509 v3扩展文件
cat > v3.ext <<-EOF
authorityKeyIdentifier=keyid,issuer
basicConstraints=CA:FALSE
keyUsage = digitalSignature, nonRepudiation, keyEncipherment, dataEncipherment
extendedKeyUsage = serverAuth
```

```
subjectAltName = IP:cdh1
EOF
```

4、使用该v3.ext文件为Harbor主机生成证书cdh1.crt

```
openssl x509 -req -sha512 -days 3650 \
-extfile v3.ext \
-CA ca.crt -CAkey ca.key -CAcreateserial \
-in cdh1.csr \
-out cdh1.crt
```

```
[root@docker-compose-harbor CA]# openssl x509 -req -sha512 -days 3650 \
> -extfile v3.ext \
> -CA ca.crt -CAkey ca.key -CAcreateserial \
> -in cdh1.csr \
> -out cdh1.crt
Signature ok
subject=/C=CN/ST=Beijing/L=Beijing/O=example/OU=Personal/CN=cdh1
Getting CA Private Key
[root@docker-compose-harbor CA]#
```

当服务端向 CA 机构申请证书的时候，CA 签发证书的过程：

- 首先 CA 会把持有者的公钥、用途、颁发者、有效时间等信息打成一个包，然后对这些信息进行 Hash 计算，得到一个 Hash 值；
- 然后 CA 会使用自己的私钥将该 Hash 值加密，生成 Certificate Signature，也就是 CA 对证书做了签名；
- 最后将 Certificate Signature 添加在文件证书上，形成数字证书；

5、转换cdh1.crt为cdh1.cert，供Docker使用

转换证书格式

Docker守护程序将.crt文件解释为CA证书，并将.cert文件解释为客户端证书

```
openssl x509 -inform PEM -in cdh1.crt -out cdh1.cert
```

修改harbor.yml文件key路径

修改对应的证书地址：

- cdh1.cert 的地址
- cdh1.key 的地址

故harbor.yml中certificate填写为如上cdh1.cert的文件目录地址: /usr/local/harbor/ssl/cdh1.cert

故harbor.yml中private_key填写为如上cdh1.key的文件目录地址: /usr/local/harbor/ssl/cdh1.key

```
[root@docker-compose-harbor harbor]# egrep -v "^$|^#" harbor.yml |head -10
hostname: cdh1
http:
# port for http, default is 80. If https enabled, this port will redirect to
https port
port: 80
https:
# https port for harbor, default is 443
port: 443
# The path of cert and key files for nginx
certificate: /opt/CA/harbor/cert/cdh1.crt
private_key: /opt/CA/harbor/cert/cdh1.key
[root@docker-compose-harbor harbor]#
```

6 运行prepare脚本以启用HTTPS

./prepare

```
[root@docker-compose-harbor harbor]# ./prepare
prepare base dir is set to /opt/harbor/harbor
Generated configuration file: /config/portal/nginx.conf
Generated configuration file: /config/log/logrotate.conf
Generated configuration file: /config/log/rsyslog_docker.conf
Generated configuration file: /config/nginx/nginx.conf
Generated configuration file: /config/core/env
Generated configuration file: /config/core/app.conf

Clean up the input dir
[root@docker-compose-harbor harbor]#
```

7 运行install.sh脚本来启动harbor

./install.sh

```
[root@docker-compose-harbor harbor]# ./install.sh
[Step 0]: checking if docker is installed ...
Note: docker version: 20.10.17
[Step 1]: checking docker-compose is installed ...
Not
```

证书复制到 docker 并且启动后登录

```
cp /usr/local/harbor/ssl/cdh1.crt /etc/docker/certs.d/cdh1/ca.crt
```

```
systemctl daemon-reload && systemctl restart docker
```

```
[root@centos1 ssl]# pwd
/usr/local/harbor/ssl
[root@centos1 ssl]# ll /usr/local/harbor/ssl/cdh1.crt
-rw-r--r--. 1 root root 2045 Mar  2 23:19 /usr/local/harbor/ssl/cdh1.crt
[root@centos1 ssl]# openssl x509 -inform PEM -in cdh1.crt -out cdh1.crt
[root@centos1 ssl]# cp /usr/local/harbor/ssl/cdh1.crt /etc/docker/certs.d/cdh1/ca.crt
cp: overwrite '/etc/docker/certs.d/cdh1/ca.crt'? y
[root@centos1 ssl]# systemctl daemon-reload && systemctl restart docker
[root@centos1 ssl]# docker login cdh1
Username: admin
Password:
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store
Login Succeeded
[root@centos1 ssl]#
```

hostname push失败

问题:

push本地镜像到 harbor私服时, push 到 docker.io仓库去了

原因:

在配置insecure-registry时, docker 必须配置服务器的 FQDN或者IP地址.不能是服务器的hostname (比如harbor)

尼恩配置的是cdh1, 推不上去。

FQDN是什么意思?

FQDN (fully qualified domain name) 完全限定域名, 是互联网上特定计算机或主机的完整域名。

FQDN 由两部分组成: 主机名和域名。

例如, 假设邮件服务器的 FQDN 可能是 mail.chenweiliang.com。

主机名为mail, 主机位于域名chenweiliang.com。

DNS (Domain Name System), 负责将 FQDN 转换为 IP地址, 是 Internet 上大多数应用程序的寻址方式。

FQDN: (Fully Qualified Domain Name) 完全限定域名: 同时包含主机名和域名的名称。(通过符号".")

以下为解决方法:

配置harbor服务器的 /etc/hosts, 将本地ip地址对应为一条FQDN记录,

比如 192.168.56.121 harbor.daemon.io

停止harbor后，修改harbor配置 harbor.yml 文件，将 hostname 配置项改为 harbor.daemon.io (一个FQDN)，然后重新配置 harbor。

```
docker-compose down -v
```

生成配置文件

```
./prepare
```

启动harbor

```
docker-comp up -d
```

(可以在另外一台能访问harbor服务器的机器上配置 hosts 记录为 FQDN，然后web访问 harbor检查是否能正常登陆.)

重新配置 docker daemon 中的配置 registry-mirrors 和 insecure-registries 然后重启 docker.

比如：

编辑客户机/etc/docker/daemon.json文件

```
{"insecure-registries":["http://harbor.daemon.io:85]}
```

重启客户机docker服务

```
systemctl restart docker #或者(service docker restart)
```

之后就能正常pull，并且push本地镜像到 harbor私服中.

附上完整的：

```
{
  "registry-mirrors": [
    "https://bjtzu1jb.mirror.aliyuncs.com",
    "http://f1361db2.m.daocloud.io",
    "https://hub-mirror.c.163.com",
    "https://docker.mirrors.ustc.edu.cn",
    "https://reg-mirror.qiniu.com",
    "https://dockerhub.azk8s.cn",
    "https://registry.docker-cn.com",
    "https://harbor.daemon.io"
  ],
  "insecure-registries": ["http://harbor.daemon.io:85"]
}
```

推送镜像到Harber



Docker 推送命令

在项目中标记镜像：

```
docker tag SOURCE_IMAGE[:TAG] harbor.daemon.io/demo/REPOSITORY[:TAG]
```

```
docker tag nginx:latest harbor.daemon.io/demo/nginx:latest
```

推送镜像到当前项目：

```
docker push harbor.daemon.io/demo/REPOSITORY[:TAG]
```

```
docker push harbor.daemon.io/demo/nginx:latest
```

错误提升：

```
[root@centos1 harbor]# docker push harbor.daemon.io/demo/nginx:latest
The push refers to repository [harbor.daemon.io/demo/nginx]
Get "https://harbor.daemon.io/v2/": x509: certificate is valid for cdh1, not harbor.daemon.io
```

需要生成证书

```
openssl req -x509 -new -nodes -sha512 -days 3650 \
-subj "/C=CN/ST=Beijing/L=Beijing/O=example/OU=Personal/CN=harbor.daemon.io" \
-key ca.key \
-out ca.crt
```

```
openssl genrsa -out harbor.daemon.io.key 4096
```

```
openssl req -sha512 -new \  
-subj "/C=CN/ST=Beijing/L=Beijing/O=example/OU=Personal/CN=harbor.daemon.io" \  
-key harbor.daemon.io.key \  
-out harbor.daemon.io.csr
```

生成一个x509 v3扩展文件

```
cat > harbor.daemon.io.v3.ext <<-EOF  
authorityKeyIdentifier=keyid,issuer  
basicConstraints=CA:FALSE  
keyUsage = digitalSignature, nonRepudiation, keyEncipherment, dataEncipherment  
extendedKeyUsage = serverAuth  
subjectAltName = DNS:harbor.daemon.io  
EOF
```

```
openssl x509 -req -sha512 -days 3650 \  
-extfile harbor.daemon.io.v3.ext \  
-CA ca.crt -CAkey ca.key -CAcreateserial \  
-in harbor.daemon.io.csr \  
-out harbor.daemon.io.crt
```

```
openssl x509 -inform PEM -in harbor.daemon.io.crt -out harbor.daemon.io.cert
```

harbor.yml中certificate填写为如上cdh1.cert的文件目录地址:

/usr/local/harbor/ssl/harbor.daemon.io.cert

harbor.yml中private_key填写为如上cdh1.key的文件目录地址:

/usr/local/harbor/ssl/harbor.daemon.io.key

```
mkdir /etc/docker/certs.d/harbor.daemon.io
```

```
cp /usr/local/harbor/ssl/harbor.daemon.io.cert  
/etc/docker/certs.d/harbor.daemon.io/ca.crt
```

```
cp /usr/local/harbor/ssl/harbor.daemon.io.cert  
/etc/docker/certs.d/harbor.daemon.io/harbor.daemon.io.cert
```

```
cp /usr/local/harbor/ssl/harbor.daemon.io.cert  
/etc/docker/certs.d/harbor.daemon.io/ca.crt
```

```
docker-compose down
```

```
prepare
```

```
systemctl restart docker

docker-compose up -d

docker login harbor.daemon.io
```

推送成功

推送镜像到当前项目：

`docker push harbor.daemon.io/demo/REPOSITORY[:TAG]`

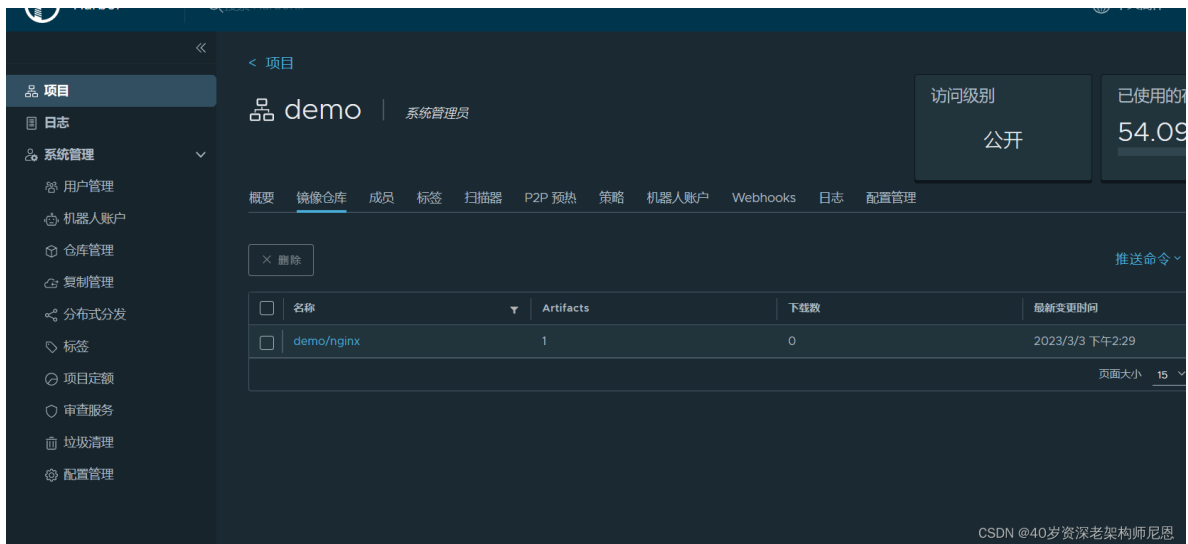
```
docker push harbor.daemon.io/demo/nginx:latest
```

```
Creating harbor-ctl ... done
Creating registryctl ... done
Creating harbor-portal ... done
Creating harbor-core ... done
Creating nginx ... done
Creating harbor-jobservice ... done
[root@centos1 harbor]# docker login harbor.daemon.io
Username: admin
Password:
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
[root@centos1 harbor]# docker push harbor.daemon.io/demo/nginx:latest
The push refers to repository [harbor.daemon.io/demo/nginx]
d874fd2bc83b: Pushed
32ce5f6a5106: Pushed
f1db227348d0: Pushed
b8d6e692a25e: Pushed
e379e8aedd4d: Pushed
2edcec3590a4: Pushed
latest: digest: sha256:ee89b00528ff4f02f2405e4ee221743ebc3f8e8dd0bfd5c4c20a2fa2aaa7ede3 size: 1570
[root@centos1 harbor]#
```

界面展示

<http://cdh1:85/>



Docker Image概述

什么是Image

- 文件和meta data的集合(root filesystem)
- 分层的，并且每一层都可以添加改变删除文件，成为一个新的image
- 不同的image可以共享相同的layer
- Image本身是read-only的

Image的获取

- 方式1: Build from Dockerfile
- 方式2: Pull from Registry

例如：输入命令 `sudo docker pull ubuntu:14.04`,则会从Registry中拉出Image，Registry类似于github的作用，默认都是从docker hub上面拉取，上面会有官方和第三方的版本

```
[root@VM_0_7_centos ~]# docker pull ubuntu:14.04
14.04: Pulling from library/ubuntu
8284e13a281d: Pull complete
26e1916a9297: Pull complete
4102fc66d4ab: Pull complete
1cf2b01777b2: Pull complete
7f7a2d5e04ed: Pull complete
Digest: sha256:71529e96591eb36a4100cd0cc5353ff1a2f4ee7a85011e3d3dd07cb5eb524a3e
Status: Downloaded newer image for ubuntu:14.04
```

输入命令 `sudo docker image ls`则可以显示所拉去的Image

```
[root@VM_0_7_centos ~]# docker image ls
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
ubuntu              14.04              971bb384a50a       3 weeks ago        188MB
```

如何做一个自己的Base Image

1.首先创建一个可以执行的程序，下面用一个C语言的hello程序做例子

```
#include<stdio.h>

int main(){
    printf("hello docker\n");
}
```

2.通过dockerfile把这个可执行文件打成一个Image
我们在hellow文件当前目录下创建一个Dockerfile文件，如下：

```
#1.从哪里拉取
FROM scratch
#2.将hello添加到根目录路径
ADD hello /
#3.运行根目录下的hello
CMD ["/hello"]
```

3.执行命令：docker build -t yunduan/hello-world . (-t表示指示表情，"."表示在当前路径下寻找dockerfile)，执行以后，出现如下界面表示执行成功。

```
[root@VM_0_7_centos docker-image]# docker build -t yunduan/hello-world .
Sending build context to Docker daemon 868.9kB
Step 1/3 : FROM scratch
--->
Step 2/3 : ADD hello /
---> 61655b869d78
Step 3/3 : CMD ["/hello"]
---> Running in cf4ed2ed1b33
Removing intermediate container cf4ed2ed1b33
---> d0e82743f7f2
Successfully built d0e82743f7f2
Successfully tagged yunduan/hello-world:latest
```

4.查看image，则可以看到成功build了一个image,执行命令docker history +[IMAGE ID]则可以查看镜像的层级

,执行命令docker run + [镜像标签名]则可以生成一个container运行程序。

```
[root@VM_0_7_centos docker-image]# docker image ls
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
yunduan/hello-world latest             d0e82743f7f2       6 minutes ago      865kB
ubuntu               14.04              971bb384a50a       3 weeks ago        188MB
hello-world          latest             2cb0d9787c4d       4 weeks ago        1.85kB
[root@VM_0_7_centos docker-image]# docker history d0e82743f7f2
IMAGE               CREATED             CREATED BY          SIZE
COMMENT
d0e82743f7f2       6 minutes ago      /bin/sh -c #(nop) CMD ["/hello"]      0B
61655b869d78       6 minutes ago      /bin/sh -c #(nop) ADD file:a407813fbf96fe4c3... 865kB
[root@VM_0_7_centos docker-image]# docker run yunduan/hello-world
hello docker
```

构建自己的Docker镜像

- 命令一：docker container commit (Create a new image from a container's changes)

这个命令表示当你在容器中做出了改变之后，可以重新构建Image

```
[root@VM_0_7_centos ~]# docker container ls -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
e2f109fc4a5f	centos	"/bin/bash"	About a minute ago	Exited (0) 6 seconds ago		wonderful_mccarthy
3f0f98b73e5b	centos	"/bin/bash"	About an hour ago	Exited (0) About an hour ago		romantic_bhaskara
64c6b5c1c8b7	yunduan/hello-world	"/hello"	2 hours ago	Exited (13) 2 hours ago		nervous_babbage

```
[root@VM_0_7_centos ~]# docker commit wonderful_mccarthy yunduan/centos-vim  
sha256:7acf3fa2acfae3318e01d849b3f85db6e6fd46e6b7bab60028ae5a03ff2a1dd5
```

```
[root@VM_0_7_centos ~]# docker image ls
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
yunduan/centos-vim	latest	7acf3fa2acfa	8 seconds ago	342MB
yunduan/hello-world	latest	d0e82743f7f2	2 hours ago	865kB
centos	latest	5182e96772bf	6 days ago	200MB
ubuntu	14.04	971bb384a50a	3 weeks ago	188MB
hello-world	latest	2cb0d9787c4d	4 weeks ago	1.85kB

```
[root@VM_0_7_centos ~]# docker history 5182e96772bf
```

IMAGE	CREATED	CREATED BY	SIZE	COMMENT
5182e96772bf	6 days ago	/bin/sh -c #(nop) CMD ["/bin/bash"]	0B	
<missing>	6 days ago	/bin/sh -c #(nop) LABEL org.label-schema.sc...	0B	
<missing>	6 days ago	/bin/sh -c #(nop) ADD file:6340c690b08865d7e...	200MB	

```
[root@VM_0_7_centos ~]# docker history 7acf3fa2acfa
```

IMAGE	CREATED	CREATED BY	SIZE	COMMENT
7acf3fa2acfa	About a minute ago	/bin/bash	142MB	
5182e96772bf	6 days ago	/bin/sh -c #(nop) CMD ["/bin/bash"]	0B	
<missing>	6 days ago	/bin/sh -c #(nop) LABEL org.label-schema.sc...	0B	
<missing>	6 days ago	/bin/sh -c #(nop) ADD file:6340c690b08865d7e...	200MB	

通过这个例子可以看出，其就在centos镜像上重新构建了一层

- 命令二：docker image build(Build an image from a Dockerfile)

Dockerfile语法

- FROM

原则：尽量使用官方的image作为base image！

```
FROM scratch #制作base image  
FROM centos #使用base image  
FROM ubuntu: 14.04
```

- LABEL

原则：Metadata不可少！相当于代码的注释

```
LABEL maintainer="yunduan@gmail.com"  
LABEL version="1.0"  
LABEL description="This is description"
```

- RUN

作用：执行命令并创建新的Image Layer

原则：为了美观，复杂的RUN请用反斜线换行！避免无用分层，合并多条命令成一行！

```
RUN yum update && yum install -y vim \  
    python-dev #反斜线换行  
RUN apt-get update && apt-get install -y perl \  
    pwgen --no-install-recommends && rm -rf \  
    /var/lib/apt/lists/* #注意清理cache  
RUN /bin/bash -c 'source $HOME/.bashrc;echo  
$HOME'
```

- WORKDIR

作用：设定当前目录，类似于cd

原则：用WORKDIR，不要用RUN cd！ 尽量使用绝对目录

```
WORKDIR /root
WORKDIR /test #如果没有会自动创建test目录
WORKDIR demo
RUN pwd      #输出结果应该是 /test/demo
```

- ADD and COPY

作用：把本地文件添加到Docker image里面

原则：大部分情况，COPY优先于ADD！ ADD除了COPY还有额外功能（解压）！ 添加远程文件/目录请使用curl或者wget！

```
ADD hello /
ADD test.tar.gz / #添加到根目录并解压
WORKDIR /root
ADD hello test/   # /root/test/hello
WORKDIR /root
COPY hello test/
```

- ENV

作用：设置一个环境变量，引用常量

原则：尽量使用ENV增加可维护性！

```
ENV MYSQL_VERSION 5.6 # 设置常量
RUN apt-get install -y mysql-server= "${MYSQL_VERSION}" \
    && rm -rf /var/lib/apt/lists/* # 引用常量
```

- VOLUME and EXPOSE (存储和网络)
- CMD and ENTRYPOINT

CMD:设置容器启动后默认执行的命令和参数

注释：1.容器启动时默认执行的命令 2.如果docker run指定了其他命令，CMD命令被忽略 3.如果定义了多个CMD，只有最后一个会执行

ENTRYPOINT：设置容器启动时运行的命令

注释：1.让容器以应用程序或者服务的形式运行 2.不会被忽略，一定会执行 3.最佳实践：写一个shell脚本作为entrypoint

1.Shell格式

```
RUN apt-get install -y vim
CMD echo "hello docker"
ENTRYPOINT echo "hello docker"
```

2.Exec格式

```
RUN ["apt-get","install","-y","vim"]
CMD ["/bin/echo", "hello docker"]
ENTRYPOINT ["/bin/echo", "hello docker"]
```

3.Shell和Exec格式

- Dockerfile1 A

```
FROM centos
ENV name Docker
ENTRYPOINT echo "hello $name"
```

- Dockerfile2

```
FROM centos
ENV name Dokcer
ENTRYPOINT ["/bin/bash", "-c","echo hello $name"]
```

镜像发布

- docker login
- docker push

Docker进程与宿主机进程的对应关系

Linux通过进程ID查看文件路径

子进程的文件路径

```
[root@VM-4-17-centos ~]# ls -l /proc/27880
total 0
dr-xr-xr-x 2 root root 0 Nov  3 16:41 attr
-rw-r--r-- 1 root root 0 Nov  3 16:41 autogroup
-r----- 1 root root 0 Nov  3 16:41 auxv
-r--r--r-- 1 root root 0 Nov  3 16:14 cgroup
--w----- 1 root root 0 Nov  3 16:41 clear_refs
-r--r--r-- 1 root root 0 Nov  3 16:15 cmdline
-rw-r--r-- 1 root root 0 Nov  3 16:41 comm
-rw-r--r-- 1 root root 0 Nov  3 16:41 coredump_filter
-r--r--r-- 1 root root 0 Nov  3 16:41 cpuset
lrwxrwxrwx 1 root root 0 Nov  3 16:41 cwd -> /
-r----- 1 root root 0 Nov  3 16:41 environ
lrwxrwxrwx 1 root root 0 Nov  3 16:14 exe -> /usr/sbin/sshd
dr-x----- 2 root root 0 Nov  3 16:14 fd
dr-x----- 2 root root 0 Nov  3 16:41 fdinfo
-rw-r--r-- 1 root root 0 Nov  3 16:41 gid_map
-r----- 1 root root 0 Nov  3 16:41 io
-r--r--r-- 1 root root 0 Nov  3 16:41 limits
-rw-r--r-- 1 root root 0 Nov  3 16:41 loginuid
dr-x----- 2 root root 0 Nov  3 16:41 map_files
-r--r--r-- 1 root root 0 Nov  3 16:41 maps
-rw----- 1 root root 0 Nov  3 16:41 mem
-r--r--r-- 1 root root 0 Nov  3 16:14 mountinfo
```



```

-r--r--r-- 1 root root 0 Nov  3 16:41 mounts
-r----- 1 root root 0 Nov  3 16:41 mountstats
dr-xr-xr-x 5 root root 0 Nov  3 16:41 net
dr-x--x--x 2 root root 0 Nov  3 16:14 ns
-r--r--r-- 1 root root 0 Nov  3 16:41 numa_maps
-rw-r--r-- 1 root root 0 Nov  3 16:41 oom_adj
-r--r--r-- 1 root root 0 Nov  3 16:41 oom_score
-rw-r--r-- 1 root root 0 Nov  3 16:41 oom_score_adj
-r--r--r-- 1 root root 0 Nov  3 16:41 pagemap
-r----- 1 root root 0 Nov  3 16:41 patch_state
-r--r--r-- 1 root root 0 Nov  3 16:41 personality
-rw-r--r-- 1 root root 0 Nov  3 16:41 projid_map
lrwxrwxrwx 1 root root 0 Nov  3 16:41 root -> /
-rw-r--r-- 1 root root 0 Nov  3 16:41 sched
-r--r--r-- 1 root root 0 Nov  3 16:41 schedstat
-r--r--r-- 1 root root 0 Nov  3 16:41 sessionid
-rw-r--r-- 1 root root 0 Nov  3 16:41 setgroups
-r--r--r-- 1 root root 0 Nov  3 16:41 smaps
-r--r--r-- 1 root root 0 Nov  3 16:41 stack
-r--r--r-- 1 root root 0 Nov  3 16:14 stat
-r--r--r-- 1 root root 0 Nov  3 16:41 statm
-r--r--r-- 1 root root 0 Nov  3 16:14 status
-r--r--r-- 1 root root 0 Nov  3 16:41 syscall
dr-xr-xr-x 3 root root 0 Nov  3 16:41 task
-r--r--r-- 1 root root 0 Nov  3 16:41 timers
-rw-r--r-- 1 root root 0 Nov  3 16:14 uid_map
-r--r--r-- 1 root root 0 Nov  3 16:41 wchan

```

以下是/proc目录中进程27880的信息说明：

proc/27880 pid为N的进程信息

/proc/27880/cmdline 进程启动命令

/proc/27880/cwd 链接到进程当前工作目录

/proc/27880/envIRON 进程环境变量列表

/proc/27880/exe 链接到进程的执行命令文件

/proc/27880/fd 包含进程相关的所有的文件描述符

/proc/27880/maps 与进程相关的内存映射信息

/proc/27880/mem 指代进程持有的内存，不可读

/proc/27880/root 链接到进程的根目录

/proc/27880/stat 进程的状态

/proc/27880/statm 进程使用的内存的状态

/proc/27880/status 进程状态信息，比stat/statm更具可读性

容器的PID namespace (命名空间)

在Docker中，进程管理的基础就是Linux内核中的PID命名空间技术。

在不同PID命名空间中，进程ID是独立的；即在两个不同命名空间下的进程可以有相同的PID。

在Docker中，每个Container进程缺省都具有不同的PID命名空间。通过命名空间技术，Docker实现容器间的进程隔离。

docker中运行的容器进程，本质上还是运行在宿主机上的，所以也会拥有相对应的PID

找出容器ID

```
# docker ps
```

输出

```
[root@VM-4-17-centos ~]# docker ps
```

CONTAINER ID	IMAGE	STATUS	COMMAND
CREATED		NAMES	PORTS
460d68823930	1emonbar/centos6-ssh:latest	Up 32 minutes	"/bin/sh -c
'/usr/sb...' 32 minutes ago		centos6-2	0.0.0.0:6021->22/tcp, 0.0.0.0:6081->80/tcp

查看容器信息

```
docker inspect id
```

输出

```
[root@VM-4-17-centos ~]# docker inspect 460d68823930
[root@VM-4-17-centos ~]# docker inspect 460d68823930
[
  {
    "Id":
    "460d688239304172f39bb9586bfc5959e0c3db64e7c3a0937f1003f94408ebbd",
    "Created": "2021-11-03T08:24:36.934129599Z",
    "Path": "/bin/sh",
    "Args": [
      "-c",
      "/usr/sbin/sshd -D"
    ],
    "State": {
      "Status": "running",
      "Running": true,
      "Paused": false,
      "Restarting": false,
      "OOMKilled": false,
      "Dead": false,
      "Pid": 4962,
      "ExitCode": 0,
```

```

        "Error": "",
        "StartedAt": "2021-11-03T08:24:37.223255812Z",
        "FinishedAt": "0001-01-01T00:00:00Z"
    },
    "Image":
    "sha256:efd998bd6817af509d348b488e3ce4259f9f05632644a7bf574b785bbc8950b8",
    "ResolveConfPath":
    "/var/lib/docker/containers/460d688239304172f39bb9586bfc5959e0c3db64e7c3a0937f10
    03f94408ebbd/resolve.conf",
    "HostnamePath":
    "/var/lib/docker/containers/460d688239304172f39bb9586bfc5959e0c3db64e7c3a0937f10
    03f94408ebbd/hostname",
    "HostsPath":
    "/var/lib/docker/containers/460d688239304172f39bb9586bfc5959e0c3db64e7c3a0937f10
    03f94408ebbd/hosts",
    "LogPath":
    "/var/lib/docker/containers/460d688239304172f39bb9586bfc5959e0c3db64e7c3a0937f10
    03f94408ebbd/460d688239304172f39bb9586bfc5959e0c3db64e7c3a0937f1003f94408ebbd-
    json.log",
    "Name": "/centos6-2",
    "RestartCount": 0,
    "Driver": "overlay2",
    "Platform": "linux",
    "MountLabel": "",
    "ProcessLabel": "",
    "AppArmorProfile": "",
    "ExecIDs": null,
    "HostConfig": {
        "Binds": null,
        "ContainerIDFile": "",
        "LogConfig": {
            "Type": "json-file",
            "Config": {}
        },
        "NetworkMode": "default",
        "PortBindings": {
            "22/tcp": [
                {
                    "HostIp": "",
                    "HostPort": "6021"
                }
            ],
            "80/tcp": [
                {
                    "HostIp": "",
                    "HostPort": "6081"
                }
            ]
        },
        "RestartPolicy": {
            "Name": "no",
            "MaximumRetryCount": 0
        },
        "AutoRemove": false,
        "VolumeDriver": "",
        "VolumesFrom": null,
        "CapAdd": null,
        "CapDrop": null,

```

```

    "Dns": [],
    "DnsOptions": [],
    "DnsSearch": [],
    "ExtraHosts": null,
    "GroupAdd": null,
    "IpcMode": "shareable",
    "Cgroup": "",
    "Links": null,
    "OomScoreAdj": 0,
    "PidMode": "",
    "Privileged": false,
    "PublishAllPorts": false,
    "ReadonlyRootfs": false,
    "SecurityOpt": null,
    "UTSMode": "",
    "UsersnsMode": "",
    "ShmSize": 67108864,
    "Runtime": "runc",
    "ConsoleSize": [
        0,
        0
    ],
    "Isolation": "",
    "CpuShares": 0,
    "Memory": 0,
    "NanoCpus": 0,
    "CgroupParent": "",
    "Blkioweight": 0,
    "BlkioweightDevice": [],
    "BlkiodeviceReadBps": null,
    "BlkiodeviceWriteBps": null,
    "BlkiodeviceReadIops": null,
    "BlkiodeviceWriteIops": null,
    "CpuPeriod": 0,
    "CpuQuota": 0,
    "CpuRealtimePeriod": 0,
    "CpuRealtimeRuntime": 0,
    "CpusetCpus": "",
    "CpusetMems": "",
    "Devices": [],
    "DeviceCgroupRules": null,
    "DiskQuota": 0,
    "KernelMemory": 0,
    "MemoryReservation": 0,
    "MemorySwap": 0,
    "MemorySwappiness": null,
    "OomKillDisable": false,
    "PidsLimit": 0,
    "Ulimits": null,
    "CpuCount": 0,
    "CpuPercent": 0,
    "IOMaximumIops": 0,
    "IOMaximumBandwidth": 0
},
    "GraphDriver": {
        "Data": {

```

```

        "LowerDir":
"/var/lib/docker/overlay2/6835c1b48237aafe27e2efabeda92a3a6623f254f88d54b5e6acce
454e560dd6-
init/diff:/var/lib/docker/overlay2/7139bf0b716c6e0b6a0c709b7043466f9bbfd7024f8ae
584061c00b0bd97348c/diff:/var/lib/docker/overlay2/66a3e278259cdc5f0741ce30a115ba
a3bd6247a60c487e4118e85f2f39328f11/diff:/var/lib/docker/overlay2/20e22c4c28ebadb
615eb4c7c290253d3eb91cb49722ee2931b0ee628352a5857/diff:/var/lib/docker/overlay2/
a3fa9dbebc83a853083205b8f7921c632cd67f64531f4a25cab419a43172e3ae/diff:/var/lib/d
ocker/overlay2/3af7958c9a4e54d24598058a9fa1e85eb35e3d40f766fa498a674b52724ae73e/
diff:/var/lib/docker/overlay2/becb65af4396137ed41fe6d516e834e6e6e9120f4edfac8e2c
a8dd67cce23268/diff:/var/lib/docker/overlay2/fef055305158cc96906514c447f0eaea059
45138896b0b35ac4146b6a2a3e273/diff:/var/lib/docker/overlay2/79158cdf3ba832493ab0
d02d560c784208fe51c74236a5a86f7fb4fb50ab6e44/diff:/var/lib/docker/overlay2/86258
a18e1110582b819719593687f11f0404d00a41667b3432c3b974fb1ce42/diff:/var/lib/docker
/overlay2/8826b2e0068653fb2c5e8a3dbf839470e2b8eef8cf752b5fe901bea1b210849f/diff:
/var/lib/docker/overlay2/145301e2738a8a7581c2bbd5beb9bf7a49b247e46642b8084efbc02
6a1826116/diff:/var/lib/docker/overlay2/f621f37535e0db1fe44902e22dba7ef0844b9a8b
562a9daa39a842a49e9cc9bb/diff:/var/lib/docker/overlay2/7b493e4a97907aaa18b97ad2e
9120b5bf87c0e9908ee390a35ea6ff546d8cec6/diff",
        "MergedDir":
"/var/lib/docker/overlay2/6835c1b48237aafe27e2efabeda92a3a6623f254f88d54b5e6acce
454e560dd6/merged",
        "UpperDir":
"/var/lib/docker/overlay2/6835c1b48237aafe27e2efabeda92a3a6623f254f88d54b5e6acce
454e560dd6/diff",
        "WorkDir":
"/var/lib/docker/overlay2/6835c1b48237aafe27e2efabeda92a3a6623f254f88d54b5e6acce
454e560dd6/work"
    },
    "Name": "overlay2"
},
"Mounts": [],
"Config": {
    "Hostname": "460d68823930",
    "Domainname": "",
    "User": "",
    "AttachStdin": false,
    "AttachStdout": false,
    "AttachStderr": false,
    "ExposedPorts": {
        "22/tcp": {},
        "80/tcp": {}
    },
    "Tty": true,
    "OpenStdin": true,
    "StdinOnce": false,
    "Env": [
        "HOME=/"
    ],
    "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
    ],
    "Cmd": [
        "/bin/sh",
        "-c",
        "/usr/sbin/sshd -D"
    ],
    "Image": "docker.io/lemonbar/centos6-ssh:latest",
    "Volumes": null,

```

```

        "WorkingDir": "",
        "Entrypoint": null,
        "OnBuild": null,
        "Labels": {}
    },
    "NetworkSettings": {
        "Bridge": "",
        "SandboxID":
"ea66261fb6d8089d5b2d585a2dc32b2003365df7118f5f5e898a152fb5b35773",
        "HairpinMode": false,
        "LinkLocalIPv6Address": "",
        "LinkLocalIPv6PrefixLen": 0,
        "Ports": {
            "22/tcp": [
                {
                    "HostIp": "0.0.0.0",
                    "HostPort": "6021"
                }
            ],
            "80/tcp": [
                {
                    "HostIp": "0.0.0.0",
                    "HostPort": "6081"
                }
            ]
        },
        "SandboxKey": "/var/run/docker/netns/ea66261fb6d8",
        "SecondaryIPAddresses": null,
        "SecondaryIPv6Addresses": null,
        "EndpointID":
"09ad719a4e9115ee56c5fb0f5b0d39c50bf5acaf0a1afacedc13969c82a2969f",
        "Gateway": "172.17.0.1",
        "GlobalIPv6Address": "",
        "GlobalIPv6PrefixLen": 0,
        "IPAddress": "172.17.0.6",
        "IPPrefixLen": 16,
        "IPv6Gateway": "",
        "MacAddress": "02:42:ac:11:00:06",
        "Networks": {
            "bridge": {
                "IPAMConfig": null,
                "Links": null,
                "Aliases": null,
                "NetworkID":
"2586283d16a08210c955d705f05e0f6999b59523a84b0c163e33f535af809ddd",
                "EndpointID":
"09ad719a4e9115ee56c5fb0f5b0d39c50bf5acaf0a1afacedc13969c82a2969f",
                "Gateway": "172.17.0.1",
                "IPAddress": "172.17.0.6",
                "IPPrefixLen": 16,
                "IPv6Gateway": "",
                "GlobalIPv6Address": "",
                "GlobalIPv6PrefixLen": 0,
                "MacAddress": "02:42:ac:11:00:06",
                "DriverOpts": null
            }
        }
    }
}

```

```
}  
]
```

进入相应目录

```
# cd  
/sys/fs/cgroup/memory/docker/460d688239304172f39bb9586bfc5959e0c3db64e7c3a0937f1  
003f94408ebbd/
```

输出

```
cd  
/sys/fs/cgroup/memory/docker/460d688239304172f39bb9586bfc5959e0c3db64e7c3a0937f1  
003f94408ebbd/  
[root@VM-4-17-centos  
460d688239304172f39bb9586bfc5959e0c3db64e7c3a0937f1003f94408ebbd]# ll  
total 0  
-rw-r--r-- 1 root root 0 Nov  3 16:24 cgroup.clone_children  
--w--w--w- 1 root root 0 Nov  3 16:24 cgroup.event_control  
-rw-r--r-- 1 root root 0 Nov  3 16:24 cgroup.procs  
-rw-r--r-- 1 root root 0 Nov  3 16:24 memory.failcnt  
--w----- 1 root root 0 Nov  3 16:24 memory.force_empty  
-rw-r--r-- 1 root root 0 Nov  3 16:24 memory.kmem.failcnt  
-rw-r--r-- 1 root root 0 Nov  3 16:24 memory.kmem.limit_in_bytes  
-rw-r--r-- 1 root root 0 Nov  3 16:24 memory.kmem.max_usage_in_bytes  
-r--r--r-- 1 root root 0 Nov  3 16:24 memory.kmem.slabinfo  
-rw-r--r-- 1 root root 0 Nov  3 16:24 memory.kmem.tcp.failcnt  
-rw-r--r-- 1 root root 0 Nov  3 16:24 memory.kmem.tcp.limit_in_bytes  
-rw-r--r-- 1 root root 0 Nov  3 16:24 memory.kmem.tcp.max_usage_in_bytes  
-r--r--r-- 1 root root 0 Nov  3 16:24 memory.kmem.tcp.usage_in_bytes  
-r--r--r-- 1 root root 0 Nov  3 16:24 memory.kmem.usage_in_bytes  
-rw-r--r-- 1 root root 0 Nov  3 16:24 memory.limit_in_bytes  
-rw-r--r-- 1 root root 0 Nov  3 16:24 memory.max_usage_in_bytes  
-rw-r--r-- 1 root root 0 Nov  3 16:24 memory.memsw.failcnt  
-rw-r--r-- 1 root root 0 Nov  3 16:24 memory.memsw.limit_in_bytes  
-rw-r--r-- 1 root root 0 Nov  3 16:24 memory.memsw.max_usage_in_bytes  
-r--r--r-- 1 root root 0 Nov  3 16:24 memory.memsw.usage_in_bytes  
-rw-r--r-- 1 root root 0 Nov  3 16:24 memory.move_charge_at_immigrate  
-r--r--r-- 1 root root 0 Nov  3 16:24 memory.numa_stat  
-rw-r--r-- 1 root root 0 Nov  3 16:24 memory.oom_control  
----- 1 root root 0 Nov  3 16:24 memory.pressure_level  
-rw-r--r-- 1 root root 0 Nov  3 16:24 memory.soft_limit_in_bytes  
-r--r--r-- 1 root root 0 Nov  3 16:24 memory.stat  
-rw-r--r-- 1 root root 0 Nov  3 16:24 memory.swappiness  
-r--r--r-- 1 root root 0 Nov  3 16:24 memory.usage_in_bytes  
-rw-r--r-- 1 root root 0 Nov  3 16:24 memory.use_hierarchy  
-rw-r--r-- 1 root root 0 Nov  3 16:24 notify_on_release  
-rw-r--r-- 1 root root 0 Nov  3 16:24 tasks
```

```
[root@VM-4-17-centos
460d688239304172f39bb9586bfc5959e0c3db64e7c3a0937f1003f94408ebbd]# cat
cgroup.procs
4962
11539
[root@VM-4-17-centos
460d688239304172f39bb9586bfc5959e0c3db64e7c3a0937f1003f94408ebbd]# cat pids.max
max
[root@VM-4-17-centos
460d688239304172f39bb9586bfc5959e0c3db64e7c3a0937f1003f94408ebbd]# cat tasks
4962
11539
[root@VM-4-17-centos
460d688239304172f39bb9586bfc5959e0c3db64e7c3a0937f1003f94408ebbd]# cat
cgroup.clone_children
0
[root@VM-4-17-centos
460d688239304172f39bb9586bfc5959e0c3db64e7c3a0937f1003f94408ebbd]# pwd
/sys/fs/cgroup/pids/docker/460d688239304172f39bb9586bfc5959e0c3db64e7c3a0937f100
3f94408ebbd
```

查看容器目录里的进程号

进程号就存在一个文件里面

```
[root@VM-4-17-centos
460d688239304172f39bb9586bfc5959e0c3db64e7c3a0937f1003f94408ebbd]#
cat cgroup.procs
4962
```

与前面利用 `docker top` 命令，可以让我们从宿主机操作系统中看到容器的进程信息。

```
[root@VM-4-17-centos ~]# docker top centos6-2
UID                PID                PPID                C
STIME              TTY                TIME               CMD
root               4962               4948                0
16:24              pts/0              00:00:00            /usr/sbin/sshd -D
```

启动一个进程

我们下面会在 centos6-2 容器中，利用 `docker exec` 命令启动一个 "sleep" 进程


```
[root@VM-4-17-centos ]# docker exec -d centos6-2 sleep 2000
[root@VM-4-17-centos ]# docker exec centos6-2 ps -ef
UID          PID  PPID  C STIME TTY          TIME CMD
root          1    0  0 08:24 pts/0      00:00:00 /usr/sbin/sshd -D
root          6    0  0 09:06 ?          00:00:00 sleep 2000
root         10    0  0 09:06 ?          00:00:00 ps -ef
```

查看宿主的进程号

```
[root@VM-4-17-centos ]# docker top centos6-2
UID          PID          PPID          C
STIME        TTY          TIME          CMD
root         4962         4948          0
16:24        pts/0        00:00:00          /usr/sbin/sshd -D
root         11539        4948          0
17:06        ?           00:00:00          sleep 2000
```

我们可以清楚的看到exec命令创建的sleep进程属 centos6-2 容器的名空间，但是它的父进程是Docker容器的启动进程。

查看容器目录里的进程号

进程号就存在一个文件里面

```
[root@VM-4-17-centos
460d688239304172f39bb9586bfc5959e0c3db64e7c3a0937f1003f94408ebbd]# cat
cgroup.procs
4962
11539
```

```
docker exec -d centos6-2 pstree -p
```

```
docker exec -d centos6-2 ps -auxf
```

```
docker exec -d centos6-2 ll /proc
```

输出

```
[root@VM-4-17-centos docker]# docker exec centos6-2 ps -ef
UID          PID  PPID  C STIME TTY          TIME CMD
root          1    0  0 08:24 pts/0      00:00:00 /usr/sbin/sshd -D
root          6    0  0 09:06 ?          00:00:00 sleep 2000
root         40    0  0 09:26 ?          00:00:00 ps -ef
[root@VM-4-17-centos docker]# docker exec centos6-2 ps -auxf
```

```

USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         44  0.0  0.0  13360  1012 ?        Rs    09:26   0:00 ps -auxf
root         6  0.0  0.0   4120   316 ?        Ss    09:06   0:00 sleep 2000
root         1  0.0  0.0  66664  3068 pts/0    Ss+   08:24   0:00 /usr/sbin/sshd
-D
Warning: bad syntax, perhaps a bogus '-'? See /usr/share/doc/procps-3.2.8/FAQ
[root@VM-4-17-centos docker]# docker exec centos6-2 pstree -p
sshd(1)

```

docker daemon (docker守护进程)

```

pidof dockerd    #查看docker守护进程pid
lsof -p 3197 | wc -l #docker守护进程打开的文件数

```

在两个容器中的"centos"是两个独立的进程，但是他们拥有相同的父进程 Docker Daemon。

所以Docker可以父子进程的方式在Docker Daemon和Redis容器之间进行交互。

另一个值得注意的方面是，`docker exec` 命令可以进入指定的容器内部执行命令。由它启动的进程属于容器的namespace和相应的cgroup。

但是这些进程的父进程是Docker Daemon而非容器的PID1进程。

我们下面会在Redis容器中，利用 `docker exec` 命令启动一个"sleep"进程

```

docker@default:~$ docker exec -d redis sleep 2000
docker@default:~$ docker exec redis ps -ef
UID          PID    PPID  C STIME TTY          TIME CMD
redis         1       0  0 02:26 ?           00:00:00 redis-server *:6379
root         11       0  0 02:26 ?           00:00:00 sleep 2000
root         21       0  0 02:29 ?           00:00:00 ps -ef
docker@default:~$ docker top redis
UID          PID          PPID          C
STIME          TTY          TIME          CMD
999            9955         1264          0
02:12          ?            00:00:00          redis-server *:6379
root           9984         1264          0
02:13          ?            00:00:00          sleep 2000

```

我们可以清楚的看到exec命令创建的sleep进程属Redis容器的名空间，但是它的父进程是Docker Daemon。

如果我们在宿主机操作系统中手动杀掉容器的启动进程（在上文示例中是redis-server），容器会自动结束，而容器名空间中所有进程也会退出。

```

docker@default:~$ PID=$(docker inspect --format="{{.State.Pid}}" redis)
docker@default:~$ sudo kill $PID
docker@default:~$ docker ps -a
CONTAINER ID        IMAGE               COMMAND             CREATED
STATUS              PORTS              NAMES
356eca186321        redis              "/entrypoint.sh redis" 23 minutes ago
Up 4 minutes        6379/tcp           redis2
f6bc57cc1b46        redis              "/entrypoint.sh redis" 23 minutes ago
Exited (0) 4 seconds ago      redis

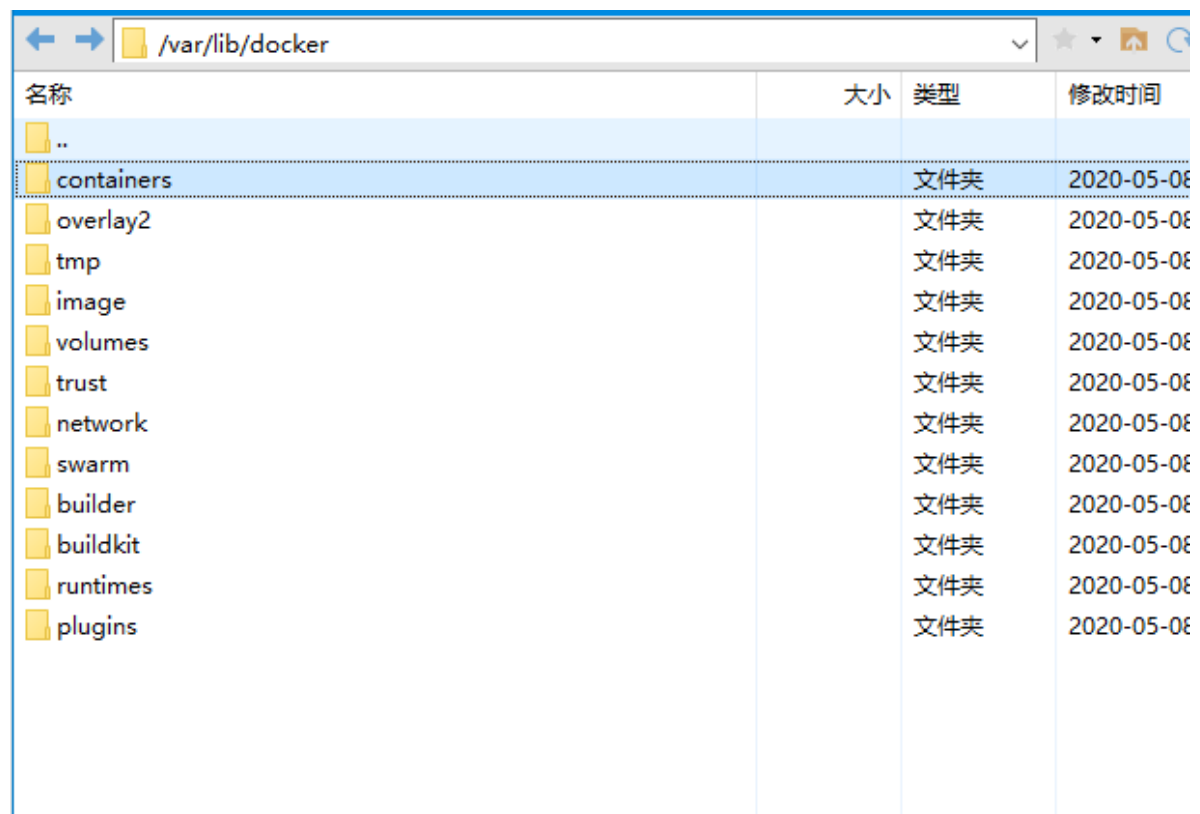
```

通过以上示例：

- 每个容器有独立的PID名空间，
- 容器的生命周期和其PID1进程一致
- 利用 `docker exec` 可以进入到容器的名空间中启动进程

Docker文件目录和容器内部操作

Docker默认的文件目录位于Linux server的/var/lib/docker 下面。目录结构如下



名称	大小	类型	修改时间
..			
containers		文件夹	2020-05-08
overlay2		文件夹	2020-05-08
tmp		文件夹	2020-05-08
image		文件夹	2020-05-08
volumes		文件夹	2020-05-08
trust		文件夹	2020-05-08
network		文件夹	2020-05-08
swarm		文件夹	2020-05-08
builder		文件夹	2020-05-08
buildkit		文件夹	2020-05-08
runtimes		文件夹	2020-05-08
plugins		文件夹	2020-05-08

|-----containers：用于存储容器信息

|-----image：用来存储镜像中间件及本身信息，大小，依赖信息

|-----network

|-----swarm

|-----tmp：docker临时目录

|-----trust：docker信任目录

|-----volumes：docker卷目录

还可以通过docker指令确认文件位置:

```
docker info
```

```
Kernel Version: 4.18.0-147.8.1.el8_1.x86_64
Operating System: CentOS Linux 8 (Core)
OSType: linux
Architecture: x86_64
CPUs: 2
Total Memory: 3.509GiB
Name: strpaydev
ID: 2ITO:EK64:JNCR:UHY5:LJYA:UN46:C2RO:MKVI:TPCU:S3NW:II6O:AZI
Docker Root Dir: /var/lib/docker
Debug Mode: false
Registry: https://index.docker.io/v1/
Labels:
Experimental: false
Insecure Registries:
  127.0.0.0/8
Live Restore Enabled: false
```

查看某个容器的文件目录:

```
docker exec 容器name ls
```

```
[root@centos6-2 ~]# docker exec sqlserver ls
bin
boot
dev
etc
home
lib
lib64
media
mnt
opt
proc
root
run
sbin
srv
sys
tmp
usr
var
```

```
docker exec centos6-2 ls /proc
```

```
[root@VM-4-17-centos containers]# docker exec centos6-2 ls /proc
1
103
acpi
buddyinfo
bus
```

cgroups
cmdline
consoles
cpuinfo
crypto
devices
diskstats
dma
driver
execdomains
fb
filesystems
fs
interrupts
iomem
ioports
irq
kallsyms
kcore
key-users
keys
kmsg
kpagecount
kpageflags
loadavg
locks
mdstat
meminfo
misc
modules
mounts
mtrr
net
pagetypeinfo
partitions
sched_debug
schedstat
scsi
self
slabinfo
softirqs
stat
swaps
sys
sysrq-trigger
sysvipc
timer_list
timer_stats
tty
uptime
version
vmallocinfo
vmstat
zoneinfo

Docker Daemon 底层原理

作为Docker容器管理的守护进程，Docker Daemon从最初集成在 `docker` 命令中（1.11版本前），到后来的独立成单独二进制程序（1.11版本开始），其功能正在逐渐拆分细化，被分配到各个单独的模块中去。

演进：Docker守护进程启动

从Docker服务的启动脚本，也能看见守护进程的逐渐剥离：

在Docker 1.8之前，Docker守护进程启动的命令为：

```
docker -d
```

这个阶段，守护进程看上去只是Docker client的一个选项。

Docker 1.8开始，启动命令变成了：

```
docker daemon
```

这个阶段，守护进程看上去是 `docker` 命令的一个模块。

Docker 1.11开始，守护进程启动命令变成了：

```
dockerd
```

其服务的配置文件为：

```
[Service]
Type=notify
# the default is not to use systemd for cgroups because the delegate issues
still
# exists and systemd currently does not support the cgroup feature set required
# for containers run by docker
ExecStart=/usr/bin/dockerd
ExecReload=/bin/kill -s HUP $MAINPID
```

此时已经和Docker client分离，独立成一个二进制程序了。

当然，守护进程模块不停的在重构，其基本功能和定位没有变化。和一般的CS架构系统一样，守护进程负责和Docker client交互，并管理Docker镜像、容器。

OCI (Open Container Initiative)

Open Container Initiative, 也就是常说的OCI, 是由多家公司共同成立的项目, 并由linux基金会进行管理, 致力于container runtime的标准的制定和runc的开发等工作。

官方的介绍是

An open governance structure for the express purpose of creating open industry standards around container formats and runtime. – Open Containers Official Site

所谓container runtime, 主要负责的是容器的生命周期的管理。oci的runtime spec标准中对于容器的状态描述, 以及对于容器的创建、删除、查看等操作进行了定义。

目前主要有两个标准文档: 容器运行时标准 (runtime spec) 和 容器镜像标准 (image spec) 。这两个协议通过 OCI runtime filesystem bundle 的标准格式连接在一起, OCI 镜像可以通过工具转换成 bundle, 然后 OCI 容器引擎能够识别这个 bundle 来运行容器。

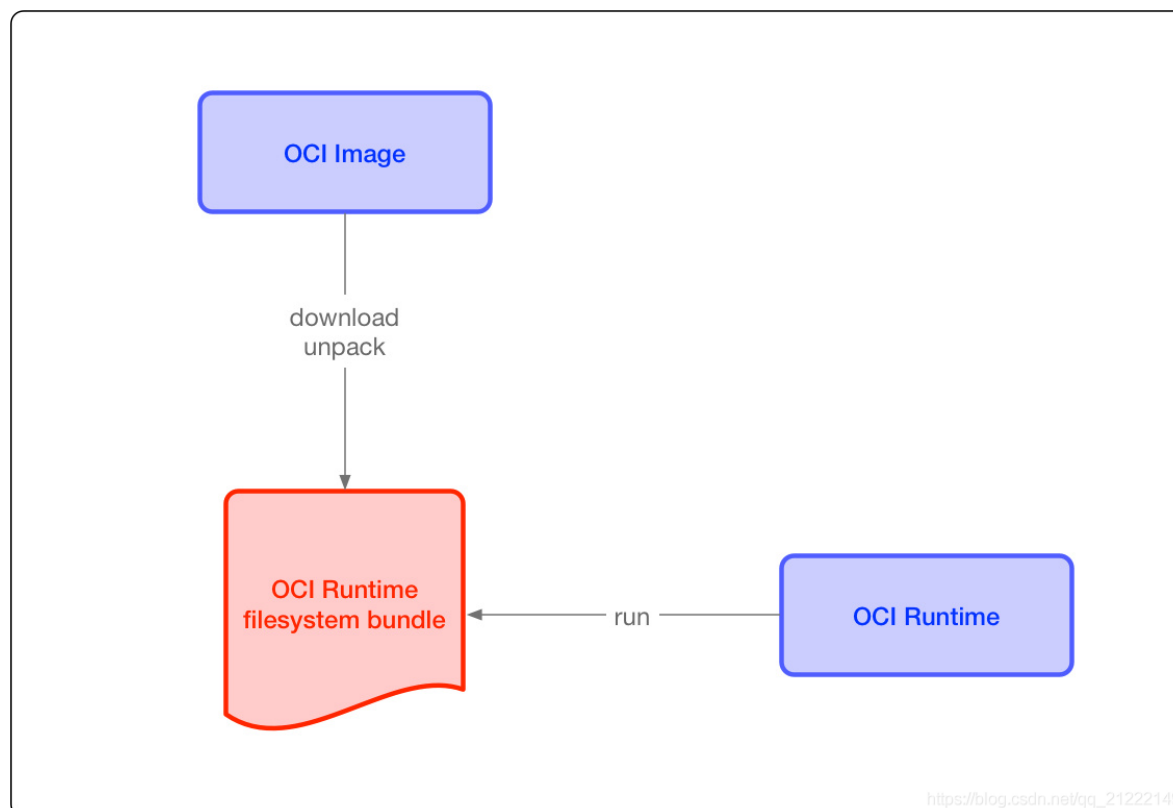


image spec

OCI 容器镜像主要包括几块内容:

文件系统: 以 layer 保存的文件系统, 每个 layer 保存了和上层之间变化的部分, layer 应该保存哪些文件, 怎么表示增加、修改和删除的文件等

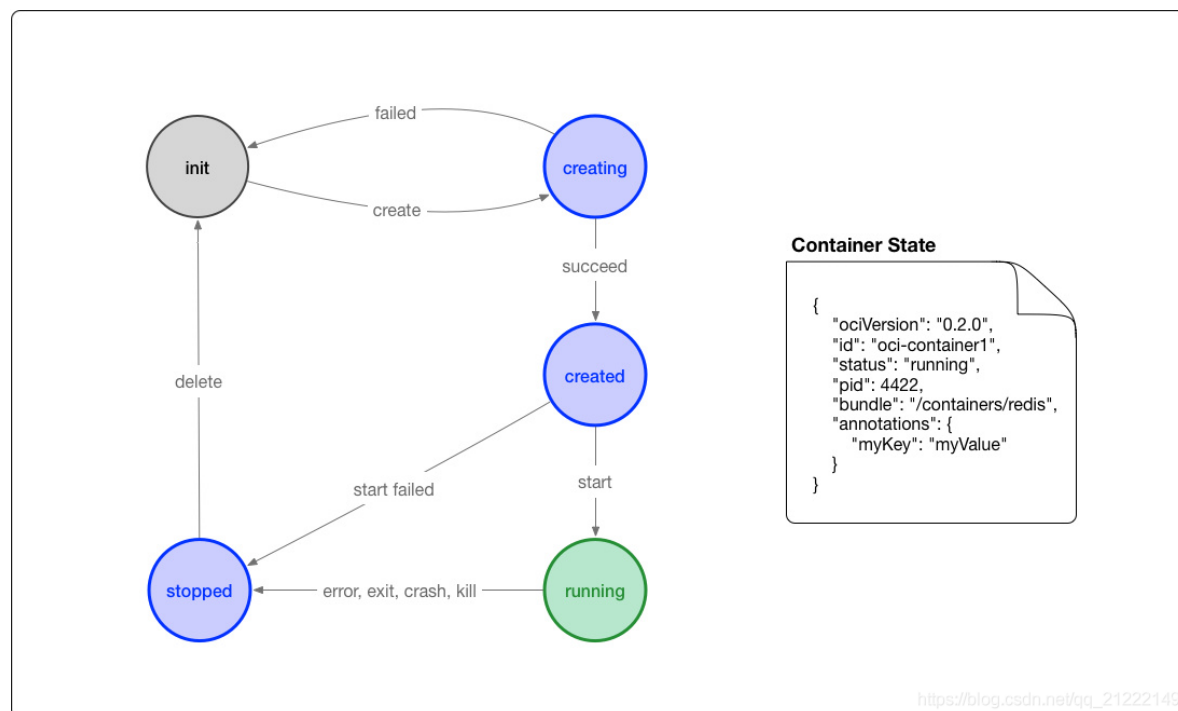
config 文件: 保存了文件系统的层级信息 (每个层级的 hash 值, 以及历史信息), 以及容器运行时需要的一些信息 (比如环境变量、工作目录、命令参数、mount 列表), 指定了镜像在某个特定平台和系统的配置。比较接近我们使用 docker inspect <image_id> 看到的内容

manifest 文件: 镜像的 config 文件索引, 有哪些 layer, 额外的 annotation 信息, manifest 文件中保存了很多和当前平台有关的信息

index 文件: 可选的文件, 指向不同平台的 manifest 文件, 这个文件能保证一个镜像可以跨平台使用, 每个平台拥有不同的 manifest 文件, 使用 index 作为索引

runtime spec

OCI 对容器 runtime 的标准主要是指指定容器的运行状态，和 runtime 需要提供的命令。下图可以是容器状态转换图：



Docker CLI客户端工具

```
/usr/bin/docker
```

Docker 的客户端工具，通过CLI与 dockerd API 交流。CLI 的例子比如docker build ... docker run ...

Docker Daemon守护进程（dockerd）

```
/usr/bin/dockerd
```

当然，守护进程模块不停的在重构，其基本功能和定位没有变化。

和一般的CS架构系统一样，守护进程负责和Docker client交互，并管理Docker镜像、容器。

Containerd

```
/usr/bin/docker-containerd
```

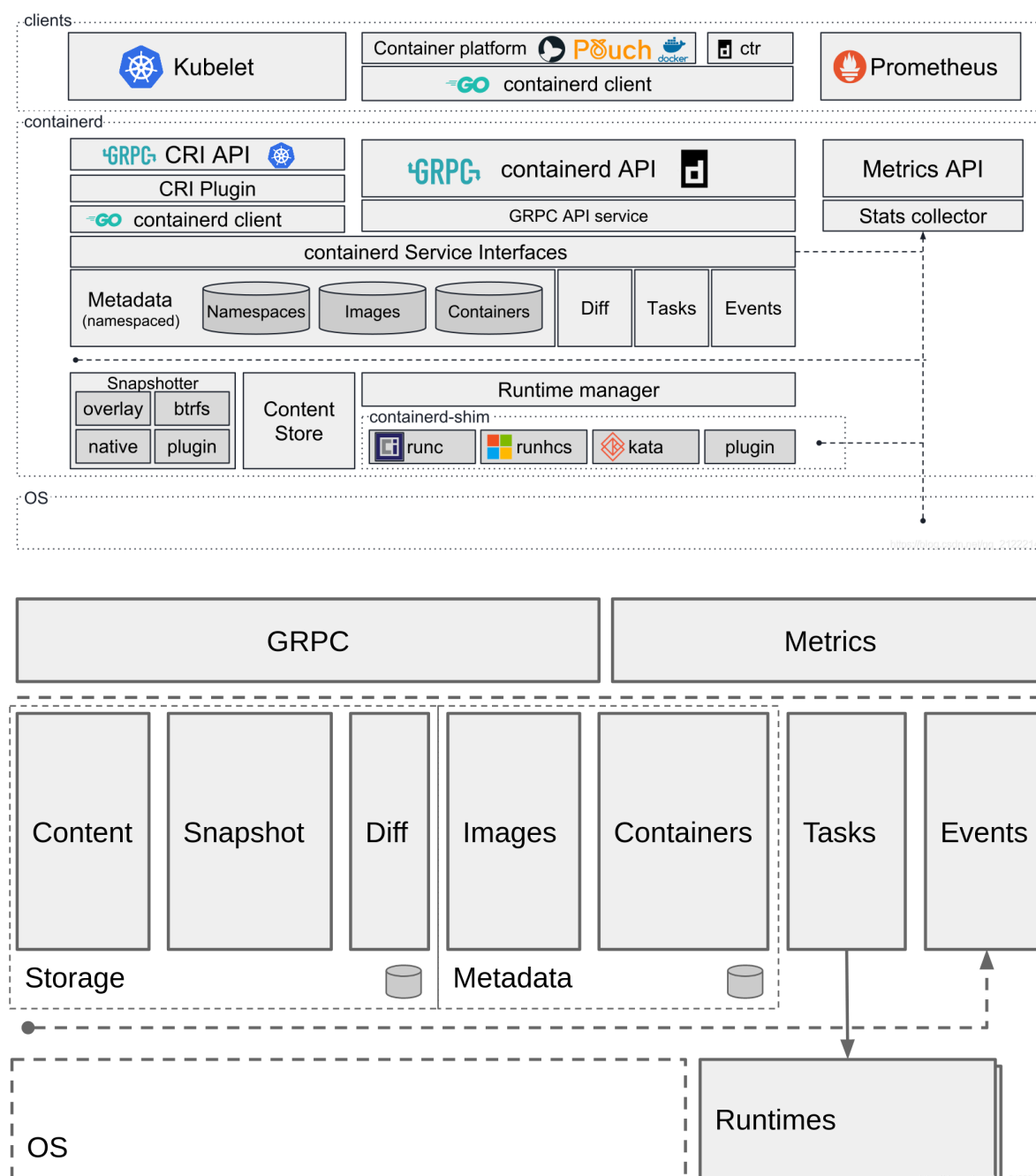
containerd是容器技术标准化之后的产物，为了能够兼容OCI标准，将容器运行时及其管理功能从 Docker Daemon剥离。

理论上，即使不运行dockerd，也能够直接通过containerd来管理容器。

当然，containerd本身也只是一个守护进程，容器的实际运行时由后面介绍的runC控制。

最近，Docker刚刚宣布开源containerd。从其项目介绍页面可以看出，containerd主要职责是**镜像管理**（镜像、元信息等）、**容器执行**（调用最终运行时组件执行）。

containerd向上为Docker Daemon提供了gRPC接口，使得Docker Daemon屏蔽下面的结构变化，确保原有接口向下兼容。向下通过containerd-shim结合runC，使得引擎可以独立升级，避免之前Docker Daemon升级会导致所有容器不可用的问题。



containerd fully leverages the **OCI runtime specification**¹, **image format specifications** and **OCI reference implementation (runc)**.

containerd includes a daemon exposing gRPC API over a local UNIX socket. The API is a low-level one designed for higher layers to wrap and extend. Containerd uses RunC to run containers according to the OCI specification.

docker-shim 容器进程

docker-shim是一个真实运行的容器的真实垫片载体，每启动一个容器都会起一个新的docker-shim的一个进程，

他直接通过指定的三个参数：容器id，bundle目录（containerd的对应某个容器生成的目录，一般位于：/var/run/docker/libcontainerd/containerID），

运行是二进制（默认为runc）来调用runc的api创建一个容器（比如创建容器：最后拼装的命令如下：runc create。。。。。）

```
/usr/bin/docker-containerd-shim
```

每启动一个容器都会起一个新的docker-shim的一个进程。他直接通过指定的三个参数来创建一个容器：

1. 容器id
2. bundle目录（containerd的对应某个容器生成的目录，一般位于：/var/run/docker/libcontainerd/containerID）
3. 运行是二进制（默认为runc）来调用runc的api（比如创建容器时，最后拼装的命令如下：runc create。。。）

他的作用是：

1. 它允许容器运行时(即 runC)在启动容器之后退出，简单说就是不必为每个容器一直运行一个容器运行时(runC)
2. 即使在 containerd 和 dockerd 都挂掉的情况下，容器的标准 IO 和它的文件描述符也都是可用的
3. 向 containerd 报告容器的退出状态
前两点尤其重要，有了它们就可以在不中断容器运行的情况下升级或重启 dockerd(这对于生产环境来说意义重大)。

runc (OCI reference implementation)

OCI定义了容器运行时标准，runC是Docker按照开放容器格式标准（OCF, Open Container Format）制定的一种具体实现。

runC是从Docker的libcontainer中迁移而来的，实现了容器启停、资源隔离等功能。

Docker默认提供了docker-runc实现，事实上，通过containerd的封装，可以在Docker Daemon启动的时候指定runc的实现。

```
/usr/bin/docker-runc
```

OCI定义了容器运行时标准OCI Runtime Spec support (aka runC),

runC是Docker按照开放容器格式标准（OCF, Open Container Format）制定的一种具体实现。

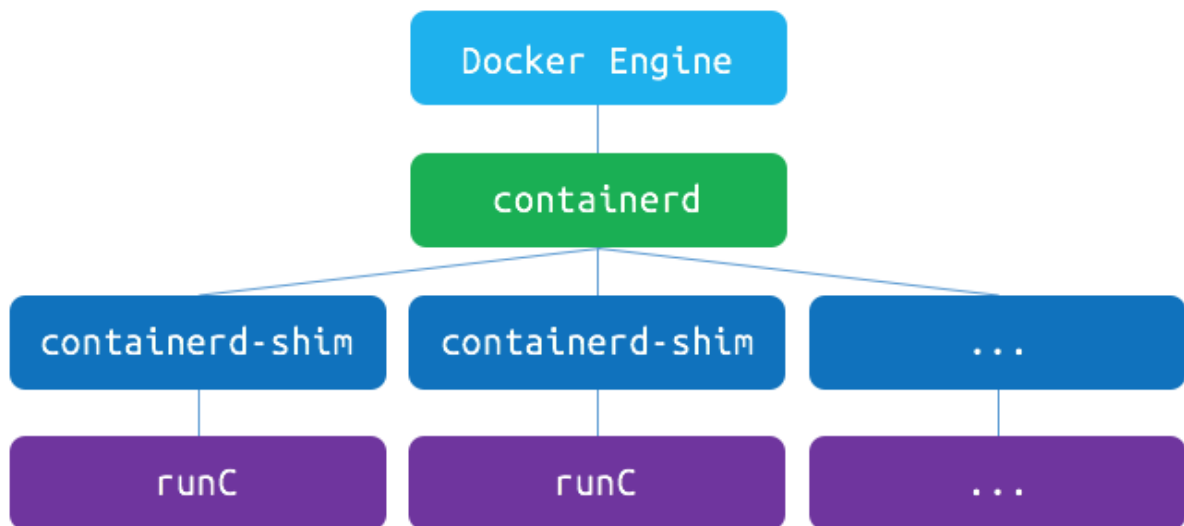
runC是从Docker的**libcontainer**中迁移而来的，实现了容器启停、资源隔离等功能。

Docker默认提供了docker-runc实现，事实上，通过containerd的封装，可以在Docker Daemon启动的时候指定runc的实现。

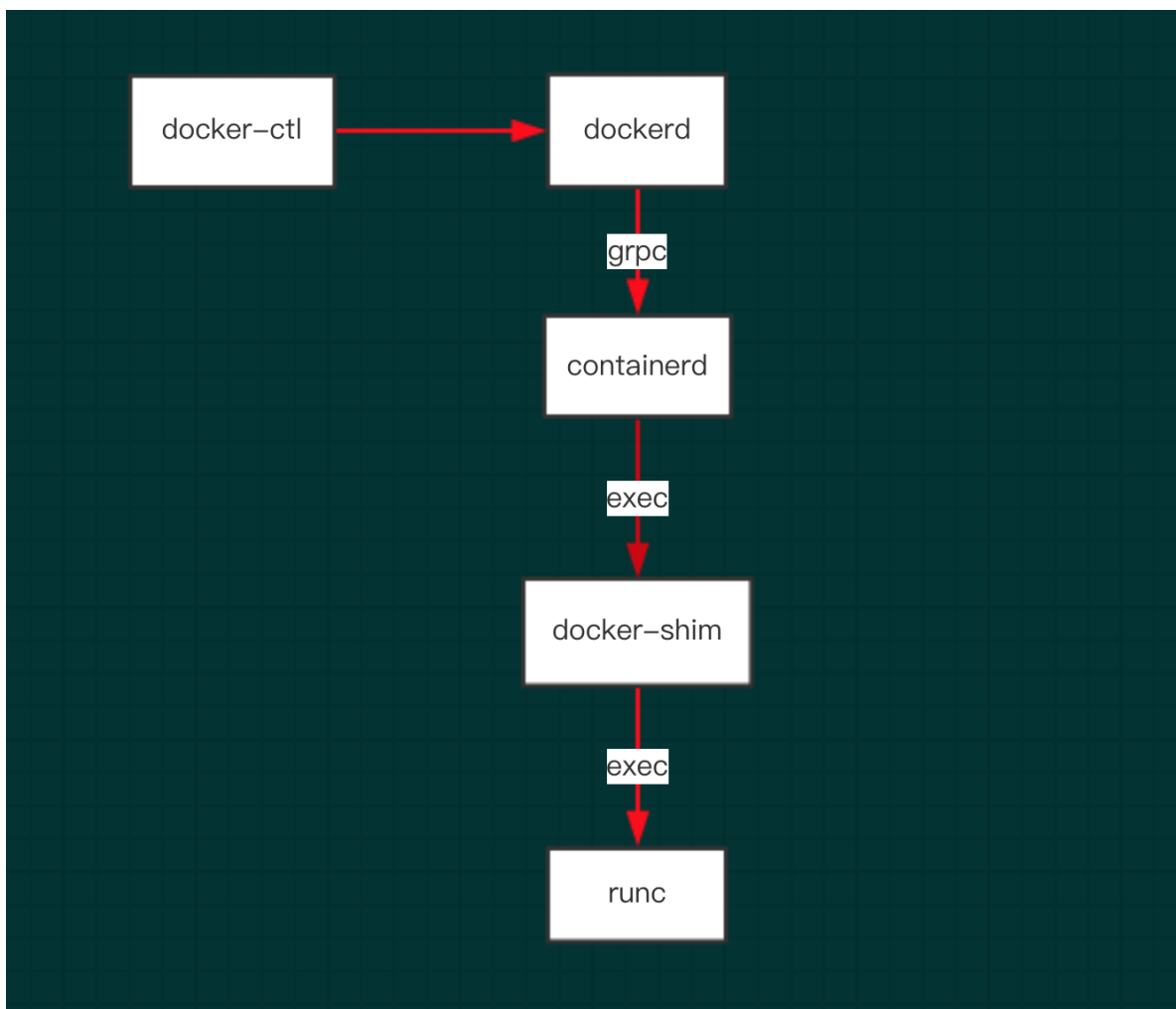
我们可以通过启动Docker Daemon时增加--add-runtime参数来选择其他的runC现。例如：

```
docker daemon --add-runtime "custom=/usr/local/bin/my-runc-replacement"
```

Docker、containerd, containerd-shim和runc之间的关系



他们之间的关系如下图：



我们可以通过启动一个Docker容器，来观察进程之间的关联。

通过runc来启动一个container的过程

查看进程信息

利用 `docker top` 命令，可以让我们从宿主机操作系统中看到容器的进程信息。

```
[root@centos1 ~]# docker top mqbroker-ha-b
UID          PID         PPID         C           TIME        CMD
3000         3473        3401         0           12:06       sh mqbroker
/opt/rocketmq-4.6.0/conf/broker.conf autoCreateTopicEnable=true &
3000         4383        3473         0           12:06       sh /home/
mq/rocketmq-4.6.0/bin/runbroker.sh org.apache.rocketmq.broker.BrokerStartup -c /opt/rocketmq-4.6.0/conf/broker.conf autoCreateTopicEnable=true &
3000         4473        4383         0           12:06       java -server -Xms1952M -Xmx1952M -Xmn400M -XX:+UseG1GC -XX:G1HeapRegionSize=16m -XX:G1ReservePercent=30 -XX:SoftRefLRUPolicyMSPerMB=0 -XX:SurvivorRatio=8 -verbose:gc -Xloggc:/dev/shm/mq_gc_%p.log -XX:+PrintGCDateStamps -XX:+PrintGCApplicationStoppedTime -XX:+PrintAdaptiveSizePolicy -XX:+UseGCLogFileRotation -XX:NumberOfGCLogFiles=5 -XX:GCLogFileSize=30m -XX:-OmitStackTraceInFastThrow -XX:+AlwaysPreTouch -XX:MaxDirectMemorySize=1952M -XX:-UseLargePages -XX:-UseBiasedLocking -Djava.ext.dirs=/usr/lib/jvm/1.8.0-openjdk-1.8.0.232.b09-0.el7_7.x86_64/jre/lib/ext:/home/rocketmq/rocketmq-4.6.0/bin/../lib -server -Xms256m -Xmx512m -Xmn128m -cp ./home/rocketmq-4.6.0/bin/../conf: org.apache.rocketmq.broker.BrokerStartup -c /opt/rocketmq-4.6.0/conf/broker.conf autoCreateTopicEnable=true &
[root@centos1 ~]#
```

尼恩提示：这里比较复杂，具体的视频介绍，请参见稍后的 穿透云原生视频进行介绍。

查看父进程信息

```
[root@VM-4-17-centos containerd]# ps -ef | grep 3401
```

```
[root@centos1 ~]# ps -ef | grep 3401
root          3401          1   0 12:06 ?        00:00:00 /usr/bin/containerd-shim-runc-v2 -namespace moby -id e9eaf999da9183b9be0b3239881bc6b9c2070f13057c322dfed3d072820e962 -address /run/containerd/containerd.sock
ed3d072820e962 -address /run/containerd/containerd.sock
3000         3473        3401   0 12:06 ?        00:00:00 sh mqbroker -c /opt/rocketmq-4.6.0/conf/broker.conf autoCreateTopicEnable=true &
root        28204        24084   0 16:37 pts/1    00:00:00 grep --color=auto 3401
[root@centos1 ~]#
```

尼恩提示：这里比较复杂，具体的视频介绍，请参见稍后的 穿透云原生视频进行介绍。

查看进程树

```
ps tree -l -a -A 3401 -p
```

```
[root@centos1 ~]# ps tree -l -a -A 3401 -p
containerd-shim,3401 -namespace moby -id e9eaf999da9183b9be0b3239881bc6b9c2070f13057c322dfed3d072820e962 -address /run/containerd/containerd.sock
|-sh,3473 mqbroker -c /opt/rocketmq-4.6.0/conf/broker.conf autoCreateTopicEnable=true &
|-sh,4383 /home/rocketmq/rocketmq-4.6.0/bin/runbroker.sh org.apache.rocketmq.broker.BrokerStartup -c /opt/rocketmq-4.6.0/conf/broker.conf autoCreateTopicEnable=true &
|-java,4473 -server -Xms1952M -Xmx1952M -Xmn400M -XX:+UseG1GC -XX:G1HeapRegionSize=16m -XX:G1ReservePercent=25 -XX:InitiatingHeapOccupancyPer=30 -XX:SoftRefLRUPolicyMSPerMB=0 -XX:SurvivorRatio=8 -verbose:gc -Xloggc:/dev/shm/mq_gc_%p.log -XX:+PrintGCDateStamps -XX:+PrintGCApplicationStoppedTime -XX:+PrintAdaptiveSizePolicy -XX:+UseGCLogFileRotation -XX:NumberOfGCLogFiles=5 -XX:GCLogFileSize=30m -XX:-OmitStackTraceInFastThrow -XX:+AlwaysPreTouch -XX:MaxDirectMemorySize=1952M -XX:-UseLargePages -XX:-UseBiasedLocking -Djava.ext.dirs=/usr/lib/jvm/java-1.8.0-openjdk-1.8.0.232.b09-0.7.x86_64/jre/lib/ext:/home/rocketmq/rocketmq-4.6.0/bin/../lib -server -Xms256m -Xmx512m -Xmn128m -cp ./home/rocketmq/rocketmq-4.6.0/bin/../conf: org.apache.rocketmq.broker.BrokerStartup -c /opt/rocketmq-4.6.0/conf/broker.conf autoCreateTopicEnable=true &
|-{java},4482
|-{java},4519
|-{java},4520
|-{java},4521
|-{java},4522
|-{java},4523
|-{java},4524
|-{java},4525
|-{java},4526
|-{java},4527
|-{java},4528
|-{java},4529
|-{java},4680
|-{java},4682
|-{java},4684
|-{java},4691
|-{java},4701
|-{java},4703
|-{java},4704
|-{java},4705
|-{java},4706
|-{java},4707
|-{java},5010
```

虽然 `ps tree` 命令截断了命令，但我们还是能够看出，当 Docker daemon 启动之后，`dockerd` 和 `docker-containerd` 进程一直存在。

当启动容器之后，docker-containerd进程（也是这里介绍的containerd组件）会创建docker-containerd-shim进程，其中的参数e9eaf999da9183b9be0b3239881bc6b9c2070f13057c322dfed3d072820e962就是要启动容器的id。

最后docker-containerd-shim子进程，已经是实际在容器中运行的进程。

docker-containerd-shim另一个参数，是一个和容器相关的目录

```
/var/run/docker/containerd/e9eaf999da9183b9be0b3239881bc6b9c2070f13057c322dfed3d072820e962
```

其中包括了容器配置和标准输入、标准输出、标准错误三个管道文件。

使用下面的

```
11
/var/run/docker/containerd/e9eaf999da9183b9be0b3239881bc6b9c2070f13057c322dfed3d072820e962
```



```
[root@centos1 ~]# ll /var/run/docker/containerd/e9eaf999da9183b9be0b3239881bc6b9c2070f13057c322dfed3d072820e962
total 0
prwx-----, 1 root root 0 Mar 12 12:06 init-stderr
prwx-----, 1 root root 0 Mar 12 12:06 init-stdout
[root@centos1 ~]#
```

尼恩提示：这里比较复杂，具体的视频介绍，请参见稍后的 穿透云原生视频进行介绍。

CRI 运行时接口

kubernetes在初期版本里，就对多个容器引擎做了兼容，因此可以使用docker、rkt对容器进行管理。

以docker为例，kubelet中会启动一个docker manager，通过直接调用docker的api进行容器的创建等操作。

在k8s 1.5版本之后，kubernetes推出了自己的运行时接口api-CRI(container runtime interface)。

cri接口的推出，隔离了各个容器引擎之间的差异，而通过统一的接口与各个容器引擎之间进行互动。

与oci不同，cri与kubernetes的概念更加贴合，并紧密绑定。

cri不仅定义了容器的生命周期的管理，还引入了k8s中pod的概念，并定义了管理pod的生命周期。

在kubernetes中，pod是由一组进行了资源限制的，在隔离环境中的容器组成。

而这个隔离环境，称之为PodSandbox。

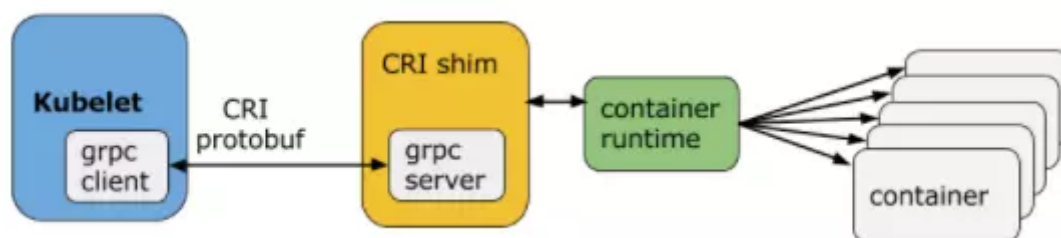
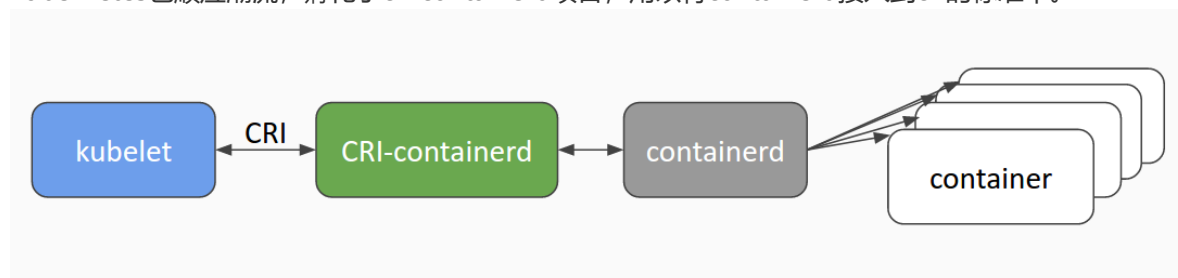
在cri开始之初，主要是支持docker和rkt两种。

其中kubelet是通过cri接口，调用docker-shim，并进一步调用docker api实现的。

如上文所述，docker独立出来了containerd。



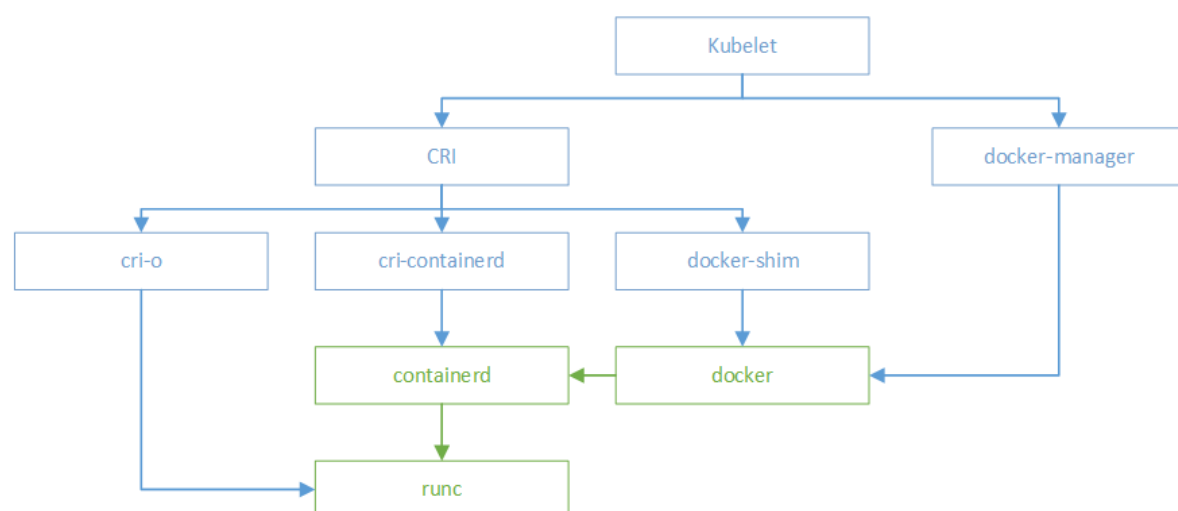
kubernetes也顺应潮流，孵化了cri-containerd项目，用以将containerd接入到cri的标准中。



为了进一步与oci进行兼容，kubernetes还孵化了cri-o，成为了架设在cri和oci之间的一座桥梁。

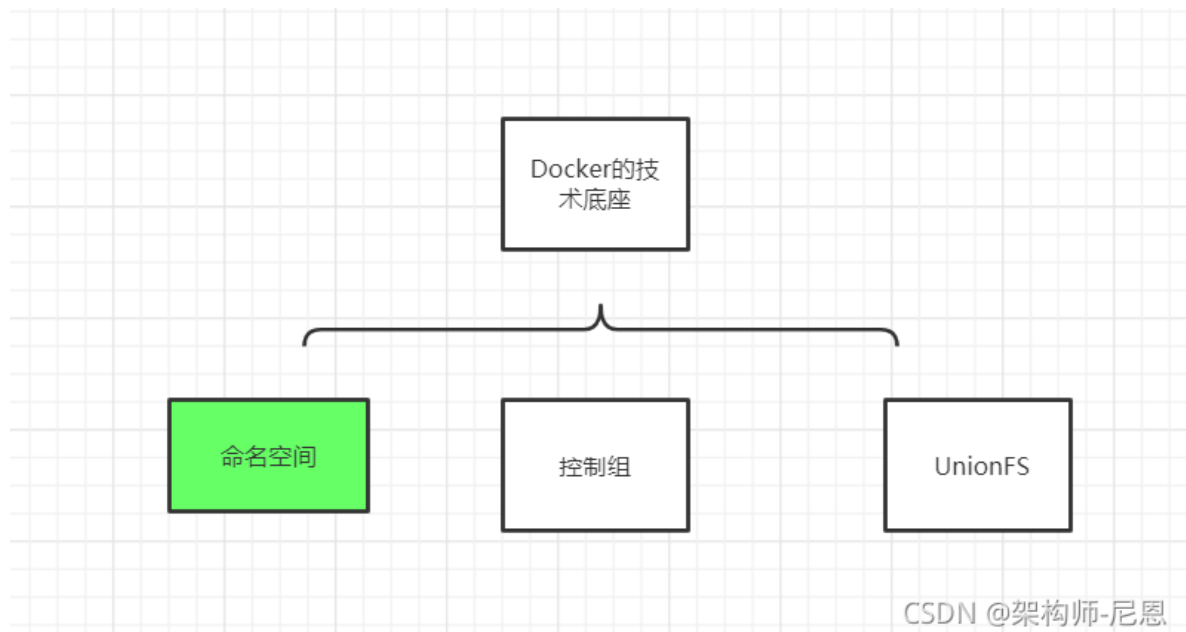
通过这种方式，可以方便更多符合oci标准的容器运行时，接入kubernetes进行集成使用。

可以预见到，通过cri-o，kubernetes在使用的兼容性和广泛性上将会得到进一步加强。



Docker的技术底座：

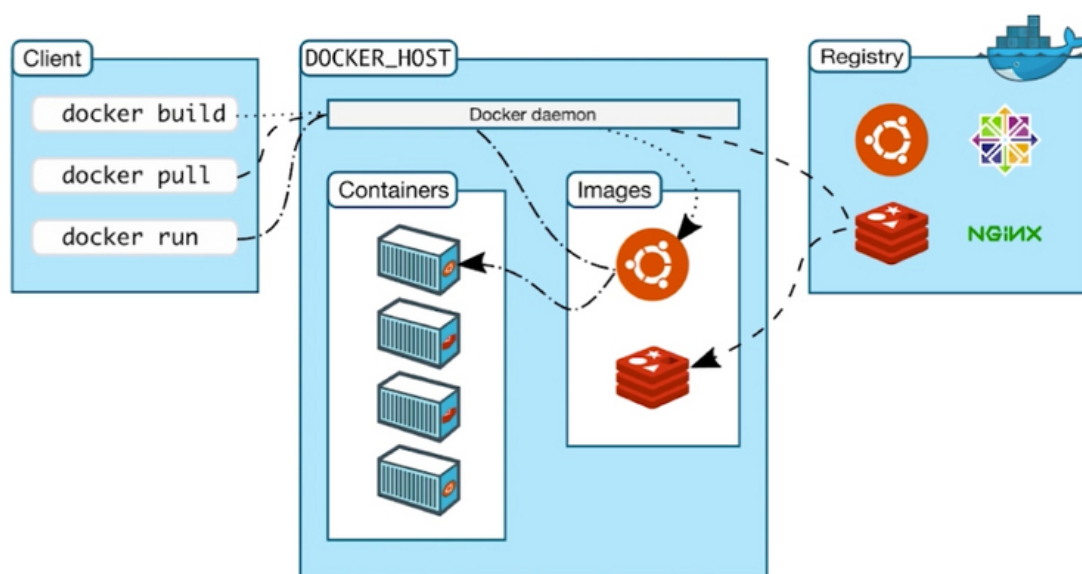
Linux 命名空间、控制组和 UnionFS 三大技术支撑了目前 Docker 的实现，也是 Docker 能够出现的最重要原因。



底层技术支持

其主要利用Linux的底层技术

- Namespaces: 做隔离pid, net, ipc, mnt, uts
- Control groups: 做资源限制
- Union file systems: Container和image的分层



- namespace, 命名空间

命名空间，容器隔离的基础，保证A容器看不到B容器。

6个命名空间：User, Mnt, Network, UTS, IPC, Pid

- cgroups, Cgroups 是 Control Group 的缩写，控制组

cgroups 容器资源统计和隔离

主要用到的cgroups子系统：cpu, blkio, device, freezer, memory

实际上 Docker 是使用了很多 Linux 的隔离功能，让容器看起来像一个轻量级虚拟机在独立运行，容器的本质是被限制了 Namespaces, cgroup, 具有逻辑上独立文件系统，网络的一个进程。

- unionfs 联合文件系统

典型：aufs/overlayfs, 分层镜像实现的基础

UnionFS 联合文件系统

什么是镜像

那么问题来了，没有操作系统，怎么运行程序？

可以在Docker中创建一个centos的镜像文件，这样就能将centos系统集成到Docker中，运行的应用就都是centos的应用。

Image 是 Docker 部署的基本单位，一个 Image 包含了我们的程序文件，以及这个程序依赖的资源的环境。Docker Image 对外是以一个文件的形式展示的（更准确的说是一个 mount 点）。

UnionFS 与AUFS

UnionFS 其实是一种为 Linux 操作系统设计的用于把多个文件系统『联合』到同一个挂载点的文件系统服务。

AUFS 即 Advanced UnionFS 其实就是 UnionFS 的升级版，它能够提供更优秀的性能和效率。

AUFS 作为先进联合文件系统，它能够将不同文件夹中的层联合（Union）到了同一个文件夹中，这些文件夹在 AUFS 中称作分支，整个『联合』的过程被称为联合挂载（Union Mount）。

概念理解起来比较枯燥，最好是有一个真实的例子来帮助我们理解：

首先，我们建立 company 和 home 两个目录，并且分别为他们创造两个文件


```
# tree .
.
|-- company
|   |-- code
|   `-- meeting
`-- home
    |-- eat
    `-- sleep
```

然后我们将通过 mount 命令把 company 和 home 两个目录「联合」起来，建立一个 AUFS 的文件系统，并挂载到当前目录下的 mnt 目录下：

```
# mkdir mnt
# ll
total 20
drwxr-xr-x 5 root root 4096 Oct 25 16:10 ./
drwxr-xr-x 5 root root 4096 Oct 25 16:06 ../
drwxr-xr-x 4 root root 4096 Oct 25 16:06 company/
drwxr-xr-x 4 root root 4096 Oct 25 16:05 home/
drwxr-xr-x 2 root root 4096 Oct 25 16:10 mnt/

# mount -t aufs -o dirs=./home:./company none ./mnt
# ll
total 20
drwxr-xr-x 5 root root 4096 Oct 25 16:10 ./
drwxr-xr-x 5 root root 4096 Oct 25 16:06 ../
drwxr-xr-x 4 root root 4096 Oct 25 16:06 company/
drwxr-xr-x 6 root root 4096 Oct 25 16:10 home/
drwxr-xr-x 8 root root 4096 Oct 25 16:10 mnt/
root@rds-k8s-18-svr0:~/xuran/aufs# tree ./mnt/
./mnt/
|-- code
|-- eat
|-- meeting
`-- sleep

4 directories, 0 files
```

通过 ./mnt 目录结构的输出结果，可以看到原来两个目录下的内容都被合并到了一个 mnt 的目录下。

默认情况下，如果我们不对「联合」的目录指定权限，内核将根据从左至右的顺序将第一个目录指定为可读可写的，其余的都为只读。那么，当我们向只读的目录做一些写入操作的话，会发生什么呢？

```
# echo apple > ./mnt/code
# cat company/code
# cat home/code
apple
```

通过对上面代码段的观察，我们可以看出，当写入操作发生在 company/code 文件时，对应的修改并没有反映到原始的目录中。

而是在 home 目录下又创建了一个名为 code 的文件，并将 apple 写入了进去。

看起来很奇怪的现象，其实这正是 Union File System 的厉害之处：

Union File System 联合了多个不同的目录，并且把他们挂载到一个统一的目录上。

在这些「联合」的子目录中，有一部分是可读可写的，但是有一部分只是可读的。

当你对可读的目录内容做出修改的时候，其结果只会保存到可写的目录下，不会影响只读的目录。

比如，我们可以把我们的服务的源代码目录和一个存放代码修改记录的目录「联合」起来构成一个 AUFS。前者设置只读权限，后者设置读写权限。

那么，一切对源代码目录下文件的修改都只会影响那个存放修改的目录，不会污染原始的代码。

在 AUFS 中还有一个特殊的概念需要提及一下：

branch - 就是各个要被union起来的目录。

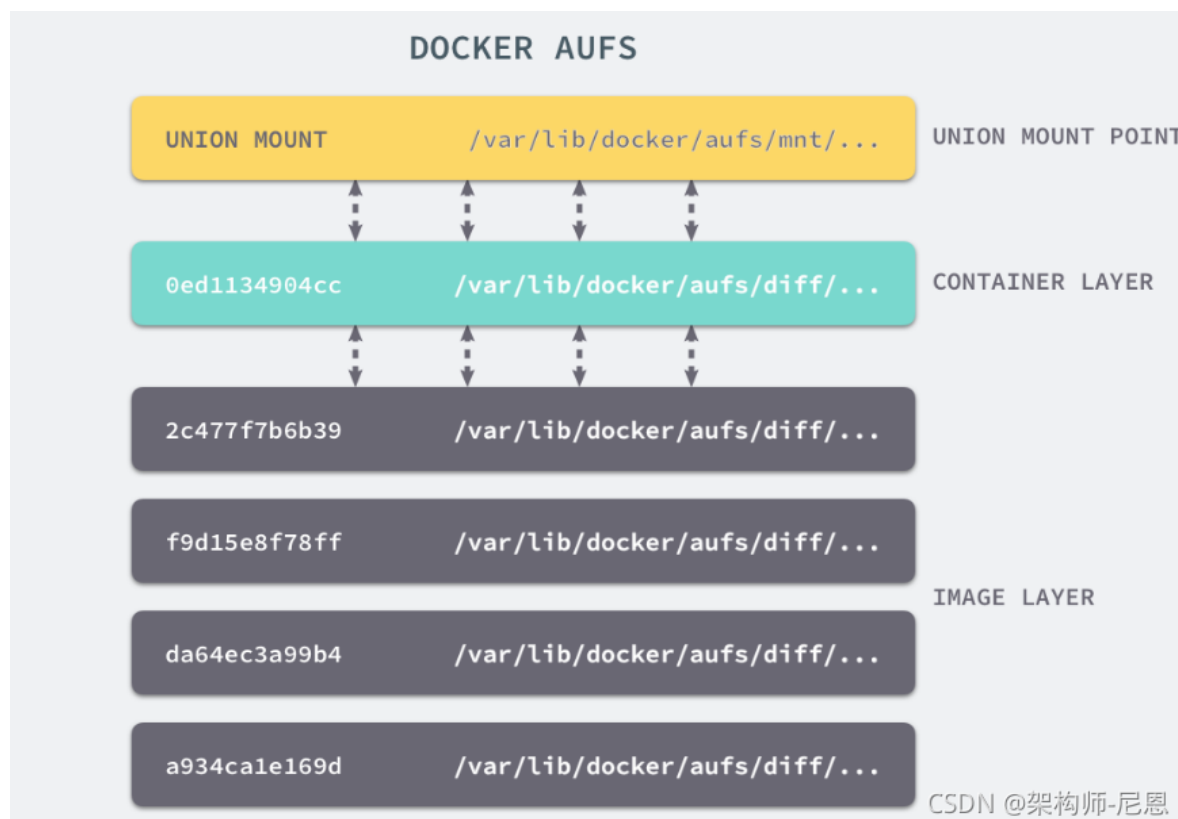
Stack 结构 - AUFS 它会根据branch 被 Union 的顺序形成一个 Stack 的结构，从下至上，最上面的目录是可读写的，其余都是可读的。如果按照我们刚刚执行 aufs 挂载的命令来说，最左侧的目录就对应 Stack 最顶层的 branch。

所以：下面的命令中，最为左侧的为 home，而不是 company

```
mount -t aufs -o dirs=./home:./company none ./mnt
```

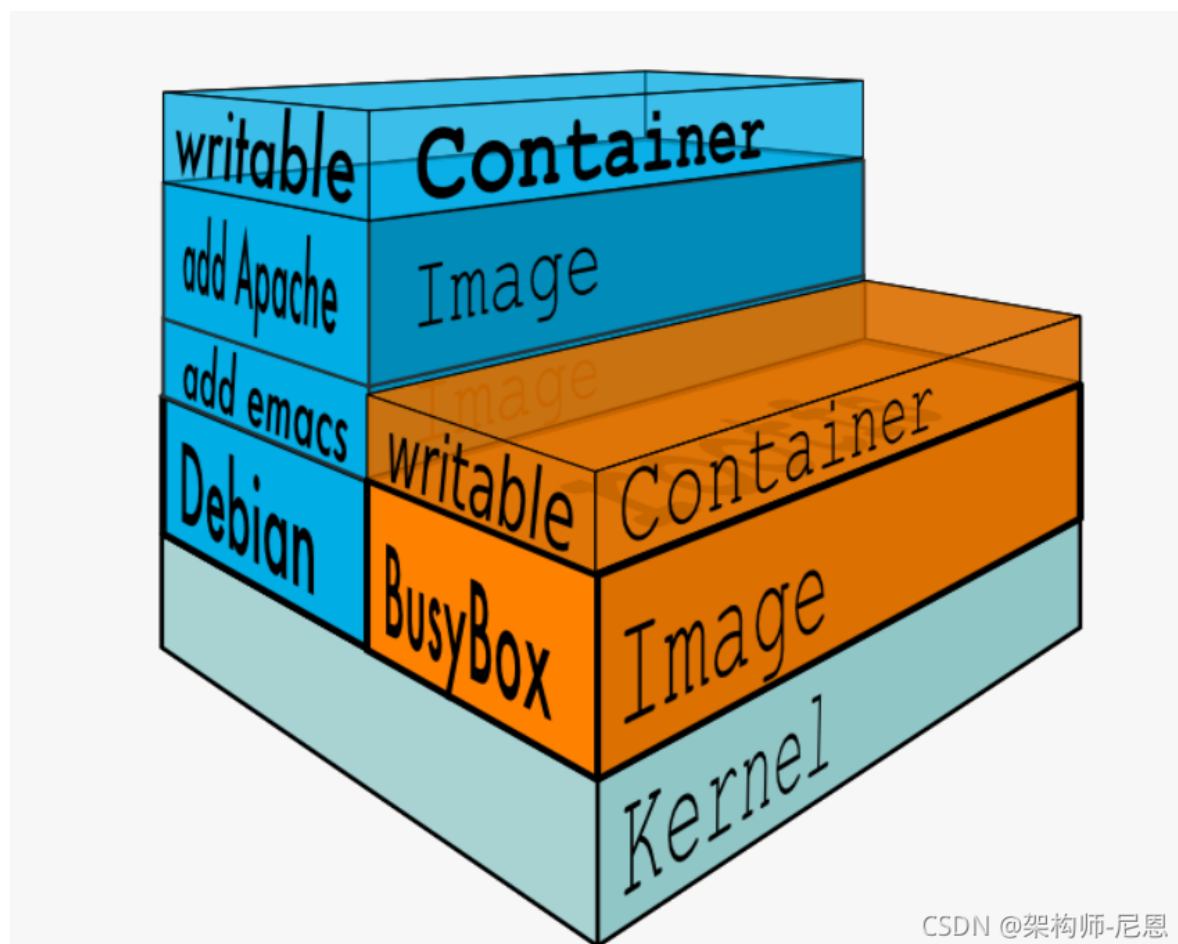
什么是 Docker 镜像分层机制？

首先，让我们来看下 Docker Image 中的 Layer 的概念：



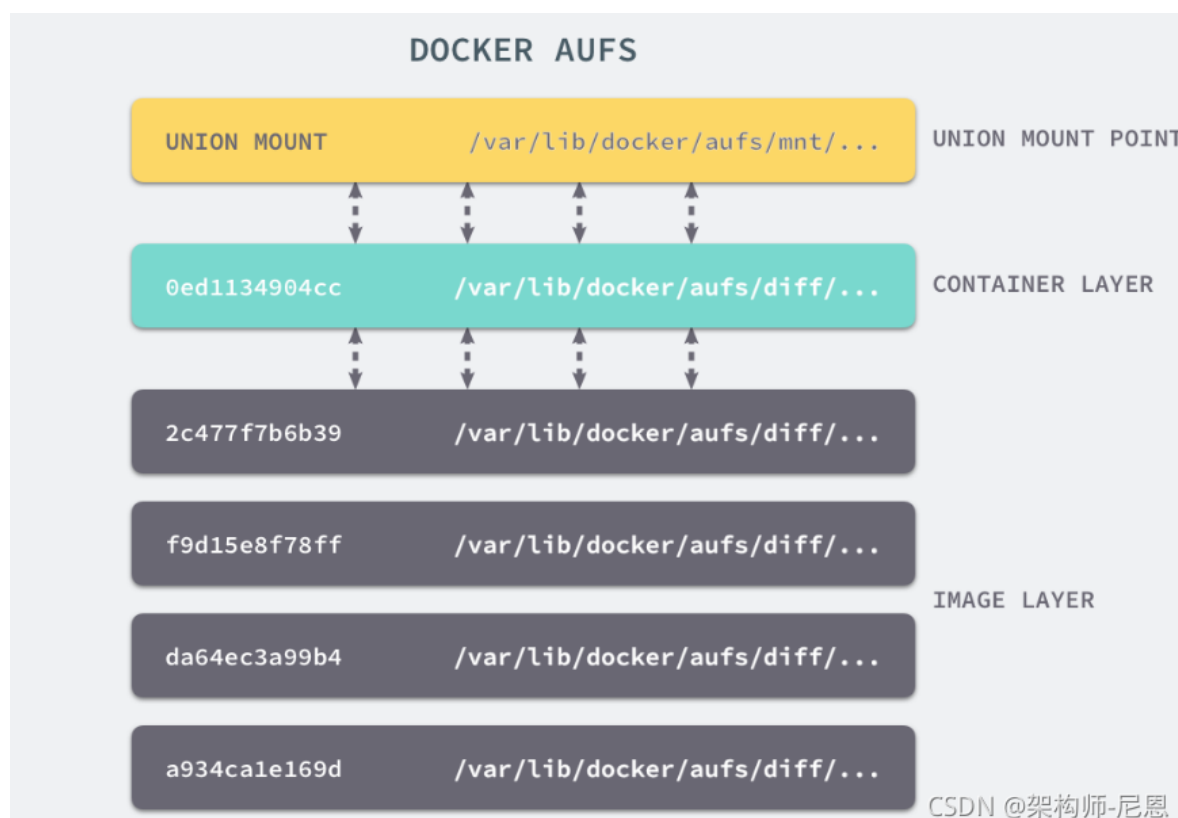
Docker Image 是有一个层级结构的，最底层的 Layer 为 BaseImage（一般为一个操作系统的 ISO 镜像），然后顺序执行每一条指令，生成的 Layer 按照入栈的顺序逐渐累加，最终形成一个 Image。

直观的角度来说，是这个图所示：



CSDN @架构师-尼恩

每一次都是一个被联合的目录，从目录的角度来说，大致如下图所示：



CSDN @架构师-尼恩

Docker Image 如何而来呢?

简单来说, 一个 Image 是通过一个 DockerFile 定义的, 然后使用 docker build 命令构建它。

DockerFile 中的每一条命令的执行结果都会成为 Image 中的一个 Layer。

这里, 我们通过 Build 一个镜像, 来观察 Image 的分层机制:

Dockerfile:

```
# Use an official Python runtime as a parent image
FROM python:2.7-slim

# Set the working directory to /app
WORKDIR /app

# Copy the current directory contents into the container at /app
COPY . /app

# Install any needed packages specified in requirements.txt
RUN pip install --trusted-host pypi.python.org -r requirements.txt

# Make port 80 available to the world outside this container
EXPOSE 80

# Define environment variable
ENV NAME World

# Run app.py when the container launches
CMD ["python", "app.py"]
```

构建结果:

```
root@rds-k8s-18-svr0:~/xuran/exampleimage# docker build -t hello ./
Sending build context to Docker daemon 5.12 kB
Step 1/7 : FROM python:2.7-slim
----> 804b0a01ea83
Step 2/7 : WORKDIR /app
----> Using cache
----> 6d93c5b91703
Step 3/7 : COPY . /app
----> Using cache
----> feddc82d321b
Step 4/7 : RUN pip install --trusted-host pypi.python.org -r requirements.txt
----> Using cache
----> 94695df5e14d
Step 5/7 : EXPOSE 81
----> Using cache
----> 43c392d51dff
Step 6/7 : ENV NAME World
----> Using cache
----> 78c9a60237c8
Step 7/7 : CMD python app.py
----> Using cache
```

```
----> a5ccd4e1b15d
Successfully built a5ccd4e1b15d
```

通过构建结果可以看出，构建的过程就是执行 Dockerfile 文件中我们写入的命令。构建一共进行了7个步骤，每个步骤进行完都会生成一个随机的 ID，来标识这一 layer 中的内容。最后一行的 a5ccd4e1b15d 为镜像的 ID。由于我贴上来的构建过程已经是构建了第二次的结果了，所以可以看出，对于没有任何修改的内容，Docker 会复用之前的结果。

如果 DockerFile 中的内容没有变动，那么相应的镜像在 build 的时候会复用之前的 layer，以便提升构建效率。并且，即使文件内容有修改，那也只会重新 build 修改的 layer，其他未修改的也仍然会复用。

通过了解了 Docker Image 的分层机制，我们多多少少能够感觉到，Layer 和 Image 的关系与 AUFS 中的联合目录和挂载点的关系比较相似。

而 Docker 也正是通过 AUFS 来管理 Images 的。

Namespaces

在Linux系统中，Namespace是在内核级别以一种抽象的形式来封装系统资源，通过将系统资源放在不同的Namespace中，来实现资源隔离的目的。不同的Namespace程序，可以享有一份独立的系统资源。

命名空间（namespaces）是 Linux 为我们提供的用于分离进程树、网络接口、挂载点以及进程间通信等资源的方法。在日常使用 Linux 或者 macOS 时，我们并没有运行多个完全分离的服务器的需要，但是如果我们在服务器上启动了多个服务，这些服务其实会相互影响的，每一个服务都能看到其他服务的进程，也可以访问宿主机器上的任意文件，这是很多时候我们都不愿意看到的，我们更希望运行在同一台机器上的不同服务能做到完全隔离，就像运行在多台不同的机器上一样。

Linux 的命名空间机制提供了以下七种不同的命名空间，包括：

- CLONE_NEWCGROUP、
- CLONE_NEWIPC、
- CLONE_NEWNET、
- CLONE_NEWNS、
- CLONE_NEWPID、
- CLONE_NEWUSER 和
- CLONE_NEWUTS、

通过这七个选项，我们能在创建新的进程时，设置新进程应该在哪些资源上与宿主机器进行隔离。具体如下：

Namespace	Flag	Page	Isolates
Cgroup	CLONE_NEWCGROUP	cgroup_namespaces	Cgroup root directory
IPC	CLONE_NEWIPC	ipc_namespaces	System V IPC,POSIX message queues 隔离进程间通信
Network	CLONE_NEWNET	network_namespaces	Network devices,stacks, ports, etc. 隔离网络资源
Mount	CLONE_NEWNS	mount_namespaces	Mount points 隔离文件系统挂载点
PID	CLONE_NEWPID	pid_namespaces	Process IDs 隔离进程的ID
Time	CLONE_NEWTIME	time_namespaces	Boot and monotonic clocks
User	CLONE_NEWUSER	user_namespaces	User and group IDs 隔离用户和用户组的ID
UTS	CLONE_NEWUTS	uts_namespaces	Hostname and NIS domain name 隔离主机名和域名信息

这里提出一个问题，在宿主机上启动两个容器，在这两个容器内都各有一个 PID=1的进程，众所周知，Linux 里 PID 是唯一的，既然 Docker 不是跑在宿主机上的两个虚拟机，那么它是如何实现在宿主机上运行两个相同 PID 的进程呢？

这里就用到了 Linux Namespaces，它其实是 Linux 创建新进程时的一个可选参数，在 Linux 系统中创建进程的系统调用是 clone()方法。

```
int clone(int (*fn) (void *), void *child_stack,
          int flags, void *arg, . . .
          /* pid_t *ptid, void *newtls, pid_t *ctid */ );
```

```
int clone(int (*fn)(void *), void *child_stack,
          int flags, void *arg, ...
          /* pid_t *ptid, void *newtls, pid_t *ctid */ );
```

通过调用这个方法，这个进程会获得一个独立的进程空间，它的 pid 是1，并且看不到宿主机上的其他进程，这也就是在容器内执行 PS 命令的结果。

不仅仅是 PID，当你启动容器之后，Docker 会为此容器创建一系列其他 namespaces。

这些 namespaces 提供了不同层面的隔离。容器的运行受到各个层面 namespace 的限制。

Docker Engine 使用了以下 Linux 的隔离技术：

The pid namespace: 管理 PID 命名空间 (PID: Process ID).

The net namespace: 管理网络命名空间 (NET: Networking).

The ipc namespace: 管理进程间通信命名空间 (IPC: InterProcess Communication).

The mnt namespace: 管理文件系统挂载点命名空间 (MNT: Mount).

The uts namespace: Unix 时间系统隔离. (UTS: Unix Timesharing System).

通过这些技术，运行时的容器得以看到一个和宿主机上其他容器隔离的环境。

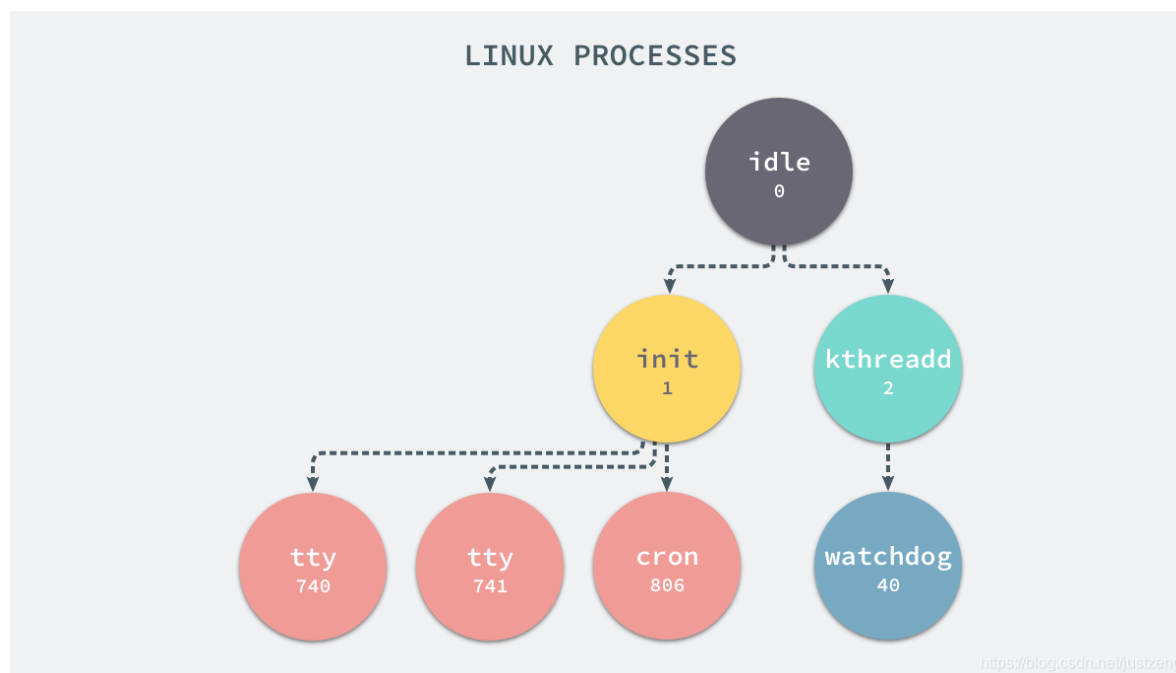
进程隔离

进程是 Linux 以及现在操作系统中非常重要的概念，它表示一个正在执行的程序，也是在现代分时系统中的一个任务单元。

在每一个 *nix 的操作系统上，我们都能够通过 ps 命令打印出当前操作系统中正在执行的进程，比如在 Ubuntu 上，使用该命令就能得到以下的结果：

```
| $ ps -ef
UID      PID     PPID    C   STIME  TTY          TIME CMD
root      1         0    0  Apr08   ?        00:00:09 /sbin/init
root      2         0    0  Apr08   ?        00:00:00 [kthreadd]
root      3         2    0  Apr08   ?        00:00:05 [ksoftirqd/0]
root      5         2    0  Apr08   ?        00:00:00 [kworker/0:0H]
root      7         2    0  Apr08   ?        00:07:10 [rcu_sched]
root     39         2    0  Apr08   ?        00:00:00 [migration/0]
root     40         2    0  Apr08   ?        00:01:54 [watchdog/0]
```

当前机器上有很多的进程正在执行，在上述进程中两个非常特殊，一个是 pid 为 1 的 /sbin/init 进程，另一个是 pid 为 2 的 kthreadd 进程，这两个进程都是被 Linux 中的上帝进程 idle 创建出来的，其中前者负责执行内核的一部分初始化工作和系统配置，也会创建一些类似 getty 的注册进程，而后者负责管理和调度其他的内核进程。



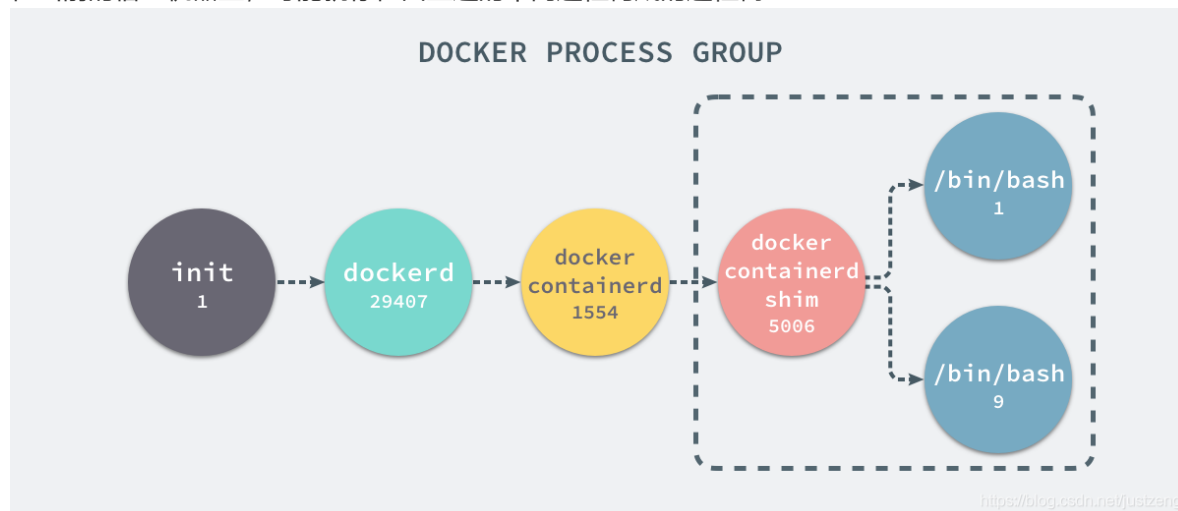
如果我们在当前的 Linux 操作系统下运行一个新的 Docker 容器，并通过 exec 进入其内部的 bash 并打印其中的全部进程，我们会得到以下的结果：

```
UID      PID  PPID  C  STIME TTY          TIME CMD
root     29407      1  0  Nov16 ?           00:08:38 /usr/bin/dockerd --raw-logs
root     1554  29407  0  Nov19 ?           00:03:28 docker-containerd -l
unix:///var/run/docker/libcontainerd/docker-containerd.sock --metrics-interval=0
--start-timeout 2m --state-dir /var/run/docker/libcontainerd/containerd --shim
docker-containerd-shim --runtime docker-runc
root      5006  1554  0  08:38 ?           00:00:00 docker-containerd-shim
b809a2eb3630e64c581561b08ac46154878ff1c61c6519848b4a29d412215e79
/var/run/docker/libcontainerd/b809a2eb3630e64c581561b08ac46154878ff1c61c6519848b
4a29d412215e79 docker-runc
```

在新的容器内部执行 ps 命令打印出了非常干净的进程列表，只有包含当前 ps -ef 在内的三个进程，在宿主机上的几十个进程都已经消失不见了。

当前的 Docker 容器成功将容器内的进程与宿主机中的进程隔离，如果我们在宿主机上打印当前的全部进程时，会得到下面三条与 Docker 相关的结果：

在当前的宿主机上，可能就存在由上述的不同进程构成的进程树：



这就是在使用 `clone(2)` 创建新进程时传入 `CLONE_NEWPID` 实现的，也就是使用 Linux 的命名空间实现进程的隔离，Docker 容器内部的任意进程都对宿主机的进程一无所知。

```
containerRouter.postContainersStart
├── daemon.ContainerStart
├── daemon.createSpec
│   ├── setNamespaces
│   │   └── setNamespace
```

Docker 的容器就是使用上述技术实现与宿主机的进程隔离，当我们每次运行 `docker run` 或者 `docker start` 时，都会下面的方法中创建一个用于设置进程间隔离的 `Spec`：

```
func (daemon *Daemon) createSpec(c *container.Container) (*specs.Spec, error) {
    s := oci.DefaultSpec()

    // ...
    if err := setNamespaces(daemon, &s, c); err != nil {
        return nil, fmt.Errorf("linux spec namespaces: %v", err)
    }

    return &s, nil
}
```

在 `setNamespaces` 方法中不仅会设置进程相关的命名空间，还会设置与用户、网络、IPC 以及 UTS 相关的命名空间：

```
func setNamespaces(daemon *Daemon, s *specs.Spec, c *container.Container) error {
    // user
    // network
    // ipc
    // uts

    // pid
    if c.HostConfig.PidMode.IsContainer() {
        ns := specs.LinuxNamespace{Type: "pid"}
        pc, err := daemon.getPidContainer(c)
        if err != nil {
```

```

        return err
    }
    ns.Path = fmt.Sprintf("/proc/%d/ns/pid", pc.State.GetPID())
    setNamespace(s, ns)
} else if c.HostConfig.PidMode.IsHost() {
    oci.RemoveNamespace(s, specs.LinuxNamespaceType("pid"))
} else {
    ns := specs.LinuxNamespace{Type: "pid"}
    setNamespace(s, ns)
}

return nil
}

```

所有命名空间相关的设置 Spec 最后都会作为 Create 函数的入参在创建新的容器时进行设置：

```

daemon.containerd.Create(context.Background(), container.ID, spec,
createOptions)

```

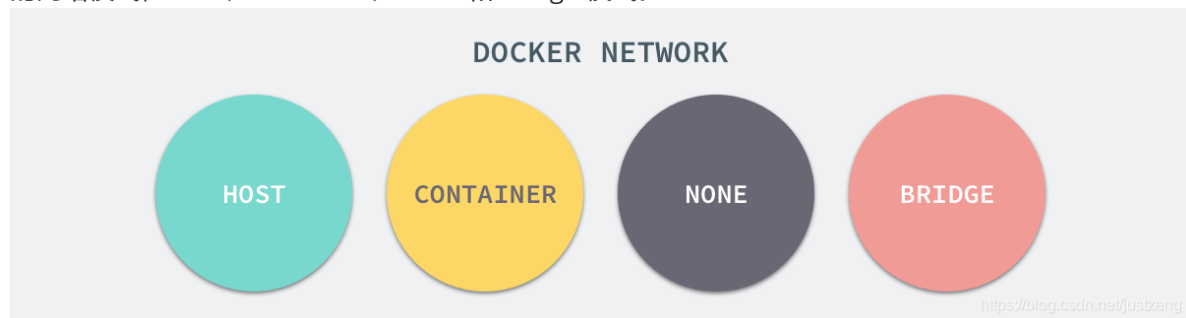
所有与命名空间相关的设置都是在上述的两个函数中完成的，Docker 通过命名空间成功完成了与宿主主机进程和网络的隔。

网络隔离

如果 Docker 的容器通过 Linux 的命名空间完成了与宿主机进程的网络隔离，但是却有没有办法通过宿主机的网络与整个互联网相连，就会产生很多限制。

所以 Docker 虽然可以通过命名空间创建一个隔离的网络环境，但是 Docker 中的服务仍然需要与外界相连才能发挥作用。

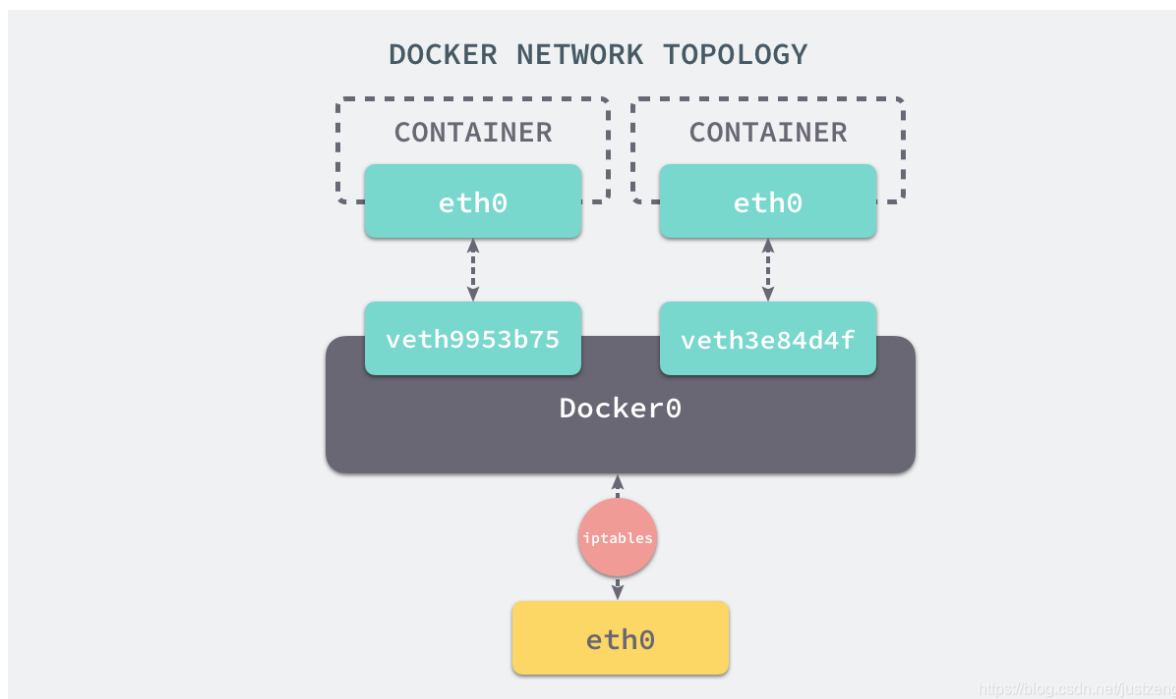
每一个使用 docker run 启动的容器其实都具有单独的网络命名空间，Docker 为我们提供了四种不同的网络模式，Host、Container、None 和 Bridge 模式。



在这一部分，我们将介绍 Docker 默认的网络设置模式：网桥模式。

在这种模式下，除了分配隔离的网络命名空间之外，Docker 还会为所有的容器设置 IP 地址。

当 Docker 服务器在主机上启动之后会创建新的虚拟网桥 docker0，随后在该主机上启动的全部服务在默认情况下都与该网桥相连。



在默认情况下，每一个容器在创建时都会创建一对虚拟网卡，两个虚拟网卡组成了数据的通道，其中一个会放在创建的容器中，会加入到名为 docker0 网桥中。

我们可以使用如下的命令来查看当前网桥的接口：

```
$ brctl show
bridge name bridge id        STP enabled interfaces
docker0      8000.0242a6654980    no        veth3e84d4f
            veth9953b75
```

docker0会为每一个容器分配一个新的 IP 地址并将 docker0 的 IP 地址设置为默认的网关。

网桥 docker0 通过 iptables 中的配置与宿主机上的网卡相连，所有符合条件的请求都会通过 iptables 转发到 docker0 并由网桥分发给对应的机器。

```
$ iptables -t nat -L
Chain PREROUTING (policy ACCEPT)
target     prot opt source                destination
DOCKER     all  --  anywhere              anywhere           ADDRTYPE match
dst-type LOCAL

Chain DOCKER (2 references)
target     prot opt source                destination
RETURN     all  --  anywhere              anywhere
```

我们在当前的机器上使用 `docker run -d -p 6379:6379 redis` 命令启动了一个新的 Redis 容器，在这之后我们再查看当前 iptables 的 NAT 配置就会看到在 DOCKER 的链中出现了一条新的规则：

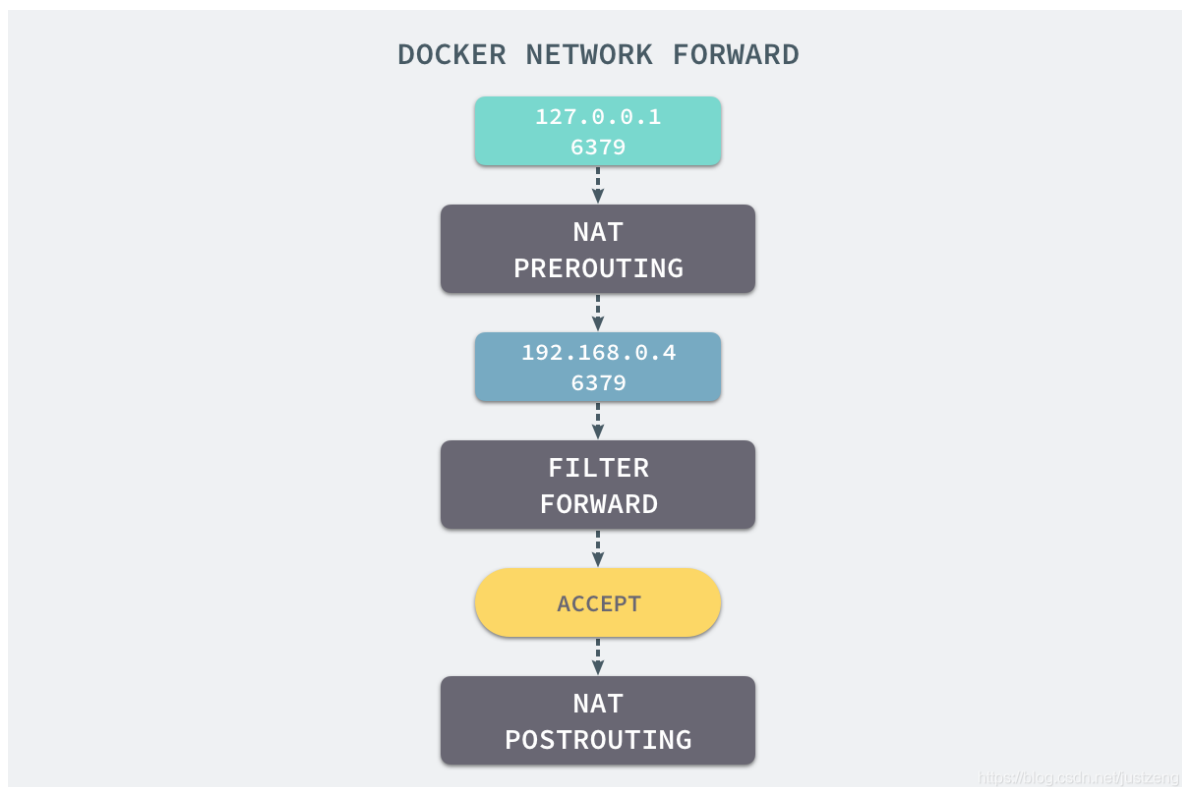
```
DNAT       tcp  --  anywhere              anywhere           tcp dpt:6379
to:192.168.0.4:6379
```

上述规则会将从任意源发送到当前机器 6379 端口的 TCP 包转发到 192.168.0.4:6379 所在的地址上。

这个地址其实也是 Docker 为 Redis 服务分配的 IP 地址，如果我们在当前机器上直接 ping 这个 IP 地址就会发现它是可以访问到的：

```
$ ping 192.168.0.4
PING 192.168.0.4 (192.168.0.4) 56(84) bytes of data.
64 bytes from 192.168.0.4: icmp_seq=1 ttl=64 time=0.069 ms
64 bytes from 192.168.0.4: icmp_seq=2 ttl=64 time=0.043 ms
^C
--- 192.168.0.4 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 999ms
rtt min/avg/max/mdev = 0.043/0.056/0.069/0.013 ms
```

从上述的一系列现象，我们就可以推测出 Docker 是如何将容器的内部的端口暴露出来并对数据包进行转发的了；当有 Docker 的容器需要将服务暴露给宿主机，就会为容器分配一个 IP 地址，同时向 iptables 中追加一条新的规则。



当我们使用 redis-cli 在宿主机的命令行中访问 127.0.0.1:6379 的地址时，经过 iptables 的 NAT PREROUTING 将 ip 地址定向到了 192.168.0.4，重定向过的数据包就可以通过 iptables 中的 FILTER 配置，最终在 NAT POSTROUTING 阶段将 ip 地址伪装成 127.0.0.1，到这里虽然从外面看起来我们请求的是 127.0.0.1:6379，但是实际上请求的已经是 Docker 容器暴露出的端口了。

```
$ redis-cli -h 127.0.0.1 -p 6379 ping
PONG
```

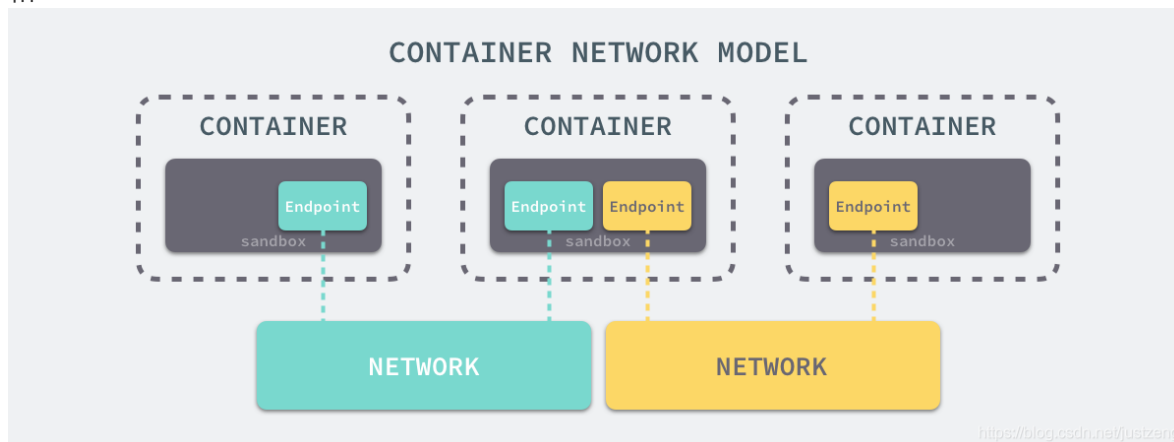
Docker 通过 Linux 的命名空间实现了网络的隔离，又通过 iptables 进行数据包转发，让 Docker 容器能够优雅地为宿主机或者其他容器提供服务。

Libnetwork

整个网络部分的功能都是通过 Docker 拆分出来的 libnetwork 实现的，它提供了一个连接不同容器的实现，同时也能够为应用给出一个能够提供一致的编程接口和网络层抽象的容器网络模型。

The goal of libnetwork is to deliver a robust Container Network Model that provides a consistent programming interface and the required network abstractions for applications.

libnetwork 中最重要的概念，容器网络模型由以下的几个主要组件组成，分别是 Sandbox、Endpoint 和 Network：



在容器网络模型中，每一个容器内部都包含一个 Sandbox，其中存储着当前容器的网络栈配置，包括容器的接口、路由表和 DNS 设置，Linux 使用网络命名空间实现这个 Sandbox，每一个 Sandbox 中都可能会有一个或多个 Endpoint，在 Linux 上就是一个虚拟的网卡 veth，Sandbox 通过 Endpoint 加入到对应的网络中，这里的网络可能就是我们在上面提到的 Linux 网桥或者 VLAN。

挂载点

虽然我们已经通过 Linux 的命名空间解决了进程和网络隔离的问题，在 Docker 进程中我们已经没有办法访问宿主机上的其他进程并且限制了网络的访问，但是 Docker 容器中的进程仍然能够访问或者修改宿主机上的其他目录，这是我们不希望看到的。

在新的进程中创建隔离的挂载点命名空间需要在 clone 函数中传入 CLONE_NEWNS，这样子进程就能得到父进程挂载点的拷贝，如果不传入这个参数子进程对文件系统的读写都会同步回父进程以及整个主机的文件系统。

如果一个容器需要启动，那么它一定需要提供一个根文件系统（rootfs），容器需要使用这个文件系统来创建一个新的进程，所有二进制的执行都必须在这个根文件系统中。

LIBCONTAINER FILESYSTEM	
Path	Type
/proc	proc
/dev	tmpfs
/dev/shm	tmpfs
/dev/mqueue	mqueue
/dev/pts	devpts
/sys	sysfs

https://blog.csdn.net/justzeng

想要正常启动一个容器就需要在 rootfs 中挂载以上的几个特定的目录，除了上述的几个目录需要挂载之外我们还需要建立一些符号链接保证系统 IO 不会出现问题。

为了保证当前的容器进程没有办法访问宿主机上其他目录，我们在这里还需要通过 libcotainer 提供的 pivot_root 或者 chroot 函数改变进程能够访问个文件目录的根节点。

```
// pivot_root
put_old = mkdir(...);
pivot_root(rootfs, put_old);
chdir("/");
umount(put_old, MS_DETACH);
rmdir(put_old);

// chroot
mount(rootfs, "/", NULL, MS_MOVE, NULL);
chroot(".");
chdir("/");
```

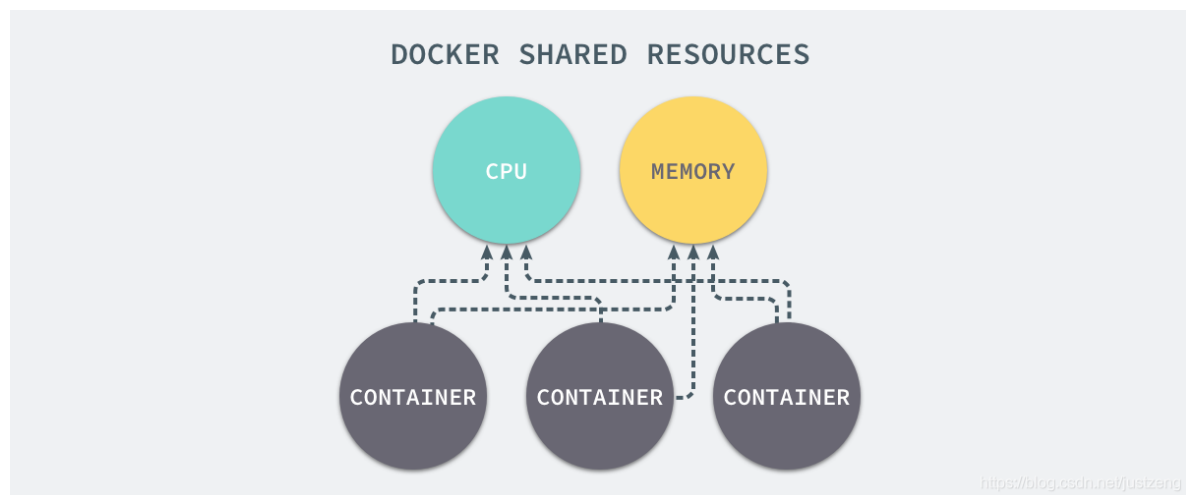
到这里我们就将容器需要的目录挂载到了容器中，同时也禁止当前的容器进程访问宿主机上的其他目录，保证了不同文件系统的隔离。

Chroot

在这里不得不简单介绍一下 chroot (change root)，在 Linux 系统中，系统默认的目录都是以 / 也就是根目录开头的，chroot 的使用能够改变当前的系统根目录结构，通过改变当前系统的根目录，我们能够限制用户的权利，在新的根目录下并不能够访问旧系统根目录的结构个文件，也就建立了一个与原系统完全隔离的目录结构。

CGroups物理资源限制分组

我们通过 Linux 的命名空间为新创建的进程隔离了文件系统、网络并与宿主机之间的进程相互隔离，但是命名空间并不能够为我们提供物理资源上的隔离，比如 CPU 或者内存，如果在同一台机器上运行了多个对彼此以及宿主机一无所知的『容器』，这些容器却共同占用了宿主机的物理资源。



如果其中的某一个容器正在执行 CPU 密集型的任务，那么就会影响其他容器中任务的性能与执行效率，导致多个容器相互影响并且抢占资源。如何对多个容器的资源使用进行限制就成了解决进程虚拟资源隔离之后的主要问题，而 Control Groups (简称 CGroups) 就是能够隔离宿主机上的物理资源，例如 CPU、内存、磁盘 I/O 和网络带宽。

每一个 CGroup 都是一组被相同的标准和参数限制的进程，不同的 CGroup 之间是有层级关系的，也就是说它们之间可以从父类继承一些用于限制资源使用的标准和参数。

Linux 的 CGroup 能够为一组进程分配资源，也就是我们在上面提到的 CPU、内存、网络带宽等资源，通过对资源的分配。

Linux 使用文件系统来实现 CGroup，我们可以直接使用下面的命令查看当前的 CGroup 中有哪些子系统：

```
$ ls /sys/fs/cgroup
cpuset /sys/fs/cgroup/cpuset
cpu /sys/fs/cgroup/cpu
cpuacct /sys/fs/cgroup/cpuacct
memory /sys/fs/cgroup/memory
devices /sys/fs/cgroup/devices
freezer /sys/fs/cgroup/freezer
blkio /sys/fs/cgroup/blkio
perf_event /sys/fs/cgroup/perf_event
hugetlb /sys/fs/cgroup/hugetlb
```

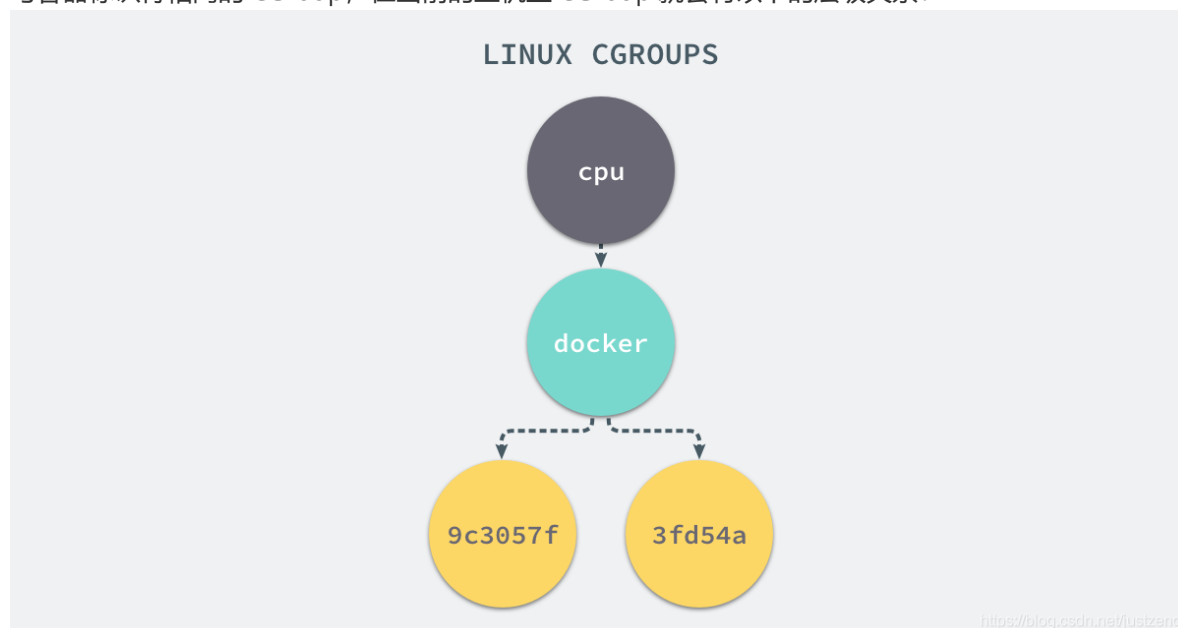
大多数 Linux 的发行版都有着非常相似的子系统，而之所以将上面的 cpuset、cpu 等东西称作子系统，是因为它们能够为对应的控制组分配资源并限制资源的使用。

如果我们想要创建一个新的 cgroup 只需要在想要分配或者限制资源的子系统下面创建一个新的文件夹，然后这个文件夹下就会出现很多的内容，如果你在 Linux 上安装了 Docker，你就会发现所有子系统的目录下都有一个名为 Docker 的文件夹：

```
$ ls /sys/fs/cgroup/cpu
cgroup.clone_children
...
cpu.stat
docker
notify_on_release
release_agent
tasks

$ ls /sys/fs/cgroup/cpu/docker/
9c3057f1291b53fd54a3d12023d2644efe6a7db6ddf330436ae73ac92d401cf1
cgroup.clone_children
...
cpu.stat
notify_on_release
release_agent
tasks
```

9c3057xxx 其实就是我们运行的一个 Docker 容器，启动这个容器时，Docker 会为此容器创建一个与容器标识符相同的 CGroup，在当前的主机上 CGroup 就会有以下的层级关系：



每一个 CGroup 下面都有一个 tasks 文件，其中存储着属于当前控制组的所有进程的 pid，作为负责 cpu 的子系统，cpu.cfs_quota_us 文件中的内容能够对 CPU 的使用作出限制，如果当前文件的内容为 50000，那么当前控制组中的全部进程的 CPU 占用率不能超过 50%。

如果系统管理员想要控制 Docker 某个容器的资源使用率就可以在 docker 这个父控制组下面找到对应的子控制组并且改变它们对应文件的内容，当然我们也可以直接在程序运行时就使用参数，让 Docker 进程去改变相应文件中的内容。

当我们使用 Docker 关闭掉正在运行的容器时，Docker 的子控制组对应的文件夹也会被 Docker 进程移除，Docker 在使用 CGroup 时其实也只是做了一些创建文件夹改变文件内容的文件操作，不过 CGroup 的使用也确实解决了我们限制子容器资源占用的问题，系统管理员能够为多个容器合理的分配资源并且不会出现多个容器互相抢占资源的问题。

总之：dockers=LXC+AUFS

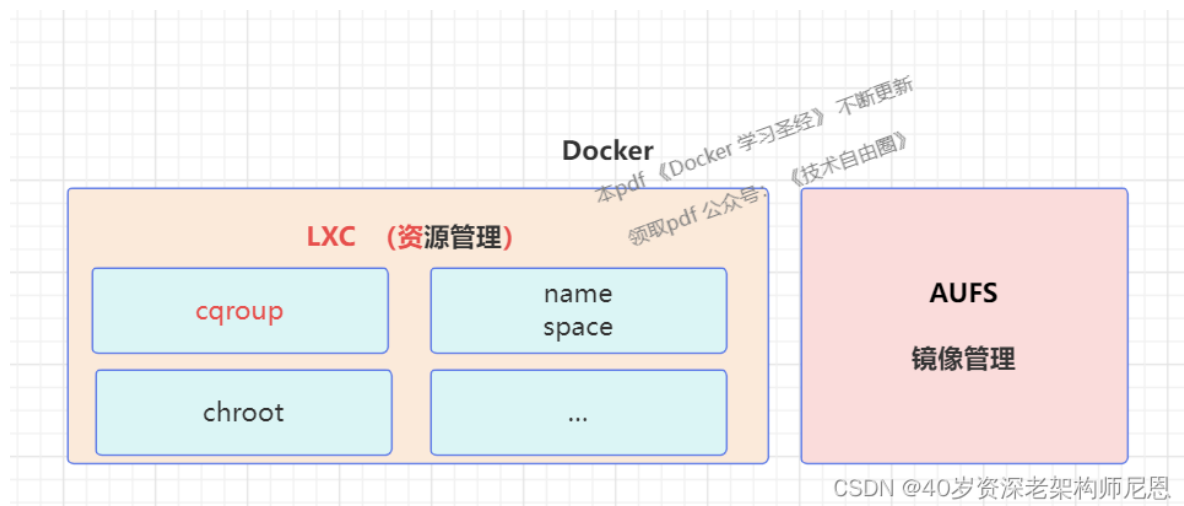
docker为LXC+AUFS组合：

- LXC负责资源管理
- AUFS负责镜像管理；

而LXC包括cgroup，namespace，chroot等组件，并通过cgroup资源管理，

那么，从资源管理的角度来看，Docker，LXC，Cgroup三者的关系是怎样的呢？

cgroup是在底层落实资源管理，LXC在cgroup上面封装了一层，随后，docker有在LXC封装了一层；



Cgroup其实就是linux提供的一种限制记录，隔离进程组所使用的物理资源管理机制；

也就是说，**Cgroup**是**LXC**为实现虚拟化所使用资源管理手段，我们可以这样说，底层没有**cgroup**支持，也就没有**lxc**，更别说**docker**的存在了，这是我们需要掌握和理解的，三者之间的关系概念

我们在把重心转移到**LXC**这个相当于中间件上，上述我们提到**LXC**是建立在**cgroup**基础上的，

我们可以粗略的认为**LXC=Cgroup+namespace+Chroot+veth+用户控制脚本**；

LXC利用内核的新特性（**cgroup**）来提供用户空间的对象，用来保证资源的隔离和对应用系统资源的限制；

Docker容器的文件系统最早是建立在**Aufs**基础上的，**Aufs**是一种Union FS，

简单来说就是支持将不同的目录挂载到同一个虚拟文件系统之下，并实现一种 **layer**的概念，

由于**Aufs**未能加入到linux内核中，考虑到兼容性的问题，便加入了**Devicemapper**的支持，**Docker**目前默认是建立在**Devicemapper**基础上，

devicemapper用户控件相关部分主要负责配置具体的策略和控制逻辑，

比如逻辑设备和哪些物理设备建立映射，怎么建立这些映射关系等，而具体过滤和重定向IO请求的工作有内核中相关代码完成，

因此整个device mapper机制由两部分组成--内核空间的device mapper驱动，

用户控件的device mapper库以及它提供的dmsetup工具；

深入解读docker网络

当你开始大规模使用**docker**时，你会发现需要了解很多关于网络的知识。**docker**作为目前最火的轻量级容器技术，有很多令人称道的功能，如**docker**的镜像管理。然而，**docker**同样有着很多不完善的地方，网络方面就是**Docker**比较薄弱的部分。因此，作为一名运维工程师有必要深入了解**docker**的网络知识，以满足更高的网络需求。作为一名微服开发工程师，简单了解**docker**网络环节即可。本章节首先介绍了**Docker**自身的3种local网络工作方式，然后介绍一些**docker**自定义网络模式。

docker安装后会自动创建3种网络：

- bridge

- host
- none

```
docker network ls
```

docker网络理论部分

docker使用Linux桥接网卡，在宿主机虚拟一个docker容器网桥(docker0)，docker启动一个容器时会根据docker网桥的网段分配给容器一个IP地址，称为Container-IP，同时docker网桥是每个容器的默认网关。

因为在同一宿主机内的容器都接入同一个网桥，这样容器之间就能够通过容器的Container-IP直接通信。

docker网桥是宿主机虚拟出来的，并不是真实存在的网络设备，外部网络是无法寻址到的，这也意味着外部网络无法通过直接Container-IP访问到容器。

如果容器希望外部访问能够访问到，可以通过映射容器端口到宿主机（端口映射），即docker run创建容器时候通过 -p 或 -P 参数来启用，访问容器的时候就通过[宿主机IP]:[容器端口]访问容器。

```
使用命令查看docker网络部分
docker info
```

Docker网络模式

Docker网络模式	配置	说明
host模式	-net=host	容器和宿主机共享Network namespace。容器将不会虚拟出自己的网卡，配置自己的IP等，而是使用宿主机的IP和端口。
container模式	-net=container:NAME_or_ID	容器和另外一个容器共享Network namespace。kubernetes中的pod就是多个容器共享一个Network namespace。创建的容器不会创建自己的网卡，配置自己的IP，而是和一个指定的容器共享IP、端口范围。
none模式	-net=none	容器有独立的Network namespace，并没有对其进行任何网络设置，如分配veth pair 和网桥连接，配置IP等。该模式关闭了容器的网络功能。
bridge模式	-net=bridge	（默认为该模式）。此模式会为每一个容器分配、设置IP等，并将容器连接到一个docker0虚拟网桥，通过docker0网桥以及iptables nat表配置与宿主机通信。
Macvlan network	无	容器具备Mac地址，使其显示为网络上的物理设备
Overlay	无	（覆盖网络）：利用VXLAN实现的bridge模式

bridge模式

默认的网络模式。bridge模式下容器没有一个公有ip,只有宿主机可以直接访问,外部主机是不可见的,但容器通过宿主机的NAT规则后可以访问外网。

Bridge 桥接模式的实现步骤主要如下:

- Docker Daemon 利用 veth pair 技术,在宿主机上创建两个虚拟网络接口设备,假设为veth0 和 veth1。而veth pair 技术的特性可以保证无论哪一个 veth 接收到网络报文,都会将报文传输给另一方。
- Docker Daemon 将 veth0 附加到 Docker Daemon 创建的 docker0网桥上。保证宿主机的网络报文可以发往 veth0;
- Docker Daemon 将 veth1 添加到 Docker Container 所属的 namespace 下,并被改名为eth0。如此一来,保证宿主机的网络报文若发往 veth0,则立即会被 eth0 接收,实现宿主机到Docker Container网络的联通性;同时,也保证 Docker Container 单独使用 eth0,实现容器网络环境的隔离性。

Bridge桥接模式的缺陷:

- 1.最明显的是,该模式下 Docker Container 不具有一个公有 IP,即和宿主机的 eth0 不处于同一个网段。导致的结果是宿主机以外的世界不能直接和容器进行通信。
- 2.虽然 NAT 模式经过中间处理实现了这一点,但是 NAT 模式仍然存在问题与不便,如:容器均需要在宿主机上竞争端口,容器内部服务的访问者需要使用服务发现获知服务的外部端口等。
- 3.另外 NAT 模式由于是在三层网络上的实现手段,故肯定会影响网络的传输效率。

注意:

veth设备是成双成对出现的,一端是容器内部命名为eth0,一端是加入到网桥并命名的veth(通常命名为veth),它们组成了一个数据传输通道,一端进一端出,veth设备连接了两个网络设备并实现了数据通信

host模式

相当于Vmware中的NAT模式,与宿主机在同一个网络中,但没有独立IP地址。

如果启动容器的时候使用host模式,那么这个容器将不会获得一个独立的Network Namespace,而是和宿主机共用一个Network Namespace。容器将不会虚拟出自己的网卡,配置自己的IP等,而是使用宿主机的IP和端口。但是,容器的其他方面,如文件系统、进程列表等还是和宿主机隔离的。

使用host模式的容器可以直接使用宿主机的IP地址与外界通信,容器内部的服务端口也可以使用宿主机的端口,不需要进行NAT,host最大的优势就是网络性能比较好,但是docker host上已经使用的端口就不能再用了,网络的隔离性不好。

host网络模式需要在容器创建时指定--network=host

host 模式是 bridge 桥接模式很好的补充。采用 host 模式的 Docker Container,可以直接使用宿主机的 IP地址与外界进行通信,若宿主机的 eth0 是一个公有 IP,那么容器也拥有这个公有IP。同时容器内服务的端口也可以使用宿主机的端口,无需额外进行 NAT 转换。

host模式可以让容器共享宿主机网络栈,这样的好处是外部主机与容器直接通信,但是容器的网络缺少隔离性。

Host 网络模式的缺陷:

最明显的是 Docker Container 网络环境隔离性的弱化。即容器不再拥有隔离、独立的网络环境。

另外,使用 host 模式的 Docker Container 虽然可以让容器内部的服务和传统情况无差别、无改造的使用,但是由于网络隔离性的弱化,该容器会与宿主机共享竞争网络栈的使用;

另外,容器内部将不再拥有所有的端口资源,原因是部分端口资源已经被宿主机本身的服务占用,还有部分端口已经用以 bridge 网络模式容器的端口映射。

Container网络模式

一种特殊host 网络模式

Container 网络模式是 Docker 中一种较为特别的网络的模式。在容器创建时使用--network=container:vm1指定。(vm1指定的是运行的容器名)

处于这个模式下的 Docker 容器会共享一个网络环境,这样两个容器之间可以使用localhost高效快速通信。

缺陷：它并没有改善容器与宿主机以外世界通信的情况（和桥接模式一样，不能连接宿主机以外的其他设备）。

这个模式指定新创建的容器和已经存在的一个容器共享一个 Network Namespace，而不是和宿主机共享。新创建的容器不会创建自己的网卡，配置自己的 IP，而是和一个指定的容器共享 IP、端口范围等。同样，两个容器除了网络方面，其他的如文件系统、进程列表等还是隔离的。两个容器的进程可以通过 lo 网卡设备通信。

none模式

使用none模式，Docker容器拥有自己的Network Namespace，但是，并不为Docker容器进行任何网络配置。也就是说，这个Docker容器没有网卡、IP、路由等信息。需要我们自己为Docker容器添加网卡、配置IP等。

这种网络模式下容器只有lo回环网络，没有其他网卡。none模式可以在容器创建时通过--network=none来指定。这种类型的网络没有办法联网，封闭的网络能很好的保证容器的安全性。

overlay 网络模式

Overlay 网络，也称为覆盖网络。主要用于docker集群部署。

Overlay 网络的实现方式和方案有多种。Docker自身集成了一种，基于VXLAN隧道技术实现。

Overlay 网络主要用于实现跨主机容器之间的通信。

应用场景：需要管理成百上千个跨主机的容器集群的网络时

macvlan 网络模式

macvlan网络模式，最主要的特征就是他们的通信会直接基于mac地址进行转发。

这时宿主机其实充当一个二层交换机。Docker会维护着一个MAC地址表，当宿主机网络收到一个数据包后，直接根据mac地址找到对应的容器，再把数据交给对应的容器。

容器之间可以直接通过IP互通，通过宿主机上内建的虚拟网络设备（创建macvlan网络时自动创建），但与主机无法直接利用IP互通。

应用场景：由于每个外来的数据包的目的mac地址就是容器的mac地址，这时每个容器对于外面网络来说就相当于一个真实的物理网络设备。因此当需要让容器来的网络看起来是一个真实的物理机时，使用macvlan模式

Macvlan是一个新的尝试，是真正的网络虚拟化技术的转折点。Linux实现非常轻量级，因为与传统的Linux Bridge隔离相比，它们只是简单地与一个Linux以太网接口或子接口相关联，以实现网络之间的分离和与物理网络的连接。

Macvlan提供了许多独特的功能，并有充足的空间进一步创新与各种模式。这些方法的两个高级优点是绕过Linux网桥的正面性能以及移动部件少的简单性。删除传统上驻留在Docker主机NIC和容器接口之间的网桥留下了一个非常简单的设置，包括容器接口，直接连接到Docker主机接口。由于在这些情况下没有端口映射，因此可以轻松访问外部服务。

Macvlan Bridge模式每个容器都有唯一的MAC地址，用于跟踪Docker主机的MAC到端口映射。

Macvlan驱动程序网络连接到父Docker主机接口。示例是物理接口，例如eth0，用于802.1q VLAN标记的子接口eth0.10（.10代表VLAN 10）或甚至绑定的主机适配器，将两个以太网接口捆绑为单个逻辑接口。指定的网关由网络基础设施提供的主机外部。每个Macvlan Bridge模式的Docker网络彼此隔离，一次只能有一个网络连接到父节点。

每个主机适配器有一个理论限制，每个主机适配器可以连接一个Docker网络。同一子网内的任何容器都可以与没有网关的同一网络中的任何其他容器进行通信macvlan bridge。相同的docker network命令适用于vlan驱动程序。在Macvlan模式下，在两个网络/子网之间没有外部进程路由的情况下，单独网络上的容器无法互相访问。这也适用于同一码头网络内的多个子网。

网络实操

拉取镜像

```
docker pull nginx:1.19.3-alpine
```

备份镜像

```
docker save nginx:1.19.3-alpine -o nginx.1.19.3.alpine.tar
```

导入镜像

```
docker load -i nginx.1.19.3.alpine.tar
```

bridge网络

本章节学习docker的bridge网络，bridge网络表现形式就是docker0这个网络接口。

容器默认都是通过docker0这个接口进行通信。

也可以通过docker0去和本机的以太网接口连接，这样容器内部才能访问互联网。

查看docker0网络，在默认环境中，一个名为docker0的linux bridge自动被创建好了，其上有一个docker0 内部接口，IP地址为172.17.0.1/16

```
ip a
```

查看docker 网络

```
docker network ls
```

查看bridge网络详情。主要关注Containers节点信息。

```
docker network inspect bridge
```

docker0详解

运行镜像

```
docker run -itd --name nginx1 nginx:1.19.3-alpine
```

查看bridge网络详情。主要关注Containers节点信息。发现nginx1容器默认使用bridge网络

```
docker network inspect bridge
```

容器创建时IP地址的分配

查看docker100主机网络。发现多出一块网卡veth62aef5e@if8

```
ip a
```

Docker 创建一个容器的时候，会执行如下操作：

- 创建一对虚拟接口/网卡，也就是veth pair，分别放到本地主机和新容器中；
- 本地主机一端桥接到默认的 docker0 或指定网桥上，并具有一个唯一的名字，如 vetha596da4；
- 容器一端放到新容器中，并修改名字作为 eth0，这个网卡/接口只在容器的名字空间可见；
- 从网桥可用地址段中（也就是与该bridge对应的network）获取一个空闲地址分配给容器的 eth0，并配置默认路由到桥接网卡 vetha596da4。

完成这些之后，容器就可以使用 eth0 虚拟网卡来连接其他容器和其他网络。

如果不指定--network，创建的容器默认都会挂到 docker0 上，使用本地主机上 docker0 接口的 IP 作为所有容器的默认网关。

第一种方式：

```
docker exec -it nginx1 sh
```

```
ip a
```

第二种方式：

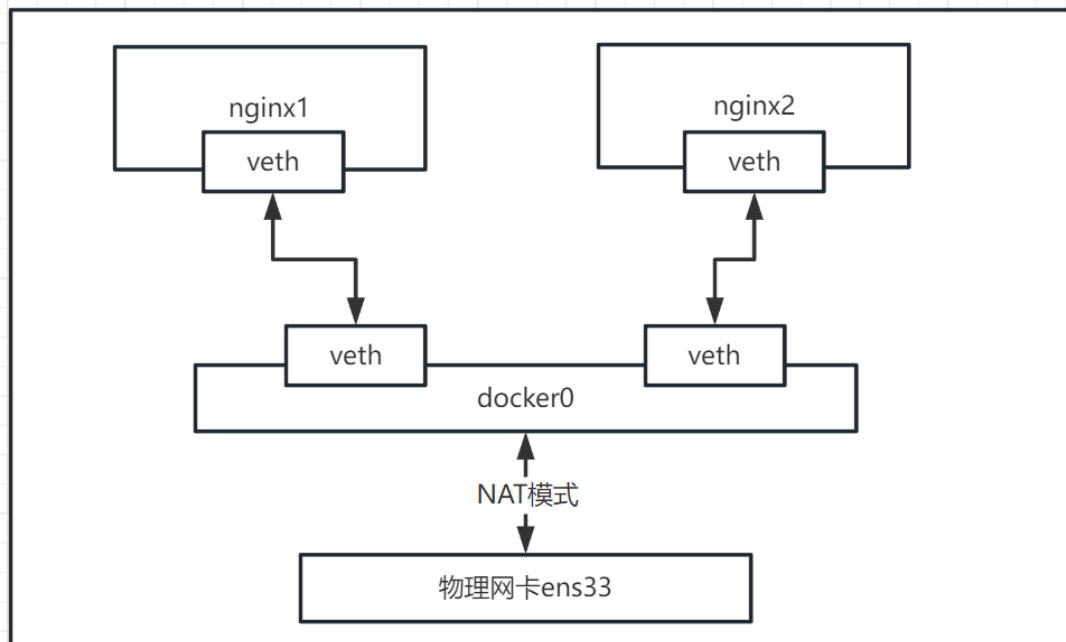
```
docker exec -it nginx1 ip a
```

安装brctl

```
yum install -y bridge-utils
```

运行命令

```
brctl show
```



CSDN @40岁资深老架构师尼恩

多容器之间通讯

```
docker run -itd --name nginx1 nginx:1.19.3-alpine
docker run -itd --name nginx2 nginx:1.19.3-alpine
docker network inspect bridge
docker exec -it nginx1 sh
ping 172.17.0.2
docker exec -it nginx2 sh
ping 172.17.0.2
ping www.baidu.com
ping nginx1
```

容器IP地址会发生变化

```
docker stop nginx1 nginx2
先启动nginx2，在启动nginx1
docker start nginx2
docker start nginx1
docker network inspect bridge
```

link容器

学习docker run命令的link参数

- `--link=[]`：添加链接到另一个容器；

使用link的场景：

在企业开发环境中，我们有一个mysql的服务的容器mysql_1，还有一个web应用程序web_1，肯定web_1这台容器肯定要连接mysql_1这个数据库。

前面网络命名空间的知识告诉我们，两个容器需要能通信，需要知道对方的具体的IP地址。

生产环境还比较好，IP地址很少变化，但是在我们内部测试环境，容器部署的IP地址是可能不断变化的，所以，开发人员不能在代码中写死数据库的IP地址。

这个时候，我们就可以利用容器之间link来解决这个问题。不推荐大家使用该参数

下面，我们来介绍如何通过容器名称来进行ping，而不是通过IP地址。

```
docker rm -f nginx2
docker run -itd --name nginx2 --link nginx1 nginx:1.19.3-alpine docker exec -it
nginx2 sh
ping 172.17.0.2
ping www.baidu.com
ping nginx1
```

- 上面link命令，是在nginx2容器启动时link到nginx1容器，因此，在nginx2容器里面可以ping通nginx1容器名，link的作用相当于添加了DNS解析。这里提醒下，在nginx1容器里去ping nginx2容器是不通的，因为link关系是单向的，不可逆。
- 实际工作中，docker官网已经不推荐我们使用link参数。
- docker用其他方式替换掉link参数

新建bridge网络

```
docker network create -d bridge lagou-bridge
```

上面命令参数-d 是指DRIVER的类型，后面的lagou-bridge是network的自定义名称，这个和docker0是类似的。下面开始介绍，如何把容器连接到lagou-bridge这个网络。

启动一个nginx的容器nginx3，并通过参数network connect来连接lagou-bridge网络。在启动容器nginx3之前，我们查看目前还没有容器连接到了lagou-bridge这个网络上。

```
brctl show
docker network ls
docker network inspect lagou-bridge
docker run -itd --name nginx3 --network lagou-bridge nginx:1.19.3-alpine brctl
show
docker network inspect lagou-bridge
```

把一个运行中容器连接到lagou-bridge网络

```
docker network connect lagou-bridge nginx2
docker network inspect lagou-bridge
docker exec -it nginx2 sh
ping nginx3
docker exec -it nginx3 sh
ping nginx2
```

none网络

环境准备，先stop和rm掉全部之前开启的容器。并且把前面创建的lagou-bridge网络也删除。当然，更简单的办法是使用快照方式。将docker-100主机恢复到docker初始化安装时。

```
docker rm -f $(docker ps -aq)
docker network rm lagou-bridge
docker network ls
```

启动一个nginx的容器nginx1，并且连接到none网络。然后执行docker network inspect none，看看容器信息

```
docker run -itd --name nginx1 --network none nginx:1.19.3-alpine
docker network inspect none
```

注意，容器使用none模式，是没有物理地址和IP地址。我们可以进入到nginx1容器里，执行ip a命令看看。只有一个lo接口，没有其他网络接口，没有IP。也就是说，使用none模式，这个容器是不能被其他容器访问。这种使用场景很少，只有项目安全性很高的功能才能使用到。例如：密码加密算法容器。

```
docker exec -it nginx1 sh
ip a
```

host网络

前面学习none网络模式特点就是，容器没有IP地址，不能和其他容器通信。下面来看看host网络是什么特点。我们使用前面命令，启动一个nginx的nginx2容器，连接到host网络。然后docker network inspect host, 看看容器信息。

```
docker run -itd --name nginx2 --network host nginx:1.19.3-alpine
docker network inspect host
```

这里来看，也不显示IP地址。那么是不是和none一样，肯定不是，不然也不会设计none和host网络进行区分。下面我们进入nginx2容器，执行ip a看看效果。我们在容器里执行ip a，发现打印内容和在linux本机外执行ip a是一样的。

```
docker exec -it nginx2 sh
ip a
```

这说明什么呢？

容器使用了host模式，说明容器和外层linux主机共享一套网络接口。VMware公司的虚拟机管理软件，其中网络设置，也有host这个模式，作用也是一样，虚拟机里面使用网络和你自己外层机器是一模一样的。这种容器和本机使用共享一套网络接口，缺点还是很明显的，例如我们知道web服务器一般端口是80，共享了一套网络接口，那么你这台机器上只能启动一个nginx端口为80的服务器了。否则，出现端口被占用的情况。

本篇很简单，就是简单了解下docker中none和host网络模式。学习重点还是如何使用bridge网络。

网络命令汇总

```
docker network --help
网络常用命令汇总
connect Connect a container to a network
create Create a network
disconnect Disconnect a container from a network
inspect Display detailed information on one or more networks ls List networks
prune Remove all unused networks
rm Remove one or more networks
```

查看网络

```
查看网络 - docker network ls
# 作用：
查看已经建立的网络对象
# 命令格式：
docker network ls [OPTIONS]
# 命令参数(OPTIONS)：
-f, --filter filter 过滤条件(如 'driver=bridge')
--format string 格式化打印结果
--no-trunc 不缩略显示
-q, --quiet 只显示网络对象的ID
# 注意：
默认情况下，docker安装完成后，会自动创建bridge、host、none三种网络驱动
# 命令演示
docker network ls
docker network ls --no-trunc
docker network ls -f 'driver=host'
```

创建网络

```
创建网络 - docker network create
# 作用：
创建新的网络对象
# 命令格式：
docker network create [OPTIONS] NETWORK
# 命令参数(OPTIONS)：
-d, --driver string 指定网络的驱动(默认 "bridge")
--subnet strings 指定子网网段(如192.168.0.0/16、172.88.0.0/24)
--ip-range strings 执行容器的IP范围，格式同subnet参数
--gateway strings 子网的IPv4 or IPv6网关，如(192.168.0.1)
# 注意：
host和none模式网络只能存在一个
docker自带的overlay 网络创建依赖于docker swarm(集群负载均衡)服务
192.168.0.0/16 等于 192.168.0.0~192.168.255.255
172.88.0.0/24 等于 172.88.0.0~172.88.0.255
# 命令演示
```

```
docker network ls
docker network create -d bridge my-bridge
docker network ls
```

网络删除

网络删除 - `docker network rm`

作用：
删除一个或多个网络

命令格式：
`docker network rm NETWORK [NETWORK...]`

命令参数(OPTIONS)：
无

查看网络详细信息

查看网络详细信息 `docker network inspect`

作用：
查看一个或多个网络的详细信息

命令格式：
`docker network inspect [OPTIONS] NETWORK [NETWORK...]`
或者 `docker inspect [OPTIONS] NETWORK [NETWORK...]`

命令参数(OPTIONS)：
`-f`, `--format string` 根据format输出结果

使用网络

使用网络 - `docker run --network`

作用：
为启动的容器指定网络模式

命令格式：
`docker run/create --network NETWORK`

命令参数(OPTIONS)：
无

注意：
默认情况下，docker创建或启动容器时，会默认使用名为bridge的网络

网络连接与断开

网络连接与断开 - `docker network connect/disconnect`

作用：
将指定容器与指定网络进行连接或者断开连接

命令格式：
`docker network connect [OPTIONS] NETWORK CONTAINER`
`docker network disconnect [OPTIONS] NETWORK CONTAINER`

命令参数(OPTIONS)：
`-f`, `--force` 强制断开连接(用于disconnect)

Docker-Compose 简介

Docker-Compose 项目是Docker官方的开源项目，负责实现对Docker容器集群的快速编排。

Docker-Compose 项目由 Python 编写，调用 Docker 服务提供的API来对容器进行管理。因此，只要所操作的平台支持 Docker API，就可以在其上利用Compose 来进行编排管理。

Docker-Compose 用来实现Docker容器快速编排

通过 Docker-Compose，不需要使用shell脚本来启动容器，而使用 YAML 文件来配置应用程序需要的所有服务，然后使用一个命令，根据 YAML 的文件配置创建并启动所有服务。

Docker-compose模板文件简介

Compose允许用户通过一个docker-compose.yml模板文件（YAML 格式）来定义一组相关联的应用容器为一个项目（project）。

Compose模板文件是一个定义服务、网络和卷的YAML文件。

Compose模板文件默认路径是当前目录下的docker-compose.yml，可以使用.yml或.yaml作为文件扩展名。

Docker-Compose标准模板文件应该包含version、services、networks 三大部分，最关键的是services和networks两个部分。

eg:

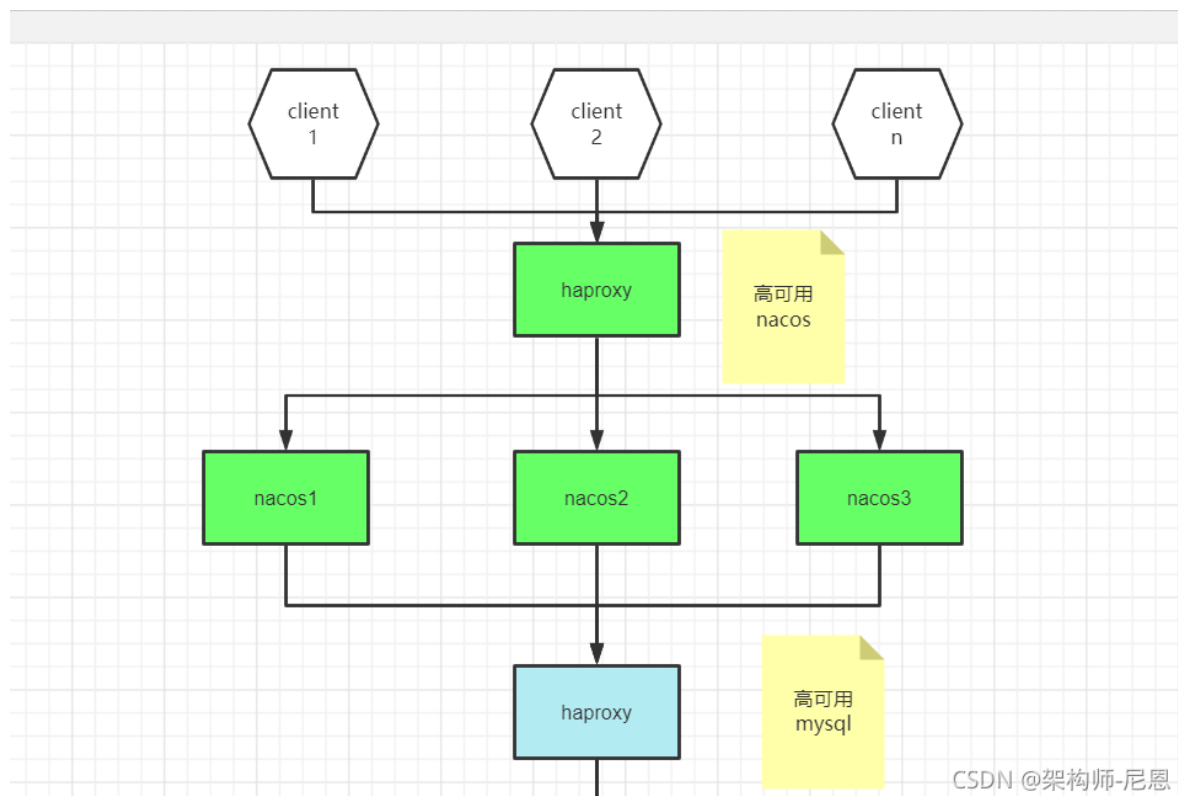
```
version: '3.5'
services:
  nacos1:
    restart: always
    image: nacos/nacos-server:${NACOS_VERSION}
    container_name: nacos1
    privileged: true
    ports:
      - "8001:8001"
      - "8011:9555"
    deploy:
      resources:
        limits:
          cpus: '0.50'
          memory: 1024M
    env_file:
      - ./nacos.env
    environment:
      NACOS_SERVER_IP: ${NACOS_SERVER_IP_1}
      NACOS_APPLICATION_PORT: 8001
      NACOS_SERVERS: ${NACOS_SERVERS}
    volumes:
      - ./logs_01:/home/nacos/logs/
      - ./data_01:/home/nacos/data/
      - ./config:/home/nacos/config/
    networks:
      - ha-network-overlay
  nacos2:
    restart: always
    image: nacos/nacos-server:${NACOS_VERSION}
    container_name: nacos2
    privileged: true
    ports:
      - "8002:8002"
```

```

    - "8012:9555"
  deploy:
    resources:
      limits:
        cpus: '0.50'
        memory: 1024M
  env_file:
    - ./nacos.env
  environment:
    NACOS_SERVER_IP: ${NACOS_SERVER_IP_2}
    NACOS_APPLICATION_PORT: 8002
    NACOS_SERVERS: ${NACOS_SERVERS}
  volumes:
    - ./logs_02:/home/nacos/logs/
    - ./data_02:/home/nacos/data/
    - ./config:/home/nacos/config/
  networks:
    - ha-network-overlay
nacos3:
  restart: always
  image: nacos/nacos-server:${NACOS_VERSION}
  container_name: nacos3
  privileged: true
  ports:
    - "8003:8003"
    - "8013:9555"
  deploy:
    resources:
      limits:
        cpus: '0.50'
        memory: 1024M
  env_file:
    - ./nacos.env
  environment:
    NACOS_SERVER_IP: ${NACOS_SERVER_IP_3}
    NACOS_APPLICATION_PORT: 8003
    NACOS_SERVERS: ${NACOS_SERVERS}
  volumes:
    - ./logs_03:/home/nacos/logs/
    - ./data_03:/home/nacos/data/
    - ./config:/home/nacos/config/
  networks:
    - ha-network-overlay
networks:
  ha-network-overlay:
    external: true

```

Docker-Compose 的编排处出来的部署架构



CSDN @架构师-尼恩

docker-compose 快速编排

Docker-Compose 项目是Docker官方的开源项目，负责实现对Docker容器集群的快速编排。

Docker-Compose 项目由 Python 编写，调用 Docker 服务提供的API来对容器进行管理。因此，只要所操作的平台支持 Docker API，就可以在其上利用Compose 来进行编排管理。

首先检查 版本

```
[root@k8s-master ~]# /usr/local/bin/docker-compose -version
docker-compose version 1.25.1, build a82fef07
```

如果安装好了，就ok了，如果没有安装，则安装docker

1.从github上下载docker-compose二进制文件安装

下载最新版的docker-compose文件

```
# curl -L https://github.com/docker/compose/releases/download/1.25.1/docker-
compose-`uname -s`-`uname -m` -o /usr/local/bin/docker-compose
```

注明：离线安装包已经提供。上传后，复制到/usr/local/bin/即可

```
cp /root/docker-compose /usr/local/bin/
```

添加可执行权限

```
# chmod +x /usr/local/bin/docker-compose
```

测试安装结果

```
[root@localhost ~]# docker-compose --version
docker-compose version 1.25.1, build a82fef07

cp /root/docker-compose /usr/local/bin/
chmod +x /usr/local/bin/docker-compose
docker-compose --version
```

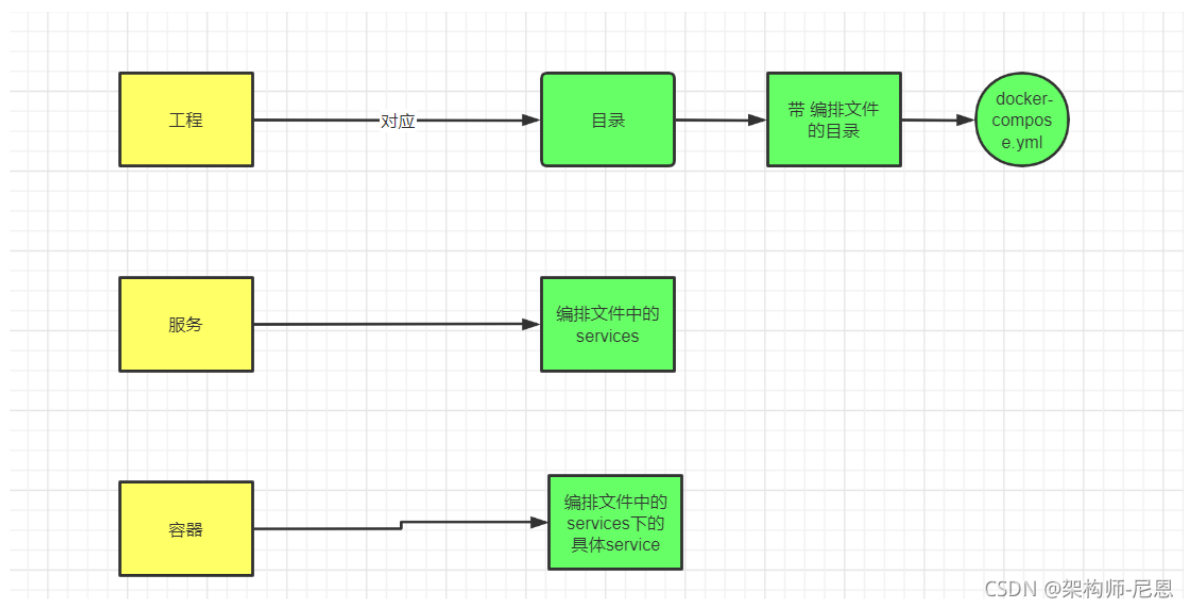
Docker-Compose 的编排结构

Docker-Compose 将所管理的容器分为三层

- **工程** (project) : 一个工程包含多个服务
- **服务** (service) : 一个服务当中可包括多个容器实例
- **容器** (container)

Docker-Compose 运行目录下的所有文件 (**docker-compose.yml**、**extends文件** 或 **环境变量文件** 等) 组成一个工程, 若无特殊指定 **工程名即为当前目录名**。

Docker Compose 的核心就是其配置文件, 采用 YAML 格式, 默认为 `docker-compose.yml` 。



CSDN @架构师-尼恩

一个工程当中可包含多个服务，每个服务中定义了容器运行的镜像、参数、依赖。

一个服务当中可包括多个容器实例，但是：Docker-Compose 并没有解决负载均衡的问题，因此需要借助其它工具实现服务发现及负载均衡，比如 Consul 技术。

Docker-Compose 的工程配置文件默认为 docker-compose.yml，可通过环境变量 COMPOSE_FILE 或 -f 参数自定义配置文件，其定义了多个有依赖关系的服务及每个服务运行的容器。

Compose 允许用户通过一个单独的 docker-compose.yml 模板文件（YAML格式）来定义一组相关的应用容器为一个项目 (project)。

YAML模板文件语法

默认的模板文件是docker-compose.yml,其中定义的每个服务都必须通过image指令指定镜像，也可以通过build指令（需要Dockerfile）来自动构建。

其他大部分都跟docker run 中类似。如果使用build指令，在Dockerfile中设置的选项（例如：CMD,EXPOSE,VOLUME,ENV等）将自动被获取，无需在docker-compose.yml中再次被设置。

docker-compose.yml 语法说明

1、image

指定为镜像名称或镜像ID。

如果镜像不存在，Compose将尝试从互联网拉取这个镜像，例如： image: ubuntu image: orchardup/postgresql image: a4bc65fd

指定服务的镜像名，若本地不存在，则 Compose 会去仓库拉取这个镜像：

```
services:
  web:
    image: nginx
```

2、build

指定Dockerfile所在文件夹的路径。Compose将会利用他自动构建这个镜像，然后使用这个镜像。

build: ./dir

3、command

覆盖容器启动后默认执行的命令。 command: bundle exec thin -p 3000

4、links

链接到其他服务容器，使用服务名称(同时作为别名)或服务别名（SERVICE:ALIAS）都可以

```
links:
  - db
  - db:database
  - redis
```

注意：使用别名会自动在服务器中的/etc/hosts 里创建，如：172.17.2.186 db，相应环境变量也会被创建。

5、external_links

链接到docker-compose.yml外部的容器，甚至并非是Compose管理的容器。参数格式和links类似。external_links:

```
- redis_1
- project_db_1:mysql
- project_db_2:sqlserver
```

6、ports

暴露端口信息。格式

宿主机器端口：容器端口（HOST:CONTAINER）

或者仅仅指定容器的端口（宿主机器将会随机分配端口）都可以。

```
ports:
  - "3306"
  - "8080:80"
  - "127.0.0.1:8090:8001"
```

注意：当使用 HOST:CONTAINER 格式来映射端口时，如果你使用的容器端口小于 60 你可能会得到错误结果，因为 YAML 将会解析 xx:yy 这种数字格式为 60 进制。所以建议采用字符串格式。

7、expose

暴露端口，与posts不同的是expose只可以暴露端口而不能映射到主机，只供外部服务连接使用；仅可以指定内部端口为参数。

```
expose:
  - "3000"
  - "8000"
```

8、volumes

设置卷挂载的路径。

可以设置宿主机路径:容器路径（host:container）或加上访问模式（host:container:ro）ro就是readonly的意思，只读模式。

```
volumes:
  - /var/lib/mysql:/var/lib/mysql
  - /configs/mysql:/etc/configs/:ro
```

9、volumes_from

挂载另一个服务或容器的所有数据卷。

```
volumes_from:  
- service_name  
- container_name
```

10、environment

设置环境变量。可以属于数组或字典两种格式。

如果只给定变量的名称则会自动加载它在Compose主机上的值，可以用来防止**泄露不必要的数据**。

```
environment:  
- RACK_ENV=development  
- SESSION_SECRET
```

11、env_file

从文件中获取环境变量，可以为单独的文件路径或列表。如果通过docker-compose -f FILE指定了模板文件，则env_file中路径会基于模板文件路径。如果有变量名称与environment指令冲突，则以后者为准。

```
env_file: .env  
env_file:  
- ./common.env  
- ./apps/web.env  
- /opt/secrets.env
```

环境变量文件中每一行都必须有注释，支持#开头的注释行。

```
# common.env: Set Rails/Rack environment  
RACK_ENV=development
```

12、extends

基于已有的服务进行服务扩展。例如我们已经有了一个webapp服务，模板文件为common.yml。

```
# common.yml  
webapp:  
build: ./webapp  
environment:  
- DEBUG=false  
- SEND_EMAILS=false
```

编写一个新的 development.yml 文件，使用 common.yml 中的 webapp 服务进行扩展。
development.yml

```
web:
extends:
file: common.yml
service:
  webapp:
    ports:
      - "8080:80"
    links:
      - db
    envelopment:
      - DEBUG=true
  db:
    image: mysql:5.7
```

后者会自动继承common.yml中的webapp服务及相关的环境变量。

13、net

设置网络模式。使用和docker client 的 --net 参数一样的值。

```
# 容器默认连接的网络，是所有Docker安装时都默认安装的docker0网络。
net: "bridge"
# 容器定制的网络栈。
net: "none"
# 使用另一个容器的网络配置
net: "container:[name or id]"
# 在宿主网络栈上添加一个容器，容器中的网络配置会与宿主的一样
net: "host"
```

Docker会为每个节点自动创建三个网络：网络名称 作用 bridge 容器默认连接的网络，是所有Docker安装时都默认安装的docker0网络 none 容器定制的网络栈 host 在宿主网络栈上添加一个容器，容器中的网络配置会与宿主的一样 附录：操作名称 命令 创建网络 docker network create -d bridge mynet 查看网络列表 docker network ls

14、pid

和宿主系统共享进程命名空间，打开该选项的容器可以相互通过进程id来访问和操作。

```
pid: "host"
```

15、dns

```
配置DNS服务器。可以是一个值，也可以是一个列表。
dns: 8.8.8.8
dns:
  - 8.8.8.8
  - 9.9.9.9
```

16、cap_add, cap_drop

添加或放弃容器的Linux能力（Capability）。

```
cap_add:
- ALL
cap_drop:
- NET_ADMIN
- SYS_ADMIN
```

17、dns_search

配置DNS搜索域。可以是一个值也可以是一个列表。

```
dns_search: example.com
dns_search:
- domain1.example.com
\ - domain2.example.com
working_dir, entrypoint, user, hostname, domainname, mem_limit, privileged,
restart, stdin_open, tty, cpu_shares
```

这些都是和 docker run 支持的选项类似。

```
cpu_shares: 73
working_dir: /code
entrypoint: /code/entrypoint.sh
user: postgresql
hostname: foo
domainname: foo.com
mem_limit: 1000000000
privileged: true
restart: always
stdin_open: true
tty: true
```

18、healthcheck

健康检查，这个非常有必要，等服务准备好以后再上线，避免更新过程中出现短暂的无法访问。

```
healthcheck:
test: ["CMD", "curl", "-f", "http://localhost/alive"]
interval: 5s
timeout: 3s
```

其实大多数情况下健康检查的规则都会写在 Dockerfile 中：

```
FROM nginx
RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*
HEALTHCHECK --interval=5s --timeout=3s CMD curl -f http://localhost/alive ||
exit 1
```

19、depends_on

依赖的服务，优先启动，例：

```
depends_on:
- redis
```

20、deploy

部署相关的配置都在这个节点下，例：

```
deploy:
  mode: replicated
  replicas: 2
  restart_policy:
    condition: on-failure
    max_attempts: 3
  update_config:
    delay: 5s
    order: start-first # 默认为 stop-first，推荐设置先启动新服务再终止旧的
  resources:
    limits:
      cpus: "0.50"
      memory: 1g
deploy:
  mode: global # 不推荐全局模式（仅个人意见）。
  placement:
    constraints: [node.role == manager]
```

若非特殊服务，以上各节点的配置能够满足大部分部署场景了。

docker-compose.yml实例

```
version: '3.5'
services:
  nacos1:
    restart: always
    image: nacos/nacos-server:${NACOS_VERSION}
    container_name: nacos1
    privileged: true
    ports:
      - "8001:8001"
      - "8011:9555"
    deploy:
      resources:
        limits:
          cpus: '0.50'
          memory: 1024M
    env_file:
      - ./nacos.env
    environment:
      NACOS_SERVER_IP: ${NACOS_SERVER_IP_1}
      NACOS_APPLICATION_PORT: 8001
      NACOS_SERVERS: ${NACOS_SERVERS}
    volumes:
      - ./logs_01/./home/nacos/logs/
```

```

- ./data_01:/home/nacos/data/
- ./config:/home/nacos/config/
networks:
- ha-network-overlay
nacos2:
  restart: always
  image: nacos/nacos-server:${NACOS_VERSION}
  container_name: nacos2
  privileged: true
  ports:
    - "8002:8002"
    - "8012:9555"
  deploy:
    resources:
      limits:
        cpus: '0.50'
        memory: 1024M
  env_file:
    - ./nacos.env
  environment:
    NACOS_SERVER_IP: ${NACOS_SERVER_IP_2}
    NACOS_APPLICATION_PORT: 8002
    NACOS_SERVERS: ${NACOS_SERVERS}
  volumes:
    - ./logs_02:/home/nacos/logs/
    - ./data_02:/home/nacos/data/
    - ./config:/home/nacos/config/
  networks:
    - ha-network-overlay
nacos3:
  restart: always
  image: nacos/nacos-server:${NACOS_VERSION}
  container_name: nacos3
  privileged: true
  ports:
    - "8003:8003"
    - "8013:9555"
  deploy:
    resources:
      limits:
        cpus: '0.50'
        memory: 1024M
  env_file:
    - ./nacos.env
  environment:
    NACOS_SERVER_IP: ${NACOS_SERVER_IP_3}
    NACOS_APPLICATION_PORT: 8003
    NACOS_SERVERS: ${NACOS_SERVERS}
  volumes:
    - ./logs_03:/home/nacos/logs/
    - ./data_03:/home/nacos/data/
    - ./config:/home/nacos/config/
  networks:
    - ha-network-overlay
networks:
  ha-network-overlay:
    external: true

```

YAML 文件格式 及 编写注意事项

使用compose对Docker容器进行编排管理时，需要编写docker-compose.yml文件，初次编写时，容易遇到一些比较低级的问题，导致执行docker-compose up时先解析yaml文件的错误。

比较常见的是yaml对缩进的严格要求。

yaml文件还行后的缩进，不允许使用tab键字符，只能使用空格，而空格的数量也有要求，经过实际测试，发现每一行增加一个空格用于缩进是正常的。

aml 是一种标记语言，它可以很直观的展示数据序列化格式，可读性高。类似于XML数据描述语言，语法比 XMAL简单的很多。

YAML数据结构通过缩进来表示，连续的项目通过减号来表示，键值对用冒号分隔，数组用中括号 [] 括起来，hash用花括号 {} 括起来。

使用 YAML时需要注意下面事项：

- 使用缩进表示层级关系，不支持制表符tab键缩进，只能使用空格键缩进；
- 缩进长度没有限制，只要元素对齐就表示这些元素属于一个层级
- 通常开头缩进2个空格；
- 字符后缩进1个空格，如冒号 : 空格 逗号 , 空格 横杠 - 空格 文本之间的空格 > ；
- 用#号注释；
- 如果包含特殊字符用单引号 ' ' 引起来；
- 布尔值必须用引号 " " 括起来；
- 区分大小写
- 字符串可以不用引号标注

Docker-compose常用命令

常用命令清单

docker-compose 命令 --help	获得一个命令的帮助
docker-compose up -d nginx	构建启动nginx容器
docker-compose exec nginx bash	登录到nginx容器中
docker-compose down	此命令将会停止 up 命令所启动的容器，并移除网络
docker-compose ps	列出项目中目前的所有容器
docker-compose restart nginx	重新启动nginx容器
docker-compose build nginx	构建镜像
docker-compose build --no-cache nginx	不带缓存的构建
docker-compose top	查看各个服务容器内运行的进程
docker-compose logs -f nginx	查看nginx的实时日志
docker-compose images	列出 Compose 文件包含的镜像
docker-compose config	验证文件配置，当配置正确时，不输出任何内容，当文件配置错误，输出错误信息。
docker-compose events --json nginx	以json的形式输出nginx的docker日志
docker-compose pause nginx	暂停nginx容器
docker-compose unpause nginx	恢复ningx容器
docker-compose rm nginx	删除容器（删除前必须关闭容器，执行stop）

```
docker-compose stop nginx      停止nginx容器
docker-compose start nginx     启动nginx容器
docker-compose restart nginx   重启项目中的nginx容器
docker-compose run --no-deps --rm php-fpm php -v  在php-fpm中不启动关联容器，并容器
执行php -v 执行完成后删除容器
```

1、Docker-Compose

```
docker-compose [-f <arg>...] [options] [COMMAND] [ARGS...]
```

命令选项如下：

- -f, -file FILE指定Compose模板文件，默认为docker-compose.yml，可以多次指定。
- -p, -project-name NAME指定项目名称，默认将使用所在目录名称作为项目名。
- -x-network-driver 使用Docker的可拔插网络后端特性（需要Docker 1.9+版本） -x-network-driver DRIVER指定网络后端的驱动，默认为bridge（需要Docker 1.9+版本）
- -verbose输出更多调试信息
- -v, -version打印版本并退出

2、docker-compose up

这个命令一定要记住，每次启动都要用到，只要学会使用的人记住这个就好了

```
docker-compose up [options] [--scale SERVICE=NUM...] [SERVICE...]
```

选项包括：

- -d 在后台运行服务容器
 - -no-color 不使用颜色来区分不同的服务的控制输出
 - -no-deps 不启动服务所链接的容器
 - -force-recreate 强制重新创建容器，不能与-no-recreate同时使用 -no-recreate 如果容器已经存在，则不重新创建，不能与-force-recreate同时使用
 - -no-build 不自动构建缺失的服务镜像
 - -build 在启动容器前构建服务镜像
 - -abort-on-container-exit 停止所有容器，如果任何一个容器被停止，不能与-d同时使用
 - -t, -timeout TIMEOUT 停止容器时候的超时（默认为10秒）
 - -remove-orphans 删除服务中没有在compose文件中定义的容器
 - -scale SERVICE=NUM 设置服务运行容器的个数，将覆盖在compose中通过scale指定的参数
- docker-compose up 启动所有服务
- docker-compose up -d 在后台所有启动服务
- -f 指定使用的Compose模板文件，默认为docker-compose.yml，可以多次指定。
- docker-compose -f docker-compose.yml up -d

3、docker-compose ps

```
docker-compose ps [options] [SERVICE...]
```

docker-compose ps 列出项目中目前的所有容器

4、docker-compose stop

```
docker-compose stop [options] [SERVICE...]
```

选项包括：

- -t, -timeout TIMEOUT 停止容器时候的超时（默认为10秒）
docker-compose stop 停止正在运行的容器，可以通过docker-compose start 再次启动

5、docker-compose -h

docker-compose -h 查看帮助

6、docker-compose down

```
docker-compose down [options]
```

停止和删除容器、网络、卷、镜像。

选项包括：

- -rmi type, 删除镜像，类型必须是：all，删除compose文件中定义的所有镜像；local，删除镜像名为空的镜像
- -v, -volumes, 删除已经在compose文件中定义的和匿名的附在容器上的数据卷
- -remove-orphans, 删除服务中没有在compose中定义的容器
docker-compose down 停用移除所有容器以及网络相关

7、docker-compose logs

```
docker-compose logs [options] [SERVICE...]
```

查看服务容器的输出。

默认情况下，docker-compose将对不同的服务输出使用不同的颜色来区分。可以通过-no-color来关闭颜色。

8、docker-compose build

```
docker-compose build [options] [--build-arg key=val...] [SERVICE...]
```

构建（重新构建）项目中的服务容器。

选项包括：

- -compress 通过gzip压缩构建上下环境
- -force-rm 删除构建过程中的临时容器
- -no-cache 构建镜像过程中不使用缓存
- -pull 始终尝试通过拉取操作来获取更新版本的镜像
- -m, -memory MEM为构建的容器设置内存大小
- -build-arg key=val为服务设置build-time变量
服务容器一旦构建后，将会带上一个标记名。可以随时在项目目录下运行docker-compose build 来重新构建服务

9、docker-compose pull

```
docker-compose pull [options] [SERVICE...]
```

拉取服务依赖的镜像。

选项包括：

- -ignore-pull-failures, 忽略拉取镜像过程中的错误
- -parallel, 多个镜像同时拉取

- -quiet, 拉取镜像过程中不打印进度信息

10、docker-compose restart

```
docker-compose restart [options] [SERVICE...]
```

重启项目中的服务。

选项包括：

- -t, -timeout TIMEOUT, 指定重启前停止容器的超时（默认为10秒）

11、docker-compose rm

```
docker-compose rm [options] [SERVICE...]
```

删除所有（停止状态的）服务容器。

选项包括：

- -f, -force, 强制直接删除，包括非停止状态的容器
- -v, 删除容器所挂载的数据卷

docker-compose rm 删除所有（停止状态的）服务容器。推荐先执行docker-compose stop命令来停止容器。

12、docker-compose start

```
docker-compose start [SERVICE...]
```

启动已经存在的服务容器。

13、docker-compose run

```
docker-compose run [options] [-v VOLUME...] [-p PORT...] [-e KEY=VAL...] SERVICE  
[COMMAND] [ARGS...]
```

在指定服务上执行一个命令。

docker-compose run ubuntu ping www.baidu.com 在指定容器上执行一个ping命令。

14、docker-compose scale

docker-compose scale web=3 db=2 设置指定服务运行的容器个数。通过service=num的参数来设置数量

15、docker-compose pause

```
docker-compose pause [SERVICE...]
```

暂停一个服务容器

16、docker-compose kill

```
docker-compose kill [options] [SERVICE...]
```

通过发送SIGKILL信号来强制停止服务容器。支持通过-s参数来指定发送的信号，例如通过如下指令发送SIGINT信号： docker-compose kill -s SIGINT

17、docker-compose config

```
docker-compose config [options]
```

验证并查看compose文件配置。

选项包括：

- `-resolve-image-digests` 将镜像标签标记为摘要
- `-q, -quiet` 只验证配置，不输出。当配置正确时，不输出任何内容，当文件配置错误，输出错误信息
- `-services` 打印服务名，一行一个
- `-volumes` 打印数据卷名，一行一个

18、docker-compose create

```
docker-compose create [options] [SERVICE...]
```

为服务创建容器。

选项包括：

- `-force-recreate`：重新创建容器，即使配置和镜像没有改变，不兼容`-no-recreate`参数
- `-no-recreate`：如果容器已经存在，不需要重新创建，不兼容`-force-recreate`参数
- `-no-build`：不创建镜像，即使缺失
- `-build`：创建容器前，生成镜像

19、docker-compose exec

```
docker-compose exec [options] SERVICE COMMAND [ARGS...]
```

选项包括：

- `-d` 分离模式，后台运行命令。
- `-privileged` 获取特权。
- `-user USER` 指定运行的用户。
- `-T` 禁用分配TTY，默认docker-compose exec分配TTY。
- `-index=index`，当一个服务拥有多个容器时，可通过该参数登录到该服务下的任何服务，例如：
`docker-compose exec -index=1 web /bin/bash`，web服务中包含多个容器

20、docker-compose port

```
docker-compose port [options] SERVICE PRIVATE_PORT
```

显示某个容器端口所映射的公共端口。

选项包括：

- `-protocol=proto`，指定端口协议，TCP（默认值）或者UDP
- `-index=index`，如果同意服务存在多个容器，指定命令对象容器的序号（默认为1）

21、docker-compose push

```
docker-compose push [options] [SERVICE...]
```

推送服务依的镜像。

选项包括：

- `-ignore-push-failures` 忽略推送镜像过程中的错误

22、docker-compose unpause

```
docker-compose unpause [SERVICE...]
```

恢复处于暂停状态中的服务。

23、docker-compose version

```
docker-compose version
```

打印版本信息。

docker-compose logs 查看日志

`docker-compose logs` 查看实时日志（日志最后的N行、某刻后日志） 实践笔记

参数说明

view output from containers.

Usage: `logs [options] [SERVICE...]`

Options:

<code>--no-color</code>	Produce monochrome output.
<code>-f, --follow</code>	Follow log output. 实时输出日志，最后一行为当前时间戳的日志
<code>-t, --timestamps</code>	Show timestamps. 显示时间戳
<code>--tail="all"</code>	Number of lines to show from the end of the logs for

each container.

显示最后多少行日志，默认是all（如：`-tail=10`：查看最后的

10 行日志。）

全屏滚到底部结束

`docker-compose logs`

```
rabbitmq | 2021-01-08 06:33:09.091 [error] <0.1007.2> channel error on
connection <0.25180.0>
rabbitmq | operation basic.ack caused a channel exception precondition_failed:
unknown delivery tag
```

全屏滚到底部并继续持续输出日志

docker-compose logs -f

```
rabbitmq | 2021-01-08 06:33:09.091 [error] <0.1007.2> channel error on  
connection <0.25180.0>  
rabbitmq | operation basic.ack caused a channel exception precondition_failed:  
unknown delivery tag
```

相当于

tail -f

全屏滚到底部并继续持续输出日志并显示时间戳

docker-compose logs -f -t

```
rabbitmq | 2021-01-08T06:33:09.092091217Z 2021-01-08 06:33:09.091 [error]  
<0.1007.2> Channel  
rabbitmq | 2021-01-08T06:33:09.092109187Z operation basic.ack caused a channel  
exception
```

全屏滚到底部显示最后N行并继续持续输出日志并显示时间戳

docker-compose logs -f -t --tail=10

```
rabbitmq | 2021-01-08T06:33:09.092091217Z 2021-01-08 06:33:09.091 [error]  
<0.1007.2> Channel  
rabbitmq | 2021-01-08T06:33:09.092109187Z operation basic.ack caused a channel  
exception
```

说在后面

通过此PDF，尼恩希望能帮助大家，早日实现 Docker 技术和底层原理的技术自由。

此文PDF 没有完，后面还有N多内容需要补充。可以关注 尼恩的《技术自由圈》公众号。

本书《 Docker 学习圣经 》PDF的 V1版本，后面会持续迭代和升级。供后面的小伙伴参考，提升大家的 3高 架构、设计、开发水平。

注：本文以 PDF 持续更新，最新尼恩 架构笔记、面试题 的PDF文件，请从这里获取：[码云](#)

参考资料

<https://docs.docker.com/storage/storagedriver/aufs-driver/#how-the-aufs-storage-driver-works>

<https://github.com/opencontainers/runc>

<http://www.sel.zju.edu.cn/?p=840>

<https://draveness.me/docker/>

<https://blog.csdn.net/wangqingjiewa/article/details/85000393>

<https://zhuanlan.zhihu.com/p/47683490>

硬核推荐：尼恩Java硬核架构班

又名疯狂创客圈社群 VIP

详情：<https://www.cnblogs.com/crazymakercircle/p/9904544.html>



尼恩java
硬核架构班

已经发布

- 《高能性能RPC的基础实操之：从0到1开始IM撸一个IM》
- 《分布式高能性能RPC的基础实操之：千万级用户分布式IM实操-含简历指导》
- 《亿级用户超高并发秒杀实操-含简历指导》
亮点：助力小伙伴搞定70W年薪，N个涨薪50%，2023春招面试涨薪神器
- 《横扫全网，工业级elasticsearch底层原理与高并发、高可用架构实操》
亮点：40岁老架构师细致解读，处处透着分布式、高性能中间件的原理和精髓
- 《第1部曲：超级底层：葵花宝典（高性能秘籍）__架构师视角解读OS操作系统》
亮点：大制作解读OS操作系统，并揭秘mmap、pagecache、zerocopy等底层的底层原理
2023春招面试涨薪大神器
- 《Rocketmq视频第2部曲：横扫全网工业级 rocketmq 高可用（HA）底层原理和实操》
亮点：起底式、绞杀式解读 rocketmq如何保障消息的可靠性？
- 《Rocketmq视频第3部曲：超级内功篇、横扫全网 rocketmq 源码学习以及3高架构模式解读》
亮点：大制作解读 Rocketmq源码以及3高架构模式，助力大家内力猛增
- 《Rocketmq视频第4部曲：10Wqps消息推送中台架构、设计、编码、测试实操》
亮点：Netty实操、分库分表实操、Rocketmq工业级使用实操
- 《架构师内功篇：横扫全网 netty 高性能、高并发架构 底层原理、源码学习》
- 《架构师实操篇：redis cluster 工业级高可用实操》
- 《架构师实操篇：100W级别QPS日志平台实操》
- 《彻底穿透：skywalking 源码(代表链路跟踪)+Java agent+bytebuddy 探针》

规划中

《架构师实操篇：基于netty 手写 rpc 框架- 参考 dubbo、seata rpc框架》

《架构师实操篇：go语言学习，以及基于 go 手写 rpc 框架》

《架构师实操篇：千万级任务调度平台 架构与实操- 基于尼恩17年的亿级搜索项目》

《架构师实操篇：工业级 亿级文档搜索 平台 架构与实操- 基于尼恩17年的亿级搜索项目》

特色

会员制

提供技术方向指导，
职业生涯指导，少躺坑，少弯路

简历指导

这个很重要，
对于挪窝涨薪来说

实操性

以上项目，都是老架构师
在生产上实操过的项目

非水货

40岁老架构师，不是水货架构师
《Java高并发三部曲》为证

架构班（社群 VIP）的起源：

最初的视频，主要是给读者加餐。很多的读者，需要一些高质量的实操、理论视频，所以，我就围绕书，和底层，做了几个实操、理论视频，然后效果还不错，后面就做成迭代模式了。

架构班（社群 VIP）的功能：

提供高质量实操项目整刀真枪的架构指导、快速提升大家的：

- 开发水平
- 设计水平
- 架构水平

弥补业务中 CRUD 开发短板，帮助大家尽早脱离具备 3 高能力，掌握：

- 高性能
- 高并发
- 高可用

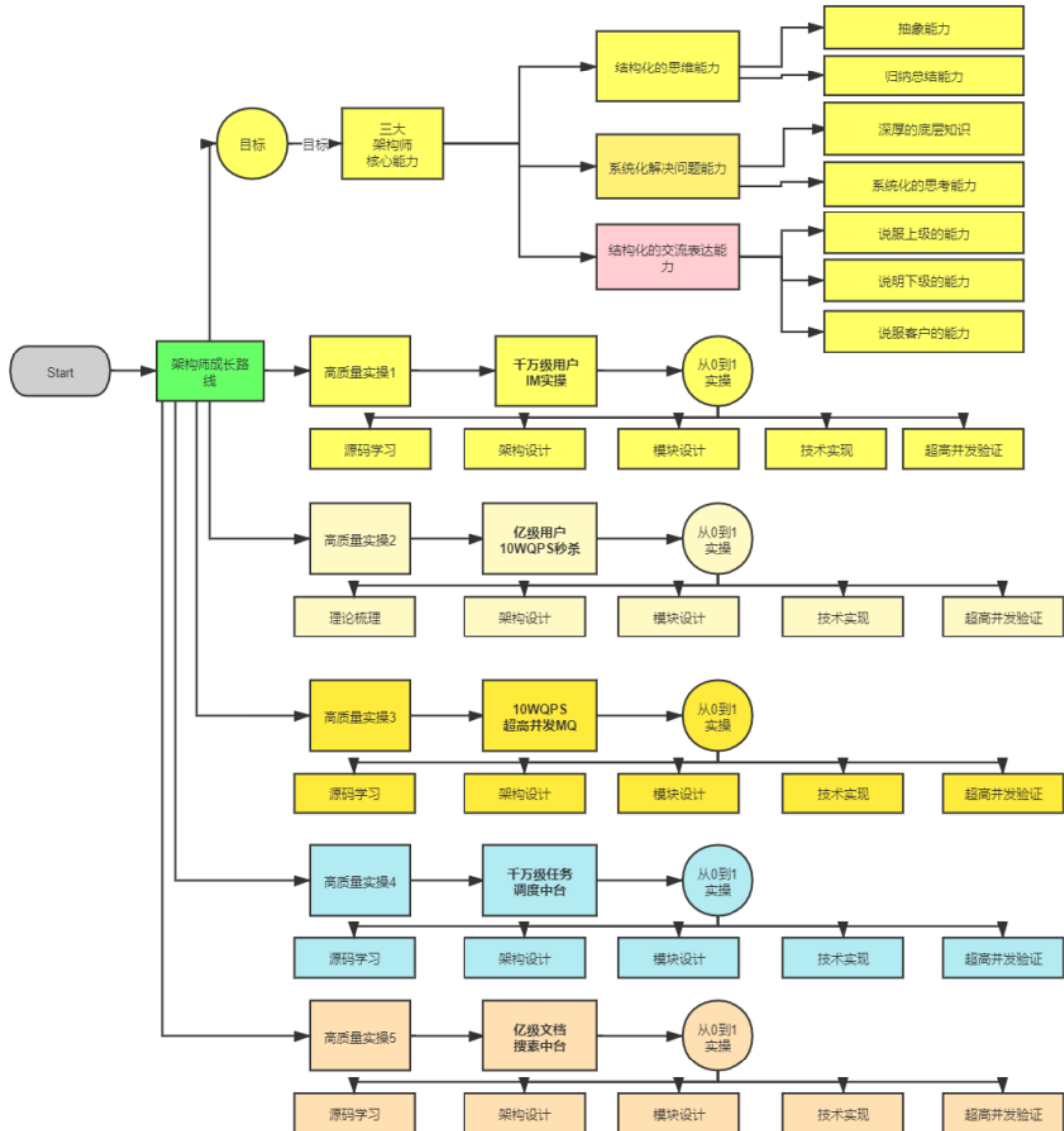
作为一个高质量的架构师成长、人脉社群，把所有的卷王聚焦起来，一起卷：

- 卷高并发实操
- 卷底层原理
- 卷架构理论、架构哲学
- 最终成为顶级架构师，实现人生理想，走向人生巅峰

架构班（社群 VIP）的目的：

- 高质量的实操，大大提升简历的含金量，吸引力，增强面试的召唤率
- 为大家提供九阳真经、葵花宝典，快速提升水平
- 进大厂、拿高薪
- 一路陪伴，提供助学视频和指导，辅导大家成为架构师
- 自学为主，和其他卷王一起，卷高并发实操，卷底层原理、卷大厂面试题，争取狠卷 3 月成高手，狠卷 3 年成为顶级架构师

N 个超高并发实操项目：简历压轴、个顶个精彩



【样章】第 17 章:横扫全网Rocketmq 视频第 2 部曲: 工业级 rocketmq 高可用(HA) 底层原理和实操

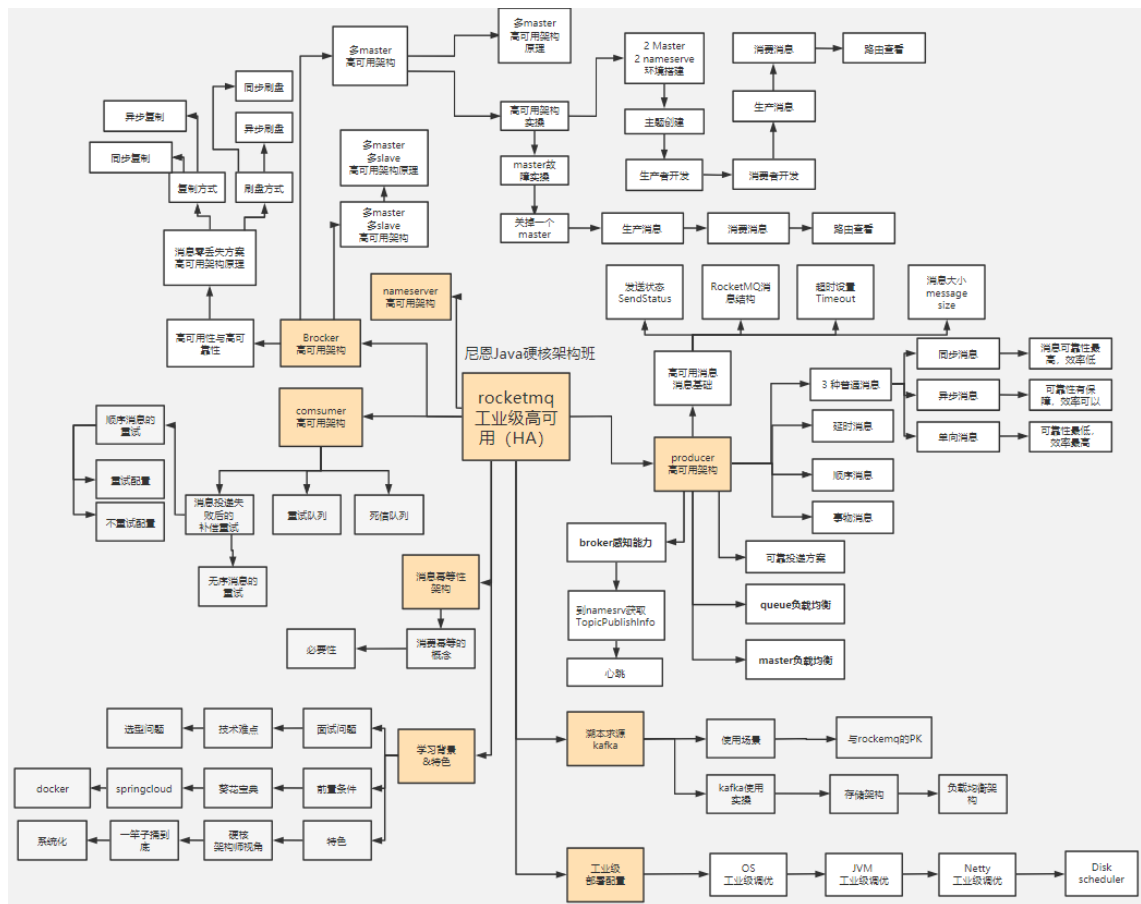
工业级 rocketmq 高可用底层原理, 包含: 消息消费、同步消息、异步消息、单向消息等不同消息的底层原理和源码实现; 消息队列非常底层的主从复制、高可用、同步刷盘、异步刷盘等底层原理。

工业级 rocketmq 高可用底层原理和搭建实操, 包含: 高可用集群的搭建。

解决以下难题:

- 1、技术难题: RocketMQ 如何最大限度的保证消息不丢失的呢? RocketMQ 消息如何做到高可靠投递?
- 2、技术难题: 基于消息的分布式事务, 核心原理不理解
- 3、选型难题: kafka or rocketmq , 该娶谁?

下图链接: <https://www.proceson.com/view/6178e8ae0e3e7416bde9da19>



成功案例：2 年翻 3 倍，35 岁卷王成功转型为架构师

详情: <http://topcoder.cloud/forum.php?mod=forumdisplay&fid=43&page=1>

[最新](#) [最后发表](#) [热门](#) [精华](#)

成功案例: [1057号卷王] 3年小伙拿到外企offer, 薪酬涨了200%

1 卷王1号 超级版主 前天 17:41

成功案例: [645号卷王] 4年经验卷王逆袭, 被毕业后, 反涨24W

1 卷王1号 超级版主 2022-9-21

成功案例: [878号卷王] 小伙8年经验, 年薪60W

1 卷王1号 超级版主 2022-8-13

年薪70W案例: 通过尼恩的指导, 小伙伴年薪从40W涨到70W

1 卷王1号 超级版主 2022-2-11

成功案例: [493号卷王] 5年小伙拿满意offer, 就业寒冬季逆涨30%

1 卷王1号 超级版主 前天 17:43

成功案例: [250号卷王] 就业极寒时代, 收offer 涨25%

1 卷王1号 超级版主 前天 17:38

成功案例: [612号卷王] 就业极寒时代, 从外包到自研

1 卷王1号 超级版主 前天 17:15

成功案例: [913号卷王] 热烈祝贺6年经验卷王, 年薪40W

1 卷王1号 超级版主 2022-9-21

成功案例: [959号卷王] 4年经验卷王, 喜获百度、Boss直聘等N个优质offer, 最高涨100%

1 卷王1号 超级版主 2022-9-21

成功案例: [529号卷王] 5年经验卷王喜收2大offer, 最高涨5K

1 卷王1号 超级版主 2022-9-21

成功案例: [811号卷王] 热烈祝贺7年经验卷王, 薪酬涨30%

1 卷王1号 超级版主 2022-9-21

成功案例: [287号卷王] 不惧大寒潮, 卷王逆市收4 offer, 涨30%, 可喜可贺

1 卷王1号 超级版主 2022-5-30

成功案例: [1002号卷王] 5月份“被毕业”, 改简历后, 斩获顶级央企Offer, 涨薪7000+

1 卷王1号 超级版主 2022-7-5

成功案例: [7号卷王] 热烈祝贺小伙伴涨薪120%

1 卷王1号 超级版主 2022-8-13

成功案例: [134号卷王] 大三小伙卷1年, 斩获顶级央企Offer, 成功逆袭

1 卷王1号 超级版主 2022-7-6

成功案例: [1008号卷王] 5年经验卷王收42W offer, 月涨8000, 可喜可贺

1 卷王1号 超级版主 2022-5-30

成功案例: [453号卷王] 非全日制 6年卷王喜提3 offer, 年薪30W, 可喜可贺

1 卷王1号 超级版主 2022-5-21

成功案例: [924号卷王] 6年卷王喜提4 offer, 最高涨薪9000, 可喜可贺

1 卷王1号 超级版主 2022-5-21

成功案例: [15号卷王] 4年卷王入职 微软, 涨薪50%, 可喜可贺

1 卷王1号 超级版主 2022-5-12

成功案例: [527号卷王] 4年卷王喜提2 offer, 涨薪50%, 可喜可贺

1 卷王1号 超级版主 2022-5-13

成功案例: [788号卷王] 3年卷王喜提优质Offer, 涨薪60%

1 卷王1号 超级版主 2022-5-11

成功案例: 热烈祝贺: 非全日制卷王, 喜提2个心仪offer, 面3家过2家

1 卷王1号 超级版主 2022-4-21

成功案例: [693号卷王] 二线城市6年卷王喜提4大优质Offer, 含央企offer, 最高薪酬35W

1 卷王1号 超级版主 2022-4-16

成功案例: [85号卷王] 双非2本小伙, 春招大捷, 喜提9个offer, 最高薪酬近30万

1 卷王1号 超级版主 2022-4-14

成功案例: [741号卷王] 卷王逆袭! 6年小伙从很少面试机会到搞定35K*14薪Offer

1 卷王1号 超级版主 2022-4-12

成功案例: [642号卷王] 热烈祝贺, 6年卷王喜提优质国企offer

1 卷王1号 超级版主 2022-4-7

成功案例: [796号卷王] 热烈祝贺, 36岁卷王喜提52万优质offer

1 卷王1号 超级版主 2022-3-25

成功案列: [15号卷王] 小伙卷1年, 涨薪9K+, 喜收ebay等多个优质offer

1 卷王1号 超级版主 2022-3-24

成功案列: [821号卷王] 小伙狠卷3个月, 喜提10多个offer

1 卷王1号 超级版主 2022-3-21

成功案列: [736号卷王] 3年半经验收22k offer, 但是小伙志存高远, 冲击25k+

1 卷王1号 超级版主 2022-3-20

成功案列: 热烈祝贺1群小卷王offer拿到手软, 甚至拒了阿里offer

1 卷王1号 超级版主 2022-3-16

简历案列: 简历一改, 腾讯的邀请就来了! 热烈祝贺, 小伙收到一大堆面试邀请

1 卷王1号 超级版主 2022-3-10

成功案列: 祝贺我国两大超级卷王, 一个过了阿里HR面, 一个过了阿里2面

1 卷王1号 超级版主 2022-3-10

成功案列: 小伙伴php转Java, 卷1.5年Java, 涨薪50%, 喜收多个优质offer

1 卷王1号 超级版主 2022-3-10

成功案列: 4年小伙狠卷半年, 拿到 移动、京东 两大顶级offer

 尼恩 超级版主 2022-3-5

成功案列: [267号卷王] 助力3年经验卷王, 拿到蜂巢的17k x 14薪的offer

1 卷王1号 超级版主 2022-2-27

成功案列: [143号卷王] 二本院校00后卷神, 毕业没到一年跳到字节, 年薪45W

1 卷王1号 超级版主 2022-2-27

成功案列: [494号卷王] 尼恩分布式事务助力卷王拿到 中信银行offer

1 卷王1号 超级版主 2022-2-27

成功案列: [76号卷王] 2线城市卷王, 狠卷1.5年, 喜收22K offer

1 卷王1号 超级版主 2022-2-27

成功案列: [429号卷王] 小伙伴在社群卷5个月, 涨8k+

1 卷王1号 超级版主 2022-2-27

成功案列: [154号卷王] 双非学校毕业卷王, 连拿 京东到家&滴滴 两个大厂 Offer

1 卷王1号 超级版主 2022-2-27

☐ 成功案例: [232号卷王] 涨薪10K, 继续卷向食物链顶端

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: 狠卷1年技术, 喜收 腾讯、阿里、微软三大Offer, 最高年薪56W

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: [449号卷王] 应届毕业生卷王喜收 滴滴offer, 年薪33W

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: [551号卷王] 小伙伴学完后, 成功进入大厂, 并且推荐自己的朋友加VIP学习

① 卷王1号 超级版主 2022-2-10

☐ 成功案例: [214号卷王] 助力2年经验卷王, 成功拿到17K月薪

① 卷王1号 超级版主 2022-2-10

☐ 成功案例: [92号卷王] 课程实操助力社群小伙伴喜收 喜马拉雅Offer

① 卷王1号 超级版主 2022-2-10

☐ 成功案例: 社群卷王小伙伴成功过了滴滴三面 获滴滴Offer

① 卷王1号 超级版主 2022-2-10

☐ [612号卷王]滴滴小伙伴, 蹲点考察半年, 觉得靠谱后加入 疯狂创客圈

① 卷王1号 超级版主 2022-2-10

☐ 成功案例: [732号卷王] 尼恩助力3年经验卷王收获 京东offer, 年薪35W

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: [558号卷王] 2年经验卷王, 喜收 网易和阿里子公司两个优质offer

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: [569号卷王] 双非应届生卷王, 喜收字节跳动实习offer

① 卷王1号 超级版主 2022-2-25

☐ 成功案例: [420号卷王] 狠卷1年, 卷王涨薪80%, 涨薪12000元!

① 卷王1号 超级版主 2022-2-25

☐ 成功案例: [76号卷王] 通过尼恩1年半的指导, 专科学历小伙伴从0.8K涨到22K

① 卷王1号 超级版主 2022-2-10

9年小伙伴拿到
年薪90W offer

9月11日改简历

11月29日晒offer

小伙8年经验
年薪60w

7月12日改简历

8月10日晒offer

6年小伙伴
年薪40w

9月6日改简历

9月21日晒offer

5年小伙喜提3个offer
年薪35个W

5月22日改简历

11月29日晒offer

1.5年小伙搞定15K offer

就业寒冬涨100%

5月7日改简历 11月21日晒offer

秘诀:
简历指导+ 狠狠卷

他那个不止再修修嘛，我才是两位，原先7k，现在15k

那也很牛

都翻倍了，都是牛人

正常就30%

好的

晚上指导你改

推送中台需要加不

我还没做完，准备八脱文新时没时间来搞这些，入职了会开始搞

OK

还有别的项目吗

一个项目，太少啦

那种练习项目也行

其实我工作一年，

下一步可以瞄准25k

一年经验，搞定15k offer，舍你其谁

那你就更牛逼啦

卷王逆袭成功案例

6年小伙从很少面试机会到搞定35K*14薪

3月5日改简历 4月11日拿offer

一个月拿到了理想的offer

面试邀请少 小伙很苦恼

理想很丰满 目标35K

面试法宝 rocketmq四部曲

6年 经验小伙伴

喜收25K offer

3月12日改简历 12月1日晒offer

秘诀:
简历指导+ 狠狠卷

咱们以这个项目为模板改哈

项目有多少人，你带领多少人?

你工作几年了?

6年了

改简历那晚，20涨到25k，高级开发，P7带的后端团队

任务重，道路远，并不是没有方向

7年经验卷王

薪酬涨30%

7月11日改简历 9月1日晒offer

秘诀:
改简历+ 狠狠卷

只要本事好，拿offer比较容易的

入职了，最终并不输于面试官，还差旧的大牛，只涨了30%

恭喜一下

现在不比往年

能涨30%是大牛

拿到offer都算大牛

现在很多人投几百发，连个电话都没有

你已经很牛啦

4年经验卷王逆袭 被毕业后，反涨24W

7月改简历 8月30日晒offer

秘诀:
改简历 + 狠卷

这就是你的简历
差得太多啦
老哥 我八月十号开始找工作，今天已入职了
能涨24W
原因大概是啥?
很多小伙伴，面试机会都没有
这个地方的要这样写
项目被终止
方便语音沟通不

尼恩 你这么指导，非常重要
老哥 我八月十号开始找工作，今天已入职了
现金基本持平，股票 +24W
总计涨了多少钱
股票这个吧只能到手了才算
也不错啦
继续跟着你学技术

小伙5月份"被毕业"，改简历后 斩获顶级央企Offer 涨薪7000+

5月29日改简历 7月5日晒offer

秘诀:
简历指导 + 狠卷3高

快速看书，就能不求甚解，把自读和场景大概一下，然后重点的地方，用到的地方，再去回炉
我被"毕业"了
这周末或下周找你改一下简历
毕业没有关系
ok，发我吧
R行业，就来说去，太复杂啦
嗯，其实有点心理准备
不太会写简历

尼恩 我拿到手写的offer了
涨20%，涨薪更多。结果人家都
看起来里面不差钱呀
超过了8000块
平均算下来
7000多
好的
有啥面试的心得吗
可以分享给其他小伙伴的

卷王逆袭成功案例 武汉6年喜收4个优质offer 最高的年薪35W

2月9日改简历 4月15日晒offer

面试法宝:
改简历 + 实操

尼恩老师，新年好!
能帮忙修改下简历吗?
金三银四准备啦
java开发-6年-简历-
拜托了，尼恩，希望能拿25K回来给你报喜
好，我加一下
还有吗?

尼恩，决赛圈offer了
截图是我自认能写出来的评分了。麻烦帮我参考下
选择大于努力，尼恩助我上岸
这么多offer，我看看哈
都是尼恩指点有方，本来还有个新能源汽车的，35W给拒了，主要太远了
跟着尼恩的时间太短了，目前实力也只能到这了
这里边有个大数据的，感觉也不错

卷王逆袭成功案例 6年小伙喜提4个Offer 最高涨9k，年薪35W

4月14日改简历 5月17日晒offer

涨薪法宝:
改简历 + 狠卷

Java开发工程师，
你看着我给你的
好的呀
谢谢大佬
麻烦大佬了
这个你自己刷
不对的，你帮自己刷
那我照着这个改一下库存系统呀
一个简历，
这么漂亮的简历，涨50%，已经没啥问题
只要准备好，不出大错，基本没问题啦

尼恩 保证押金收起来，你的offer最高涨了9k，多返现100
后面继续跟着哈，感觉卷的时间
感觉自己学的不太透彻了
嗯哪
跟着大佬一起
我周围好几个年薪百万的，都是这

卷王逆袭成功案例

5年经验小伙收2个offer
最高涨薪8k，年薪42W

5月9日改简历 5月30日晒offer

秘诀：
简历指导+ 狠卷3高

以此为例
大家狠狼卷
打造最卷IT社群

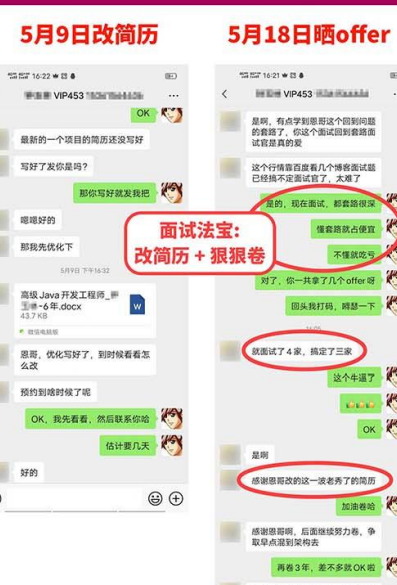


卷王逆袭成功案例

非全日制 6年经验卷王
喜提3个Offer，年包30W

5月9日改简历 5月18日晒offer

面试法宝：
改简历+ 狠狼卷

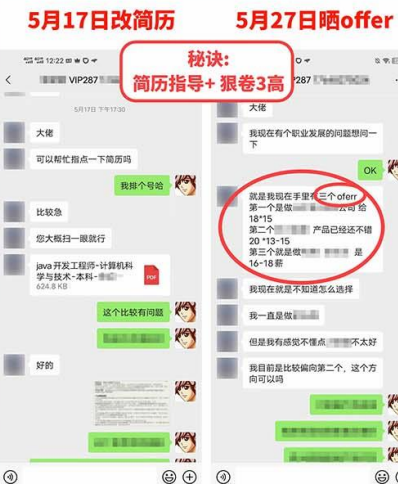


卷王逆袭成功案例

寒五冻六之际卷王大逆袭
收3大offer，涨30%

5月17日改简历 5月27日晒offer

秘诀：
简历指导+ 狠卷3高



卷王逆袭成功案例

4年卷王入职微软，涨50%

3月7日改简历 5月12日晒offer

涨薪法宝：
改简历+ 狠狼卷



4年小伙喜收百度、Boss直聘
等N个顶级Offer
最高涨幅100%

6月27日改简历

9月19日晒offer

秘诀:
改简历+狠狠卷

boss直聘和小满之间, boss那
边给的offer, 工资比小满高, 但是
小满那边给的offer, 工资比boss那
边高, 所以我就在boss那边和boss
那边的人聊, 让他们知道我在boss那
边, 然后他们就给我发了offer, 我
不知道怎么办, 如果是我应该怎么
选呢?

还有其他的offer, 还有一个养
育的。

总体上是想着boss和小满之间
选一个

了

boss比小满高包多7w以上。

我这涨幅都快百分之百了

大牛说

长那个那个思路结合上次指导的
内容改了几个点, 发现确实高大了
很多, 加上一张指标的说明确实会
让人眼前一亮的感觉。

明天睡醒时候有空了再和我看下
下, 一是有两个点我实在不知道怎么
改了, 二是在排版表达这方面确实
不行, 还得让你帮忙把关。

卷王逆袭成功案例

4年卷王入收2个offer, 涨50%

3月23日改简历

5月12日晒offer

offer决策
offer决策: n+1 电商erp, 部门成
熟, 人数多, 加班少
offer决策: n+2 做业务中心 内部系
统, 新部门, 人数多, 加班多

又搞到个offer

能搞定两个offer, 不尴尬

现在很多小伙伴面试机会都没有

涨了没呀

地点在哪里

深圳

涨50%的样子

忘记你工作几年啦, 大概工作几年
呢?

涨薪法宝:
改简历+狠狠卷

这个不用加嘛

都是一样, 加粗了反而没有效果

小伙大三暑期很焦虑
跟着尼恩卷一年
校招斩获顶级央企Offer

去年5月19日加入VIP群

今年7月5日晒offer

秘诀:
狠狠卷书+视频

尼恩老师

我校招去华润电力控股有限公司了

跟着你卷了一年 大学顺便拿了几个
国奖

不错不错, 这是央企

放空啦

这太牛啦

跟着你卷了一年 大学顺便拿了几个
国奖

其实拿到手的也就一个a美国一

邀请你加入社群

“尼恩老师”邀请你加入社群
尼恩老师邀请你加入社群
尼恩老师邀请你加入社群

2021年5月19日 晚上20:04

尼恩老师 大三的暑期实习找不到
现在准备秋招应该没关系吧

看身边 总是很焦虑 自己算法这一
块卡住了

网盘里边有算法视频

去听一听吧

秋招来得及

谢谢大佬 不过经过几个月练习 看
你写的书比之前轻松多了

趁着还是学生这段时间 慢慢把知识
吃透

嘿嘿, 我的书, 比较深

期待大佬的下一本书, 已经迫不及待去
学习了

小伙高中学历
薪酬涨120%

5月6日改简历

7月22日晒offer

你这块估计要涨668

¥668.00

哈哈, 不用了大佬, 你帮我涨了
就行 光今天晚上辅导就值几千了

老板很感谢

还有很多其他的模块

面试官要问

就是其他人做的

嘿嘿, 好

之前你的工资是多少的

翻了一翻

我涨你多少涨上来的

不知道, 原价给老板就行了

就是这工资

咱们得说话算话呀

老板拿着您的那个简历开发改进了
点, 所向披靡。

ok

从头到尾给面试官讲的清清楚楚

后面继续狠狠卷哈

秘诀:
改简历+狠狠卷

5年卷王喜收2大Offer
最高涨5K

5月19日改简历 9月13日晒offer

秘訣:
改简历 + 狠简历

发我吧

辛苦了!

OK, 简历还需好好改改哈

我主要介绍思路, 方法

嘿嘿, 我先消化一下。有问题请在
教教

OK

12:25

崽哥, 我就搞成功了。目前手上有
两家相对心仪的公司。崽哥, 有空
你帮我参考下呗

恭喜恭喜!

一家是字节, 一家是小米云相关的
的, 给了23K, 一家是自研, 做数
据库的, 给22K

自研的是啥公司呢

薪资涨幅都不多, 涨幅都不高,
都没达到你涨幅标准

继续

好的, 稍等哈

标题: 简历 202205.docx
4.8 KB

微信电脑端

辛苦崽哥了!

目前薪资是18K, 目标想冲击到
24K

OK, 我晚点看看回复哈

好的哈

5月21日 晚上20:24

崽哥, 今天有空看一看我的简历
嘛? 帮稍微优化优化

后面跟着我在讯国一溜

[illegible]

修改简历找尼恩（资深简历优化专家）

- 如果面试表达不好，尼恩会提供 简历优化指导
- 如果项目没有亮点，尼恩会提供 项目亮点指导
- 如果面试表达不好，尼恩会提供 面试表达指导

作为 40 岁老架构师，尼恩长期承担技术面试官的角色：

- 从业以来，“阅历”无数，对简历有着点石成金、改头换面、脱胎换骨的指导能力。
- 尼恩指导过刚刚就业的小白，也指导过 P8 级的老专家，都指导他们上岸。

如何联系尼恩。尼恩微信，请参考下面的地址：

语雀：<https://www.yuque.com/crazymakercircle/gkkw8s/khigna>

码云：<https://gitee.com/crazymaker/SimpleCrayIM/blob/master/疯狂创客圈总目录.md>